OXSTEED

Enterprise operations audit

Version 2 - rebuilt with actionable operator design

Operating target: 3,500 worldwide employees and six million registered users

Source baseline: e47531c76b320c8145e15415774d05d698b456d1

Final refresh: 2f93a4264d604d1275e82ecbcf28faa72e52b599

Evidence refreshed October 4, 2026, 03:47 UTC

This edition replaces generic per-tab boilerplate with distinct findings, screen specifications and testable closure criteria. Availability is observed; enterprise readiness is not certified.

Contents

Version 2 - rebuilt with actionable operator design 1

Contents 2

Executive decision and scope 5

Priority decisions and implementation sequence 7

Operator design contract used in every review 9

Additional business capabilities and why navigation alone is insufficient 10

Completion and certification gates 11

1. Dashboard 12

2. Operations Overview 14

3. Users 16

4. Jobs 18

5. Reports 20

6. Moderation 22

7. Enforcement 24

8. Content Removals 26

9. Skills 28

10. Support Tickets 30

11. Live Support 32

12. Quote Requests 34

13. Direct Ops 36

14. Direct Service Areas 38

15. Direct Catalog 40

16. Messages 42

17. Search 44

18. Businesses 46

19. Tool Rentals 48

20. Planned Needs 50

21. Payments 52

22. Orders 54

23. Templates 56

24. Consulting 58

25. Consulting Pay 60

26. Scheduling 62

27. Business Launch 64

28. Consulting Sessions 66

29. Org Chart 68

30. Corporate Titles 70

31. Logistics Ops 72

32. Learn Submissions 74

33. Learn Product Links 76

34. Learn AI Suggestions 77

35. Super Dashboard 78

36. Money 80

37. Restaurant Credit 82

38. OxSteed Credit, overview and all thirteen subpages 84

39. Revenue 87

40. Fee Config 89

41. Pricing Config 91

42. Pricing Overrides 93

43. Financials 95

44. Payouts 97

45. Conversations 99

46. Admin Accounts 100

47. Permissions 101

48. Regions 102

49. Internal Access 103

50. Gate Lockouts 104

51. Create User 105

52. Search Logs 106

53. Settings 107

54. Audit Log 109

55. User Action Log 111

56. Legal: acceptances, legal audit, compliance metrics, privacy requests 113

57. Visitors 115

58. Profile Visibility 117

59. One Truth 119

60. Franchise HQ 121

61. Careers 123

62. Country Packs 125

63. Corporate Setup 126

64. Mode Policies 127

65. Mode Approvals 128

Additional review: Restaurant Top-up Approvals 129

Workforce Identity and Access Lifecycle workspace 131

Treasury and Entity Accounting workspace 132

Global Service Operations workspace 133

Reliability and Release Operations workspace 134

Governance, Policy and Contract Obligations workspace 135

Proposed workspace 1: Workforce Recertification and Continuity 136

Proposed workspace 2: Financial Reconciliation Exceptions 137

Proposed workspace 3: Country Launch Readiness and Obligations 138

Proposed workspace 4: Recovery and Release Certification 139

Proposed workspace 5: Cross-product Incident and Customer Impact 140

Current-source companion review: Provider Health 141

Current-source companion review: Media Safety and Processing Operations 143

Source coverage appendix 145

Executive decision and scope

The operating target is 3,500 employees worldwide and six million registered users. The application is available at the observed time, but enterprise readiness is not established. This report is an engineering and operating-model assessment, not a promise that no future defects can occur. The practical release standard is no known critical integrity/privacy/control defects, demonstrated recovery for foreseeable failures, and accountable monitoring for new risks.

The earlier Version 2 was rejected for good reason: its per-tab pages repeated generic acceptance language, and its operator-design paragraphs were not implementation specifications. This rebuild uses distinct source-backed reviews. Each tab has a business purpose, observed implementation, classified findings, expansion scenario, an operator screen specification, and acceptance criteria. Design recommendations specify actual columns, filters, drawers, decision previews, conflict handling, and staff next actions. Proposed controls are requirements, not a claim that the code already implements them.

Source baseline: repository oxsteed/old-file, branch old-files-claude, revision e47531c76b320c8145e15415774d05d698b456d1, inspected October 3, 2026. Final refresh inspected the intervening diff through 2f93a4264d604d1275e82ecbcf28faa72e52b599 and current CI/readiness at October 4, 03:47 UTC. This is a bounded refresh, not a claim that every new feature added after the baseline received a complete authenticated audit. Most per-tab file/line citations remain anchored to the baseline; the newly completed reviews use unchanged controller source and identify refreshed UI observations. New franchise, ownership/entity-ID and identity-verification features require their own expanded action matrix. Current navigation retains 71 primary entries: 34 regular and 37 super-admin entries, before grant/mode filtering. The original 65 review areas are retained, alongside the five enterprise workspaces now in source and Restaurant Top-up Approvals. Provider Health is an additional Operations drilldown. Detail and sub-workspace routes are covered within their parent review; route presence is not a claim of action-level testing.

Final refresh and findings superseded during this rebuild

The current CI run 37174568453 at 2f93a426 completed successfully: 365 server suites and 3,959 tests passed; three suites and 46 tests were skipped. The active-job follow-up suite passed its three tests. CI now installs FFmpeg before the media regressions; the earlier runner-missing-FFmpeg defect is closed, not a present release blocker. The fresh PostgreSQL/PostGIS migration job applied the migration set twice successfully, including migrations 667-671. A clean-database rehearsal does not replace a populated production-like upgrade or restore test. The latest run remains distinct from authenticated browser-matrix execution; no skipped matrix is credited as coverage.

Public /ready at 03:47:09 UTC returned ready, release content-a627ecf76102a039, 666 expected/applied migration files and zero pending/unexpected files. Required health checks were positive and the Credit rail was explicitly disabled. Git SHA, image digest and deployment ID remain null. Therefore the current public response alone cannot identify the running commit; do not claim it proves 2f93a426 is deployed. Readiness counts are migration-file counts, not the highest numeric migration name. The earlier 661/661 observation below is historical and superseded as a current count.

The intervening changes add typeahead/inline-feedback and navigation improvements to many public/admin pages, restaurant training/audit/reporting and offline components, entity ownership identifiers and entity-linked verification. These improvements are acknowledged rather than reissued as absent features. Typeahead suggestions are generally over already loaded records; they do not automatically close API pagination, truthful totals, authority scope or concurrent decision gaps. Restaurant menu hook-order and Learn initialization crash fixes are also present in the final revision. Source presence and passing unit CI do not certify their authenticated operating journeys.

Earlier baseline evidence, with current supersessions above

- Public /ready at 22:30:15 UTC returned ready, release content-522f39e3a5d1b20e, 661 expected/applied migration files, zero pending/unexpected files. Database, Redis, configuration, admin control plane, realtime initialization, worker initialization, traffic acceptance and shutdown checks reported healthy. The Credit rail was explicitly disabled, not certified live.

- Read-only Docker inspection found the running application image tagged with the audited e47531c7 revision and healthy application, PostgreSQL/PostGIS, Redis, ClamAV, Coolify and proxy containers. These services share the local Colima physical-host failure domain. A content release string and image tag are useful observations, but do not substitute for an immutable digest and a promotion record.
- Baseline GitHub CI run 37147612778 failed: 359 suites passed, one failed, three skipped; 3,910 tests passed, three failed, 46 skipped. Its media runner dependency failure is now superseded by the final successful run described above. Retain this as chronology, not an unresolved current CI issue.
- Earlier run 37110020289 at 893f1bc3 passed 353 server suites and 3,716 tests, and a twice-applied clean PostGIS rehearsal. That evidence proves that earlier revision only. Its authenticated browser matrix explicitly skipped both tests. A green step with skipped tests is not successful authenticated coverage.
- The latest static route check found 138 route definitions but exited with one violation: it does not recognize requireSuperAdminOrPermission('manage_financials'). The top-up router actually uses that gate; the middleware delegates to requirePermission for non-super administrators. This is a static-check coverage defect, not evidence of an unguarded top-up API. Fix and test the checker, and separately test direct endpoint authorization.
- The latest local npm run verify:i18n passed: all 20,905 static translation references resolved, five dynamic keys were skipped, interpolation/array/brand parity passed and the checker found zero still-English values across namespaces. Earlier missing-key and English-copy debt was repaired at 25790585; do not repeat the rejected report's 324 unresolved/703 still-English figures as current. Static parity does not prove that translated copy is natural or that every Arabic screen is usable.
- Read-only Coolify backup collection reported the October 1, 2 and 3 daily executions as successful with retained local copies and confirmed S3-upload metadata for the oxsteed-backups R2 destination. October 3 started at 06:00:03 UTC. That proves execution metadata, not an independent current object checksum, successful application recovery or a measured RTO. The September 30 older execution with unconfirmed upload remains distinct.
- A read-only query of the live backup evidence table returned no restoration_test entries. This proves no such restore evidence is recorded there, not that nobody ever ran a restore elsewhere. The monitor distinguishes backup health from verified recovery; preserve that distinction in release decisions.
- At the baseline, GitHub branch metadata reported protected: false for old-files-claude, and the running image corresponded to a failed-CI revision. The final CI is green; a protected, artifact-bound promotion gate still requires evidence. Coolify availability and native automatic deployment do not alone establish that gate.

What is not established

No current evidence certifies all privileged mutations through real staff accounts, all record-level denials, every provider callback, an independent complete restore of the latest snapshot, production-sized upgrades, global latency, or six-million-user capacity. The audit does not execute production refunds, approvals, account suspensions, permissions, media publication or high-volume load. A missing evidence record is labelled UNVERIFIED, not converted into an invented defect or a passing certification.

Priority decisions and implementation sequence

Historical findings that are closed or substantially improved

The Orders transaction-helper calls now supply the database pool (server/controllers/adminOrdersController.js:229, :339). Quote updates preserve omitted currency instead of silently replacing it with USD (server/controllers/quoteRequestsController.js:645-659). Support ticket claim/unclaim uses conditional eligibility updates instead of an unconditional competing ownership write (server/controllers/supportController.js:592-616, :651-658). Planned Needs pause is durable and scheduling rereads eligibility under lock; its tab review identifies remaining content/visibility issues separately. Those old reproductions must not be assigned as unchanged defects.

The realtime Redis adapter, process-state registry, shared revocation/presence infrastructure, worker fencing and database deadlines are substantial current repairs. Audit residual call-site coverage and failure semantics rather than recommending that everything be built from scratch. The uniform provider registry and scale harness also now exist; operational measurements and caller adoption remain separate requirements. The client localization/type/production-build step passed at the latest audited revision even though the server media suite failed. A precise report credits repairs while preserving the remaining gates.

Gate A - dependable release and recovery

Release Engineering has closed the CI runner's missing-FFmpeg issue in the final refresh. The remaining media release requirement is a supported, verified production worker toolchain and an executed safety/processing journey. Verify ffmpeg and ffprobe capabilities rather than assuming /usr/bin on every operating system. Keep dependency-independent unit tests separate from genuine transcode integration tests, but make the media integration job mandatory when that product ships. Prove a real video produces the expected renditions and remains quarantined on failure. A passing runner suite does not establish the deployed worker's capabilities.

SRE owns promotion control. Generate a release record binding commit, artifact digest, build/check results, migration plan, approving operator, environment and rollout timestamps. Protect the production branch, require passing checks and review, then let Coolify deploy the approved artifact through its existing native flow. Keep an emergency path with reason, expiry and post-incident review. /ready currently has null git SHA, digest and deployment ID; populate and verify them rather than inferring deployed identity from page content. Acceptance includes a rejected red revision, a verified healthy promotion and a tested rollback that does not reverse committed domain history.

Continuity Engineering owns complete restoration. Daily R2 dump metadata is a meaningful improvement; the next gate is downloading a specific confirmed object through the private operator workflow, validating integrity/provenance and restoring into an independent disposable environment. Verify database records, financial invariants, media/uploads, configuration dependencies, background work and a representative authenticated journey with provider test endpoints. Never create test charges. The only registered restore profile is legacy-exact-v1 (server/services/restoreVerificationProfiles.js:3-5), which checks fixed catalog counts and the exact checked-out migration set. Create versioned semantic recovery profiles so an older valid backup can restore and then upgrade; a changed catalog count is not automatically corrupt data. Measure RPO from the last recoverable committed snapshot and RTO until usable service, not until pg_restore exits.

Gate B - trustworthy operator decisions

Fix tab-specific confirmed defects before adding more dashboards. Important observed themes in the accompanying reviews include ordinary-dashboard response/render mismatches, swallowed evidence failures, incomplete administrative-hold ownership, query scope ambiguity, stale dependency previews and unavailable values rendered as zero. Each finding needs a reproduction fixture, owning team, migration/API/UI plan and proof that the original journey remains correct. Existing authoritative payment, credit, inventory, support and dispatch services remain the source of truth.

Adopt a shared command envelope only where the domain requires it: command ID, actor and authority, expected record version, explicit intended transition, reason/evidence, domain transaction result and

reconciliation status. A timeout must lead to a command-status read, not a second unqualified money operation. A pending external outcome must remain pending. Link records by explicit IDs; matching a name, email or text is not an authorization or relationship rule.

Identity Operations owns global staff scope. Existing middleware deliberately rejects region-only grants on platform-wide endpoints. Preserve that boundary until each list, detail, export, action and socket route has an authoritative scope predicate. A country picker cannot turn global data into regional authorization. Use role bundles and delegated approval limits with effective dates, offboarding and recertification. Test one region's employee against another region's IDs, exports and subscriptions; prove revoked access stops ongoing sessions without deleting evidence.

Gate C - measured expansion, not a bigger server guess

The scale harness now exists (scripts/scale/README.md:3-21); it is no longer accurate to say no tooling exists. Its own README says fixture materialization and live runs remain operator steps. It models a workload slice rather than six million concurrent users. Separate registered population, active sessions, requests per second, realtime connections, message fanout, database size and tenant skew. Use measured ratios before purchasing capacity.

The scenario catalogue intentionally accepts some 400/404/422/429 outcomes and synthetic IDs (scripts/scale/scenarios.cjs:117-121, :133, :157, :179). Those checks exercise handling paths, not successful checkout, payment settlement, delivery proof or a real fanout conversation. In this audit the harness's 22 local regression tests passed against fixtures/stub servers, not production. Its missing-auth test printed zero requests, 8,584 skipped steps and an SLO verdict of PASS. That is an execution-coverage gap in a release interpretation: a benchmark with no executed required steps must be INSUFFICIENT, regardless of latency calculation. Add a required-scenario coverage gate, seeded valid journeys with semantic assertions, and separate business failures from intended rejection/rate limits. Test at least one skewed large merchant and one regional outage. Load must run in an isolated environment, not against this Mac's production resources.

Database deadlines and workload budgeting now exist; the old claim of no statement timeout is stale. Validate deadlines, connection budgets, indexes, reporting snapshots, queue fairness and asynchronous exports under realistic competing workloads. More HTTP replicas alone will not distribute arbitrary process-local state or make a single database/host highly available. The process-state registry and Redis fencing are current improvements; prove their authoritative writes and lease-loss behavior with two processes and dependency failures before scaling out.

Operator design contract used in every review

This is a reusable interaction standard, not a substitute for the tab-specific specifications that follow. A designer and developer should use it to make the company's controls coherent while retaining domain differences.

Orientation and evidence. A screen starts with its business task, effective authorized scope, current filter/window and last successful source time. Explain financial KPI units and whether a count covers the complete predicate or just the visible page. Distinguish missing configuration, denied access, empty data, degraded dependency and stale evidence. “Zero” is only valid when a source actually measured zero.

Investigation before mutation. The queue leads to a keyboard-operable record view. Show authoritative state, linked obligations, event history, required evidence, assigned owner and next action. The primary button is named for the permitted transition, not a generic “Update.” A filter or saved view carries only allowed query state; it must not retain private customer payloads in browser storage. Back navigation preserves query and selection without retaining stale approval authority.

Decision preview. Confirm consequential actions with exact target(s), current version, amount/currency where relevant, downstream obligations, reason and approval requirement. Money, access, enforcement and schedule commitments are never incidental side effects of bulk selection. Allow only explicitly safe bulk operations; show the exact ID set and per-item outcomes. Delegated staff see why an action is unavailable and the correct escalation path, not a button that predictably returns forbidden.

Conflict and failure. A 409 shows the newly committed state beside the operator's draft and forces a new review. Unknown outcome shows a command/reference ID and status lookup. A dependency outage preserves the last sample with a timestamp; unsafe decisions disable while investigation remains possible. Retrying an unrelated panel must not clear an unsaved decision. On reconnect reload authoritative permitted state; socket events are invalidation signals, not proof that every change was received.

Accessible operational use. On narrow screens place urgent work before decorative charts, provide a full-screen detail workflow and preserve filter controls. Label state with text as well as color; restore focus after dialogs and announce material errors. Use logical spacing for RTL and bidirectional isolation for IDs/emails/amounts. Localize dates, currency, units, timezones, validation and notifications in en/es/fr/ht/ar. Reduced motion keeps a complete static interface.

Additional business capabilities and why navigation alone is insufficient

The five enterprise workspace reviews cover Workforce, Treasury, Service Operations, Reliability and Governance. These routes now exist; calling them missing would be misleading. They must earn readiness by completing accountable end-to-end workflows. Five additional substantive proposed-workspace chapters specify the missing coordinated operating capabilities: workforce recertification/continuity, financial reconciliation exceptions, country launch obligations, recovery/release certification and cross-product incident impact. Existing partial services are acknowledged in each proposal. Treat these as required views/workflows over authoritative services, not five new competing databases or an instruction to create redundant navigation before validating the existing modules.

The missing capability backlog should also be assessed for: supplier/vendor obligations and procurement authority; incident/on-call and customer communication; country-specific tax and legal launch evidence; enterprise fraud and abuse investigations; media safety/quarantine recovery; partner disputes and SLA obligations; asynchronous scoped exports; executive performance definitions; and workforce continuity when a key operator leaves. Check the existing Legal, Logistics, Support, Credit and Corporate modules first. A separate tab is justified only when an identifiable employee role needs a distinct queue, decision authority and operating cadence not met by those modules.

Completion and certification gates

1. Assign every confirmed finding to a team and severity; record reproducer, affected domain, migration compatibility, implementation links, acceptance tests and closure evidence. Do not close an issue because its button now exists.
2. Run mandatory seeded authenticated tests: super administrator, delegated global staff, regional staff, revoked account and read-only staff. Verify success, denial, stale record, competing decision, provider timeout, unknown outcome and recovery. Include all primary tabs and dynamic detail routes. Keep testing identities and financial test mode out of production.
3. Validate each domain journey from intake to terminal evidence: jobs/quotes/booking/payment/support; food/order/credit/receipt/refund; delivery/internal-or-marketplace/proof/extra/support; staff grant/delegation/expiry/offboarding; settings proposal/approval/application/rollback.
4. Apply additive migrations twice on fresh PostGIS and rehearse an upgrade on a representative populated restored snapshot. Verify lock time and compatibility with old workers during deployment. Preserve committed ledger and audit evidence.
5. Execute measured load, soak, failure, mass reconnect and recovery drills. Document approved latency/error budgets, zero-duplicate-money/inventory invariants, alert ownership and a recovery procedure a second employee can follow.
6. Inspect rendered operator screens in five locales, RTL, mobile, both themes, keyboard and screen-reader workflows. Static translation success is a gate, not the full inspection.
7. Record a release decision with unresolved limitations explicitly accepted by the accountable business owner. Enterprise readiness remains unapproved while critical proof, ownership, privacy, monetary, idempotency, scope or recovery tests fail.

1. Dashboard

Route and business purpose

/admin/dashboard is the shift-start summary for ordinary operations staff and also renders the richer dashboard for super administrators. It should answer whether marketplace work and customer problems are moving normally, then take staff directly to the workload requiring intervention. At the target size, a trustworthy definition of each number matters more than the number of attractive cards: a seven-day intake count must not masquerade as a selectable thirty-day performance metric.

Verified current implementation

The ordinary route requires manage_operations (server/routes/admin.js:66). The shared component chooses /admin/dashboard for regular admins and /admin/super/dashboard?period=… for super admins (client/src/admin/pages/super/Dashboard.jsx:150-154). Ordinary output contains six operational fields, including new_users_7d and new_jobs_7d (server/controllers/adminController.js:24-42). The super path counts both completions and cancellations over the same creation cohort; the historical mixed-lifetime completion-rate defect is fixed (server/controllers/superAdminController.js:239-240, :357-363). Auxiliary activity, geography, monitoring, and incidents load independently with settled promises (client/src/admin/pages/super/Dashboard.jsx:159-188). Incident resolution now collects a note (:211-219). Relevant schema includes the audit infrastructure in server/migrations/005_admin.sql:11-30; shared permission tests explicitly reject regional grants (server/__tests__/adminPermissionScope.test.js:23-40).

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: ordinary-admin metric contract does not match rendered cards.** The renderer expects new_users_period, new_jobs_period, and open_support_tickets (client/src/admin/pages/super/Dashboard.jsx:362, :391, :408), but the ordinary controller returns the seven-day fields and no ticket count. Values become unavailable despite real data. The period selector also refetches an endpoint with no period parameter. This is distinct from the fixed super completion calculation.
2. **DESIGN GAP, P1: optional-query failure can look like zero activity.** Super statistics catch failed queries into empty objects or initialized zeros (server/controllers/superAdminController.js:199, :212-224). The component receives no per-metric failed-source flag. A partial database issue can therefore suppress a staffing or fraud signal while the overall page succeeds.
3. **DESIGN GAP, P2: summary freshness is not panel freshness.** lastUpdated changes when statistics arrive, while auxiliaries are fetched on the super-role dependency and manual refresh (client/src/admin/pages/super/Dashboard.jsx:157, :188, :227-229). An operator can read fresh totals beside older incidents or activity. The eight-row geography display (:111) is a ranking sample, not global market coverage.

Expansion failure scenario

A regional incident produces thousands of tickets. Ordinary operations staff see unavailable ticket/new-user cards; a super administrator sees a freshly updated main summary alongside old incident information. A failing optional count becomes zero. Staffing is directed to the wrong queue even though authentication works correctly. Operations Overview, Support Tickets, and Incident Management must share definitions and source freshness, rather than each producing a different understanding of the incident.

Required changes and operator design

Normalize the ordinary response to a documented operational metric contract, or render a separate six-field ordinary view. Return metric-level value, window, cohort, asOf, and availability, with explicit failure codes. Keep money cards on the existing super path. Use aggregate snapshots for repeated counts and preserve links to authoritative work queues. Completion percentage must be labeled as the share of terminal jobs in the selected creation cohort, unless product deliberately adopts a completion-event cohort.

Operator screen design recommendations

Replace the regular-admin view's unsupported cards with a header reading “Marketplace operations,” its effective global scope, selected interval, and independent statistics/incident refresh times. Restrict interval choices to those actually supported by the endpoint. Four primary cards should show published jobs, in-progress jobs, pending reports, and active support cases; define active cases explicitly, including waiting-user cases if they remain company obligations. Put cohort intake and trend cards below, with accessible definition popovers.

Add a compact “Needs intervention” list beneath the cards: workload type, current count, oldest outstanding item, deterioration since previous snapshot, responsible queue, and a link carrying the matching filter. A support count should open the relevant active-state filter rather than an unfiltered inbox. Saved views should store metric window and chosen operational domains; avoid saving customer details in browser state.

Selecting a card opens a side drawer containing its definition, source, query interval, last successful sample, trend, and underlying queue link. Incident rows may acknowledge or open the existing resolution-note dialog only where authority permits; ordinary KPI rows have no destructive bulk actions. Add “Create incident from signal” as a separately authorized, previewed handoff with metric evidence attached.

On partial failure retain the last sample with a stale badge and source-specific retry; never draw zero as healthy. Empty means zero measured work in the displayed scope. No access names the missing domain permission without exposing its count. On mobile stack the intervention list before secondary charts; use logical spacing for RTL and textual severity labels beside colored dots.

Acceptance tests and owner

Operations product and analytics engineering own the metric contract; SRE owns signal freshness. Test ordinary-role rendering with the actual controller response: every supported card displays its fixture count and every unsupported card is absent. Selecting seven/thirty days must alter the declared query window or be unavailable. Inject one failed source: its card becomes unavailable/stale, never zero, while others remain usable. Seed historical cancellations outside the selected cohort: completion rate remains unchanged. Verify a support card opens exactly its counted statuses, and keyboard users can read definitions and resolve an authorized incident with a mandatory note. These are required future acceptance tests, not executed certifications.

2. Operations Overview

Route and business purpose

/admin/operations is the cross-domain command center for an operations shift. It should reveal which service needs action and which facts cannot currently be trusted. Its value is coordination between delivery, jobs, food orders, support, disputes, privacy, CRM, and financial exceptions. It must preserve the independent authority of those systems instead of inventing another payment ledger or dispatch engine.

Verified current implementation

The route and API require manage_operations (client/src/admin/adminRouteManifest.js:22, server/routes/admin.js:69). The controller probes relation columns and groups table statuses (server/controllers/adminOperationsController.js:15-47), then loads fifteen sections concurrently (:114-207). Financial, settings, audit, and credit cards have additional access decisions; inaccessible financial cards are redacted (:109-112, :132-134, :183-184). Every successful/degraded section carries a correlation ID (:79-102). The browser refreshes every thirty seconds only while visible (client/src/admin/pages/OperationsOverview.jsx:193-216). Existing tests cover bounded section output and finance/credit redaction (server/__tests__/adminOperationsController.test.js:26, :61) and regional denial (:85). This is an existing command center, not a recommendation to build a replacement dashboard.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: delegated-domain authority is inconsistent.** permissionGranted queries only admin_permission_grants (server/controllers/adminOperationsController.js:62-72); authoritative middleware also permits validated delegation_grants (server/middleware/adminAuth.js:143-156). An admin authorized through delegation for financials/settings/audit can reach the domain endpoint yet sees its overview card denied. This is erroneous denial, not disclosure.
2. **DESIGN GAP, P1: response limits do not limit aggregation work.** statusSummary groups every row before LIMIT 50; countTable counts an entire selected relation (server/controllers/adminOperationsController.js:33-38, :55). Fifteen concurrent sections contain multiple queries each. With hundreds of active employee sessions, the thirty-second poll can create heavy repeated scans. Actual plans and concurrency capacity are **UNVERIFIED**, so no specific overload rate is asserted.
3. **DESIGN GAP, P2: table totals are weak action signals.** Jobs/orders include all historical statuses, and SLA uses open breach flags but no oldest breach, accountable queue, or breach duration (server/controllers/adminOperationsController.js:123-130, :160-166). Security failures and failed webhook totals are counts without a specific recovery task (:168-174, :192-198). A card cannot tell the commander whether today's actionable failures are growing.

Expansion failure scenario

Delivery and support incidents occur simultaneously. Thousands of employees open the command center; repeated historic counts contend with operational queries. A delegated financial responder cannot see the finance card despite having access to Payments. Large lifetime totals conceal a handful of newly stuck events. The correct dependency chain is incident signal → scoped exception queue → domain action → refreshed evidence, with finance and privacy data remaining controlled by their existing endpoints.

Required changes and operator design

Reuse one authority resolver for direct and delegated grants; test its result against the actual downstream middleware. Produce bounded, periodically computed summary snapshots with domain-specific freshness rather than promising that LIMIT bounds aggregate cost. Add active-window counts, oldest unresolved timestamps, severity and exception IDs, each mapped to an existing recovery queue. New overview drilldown endpoints should accept domain/window/cursor and return permitted exception summaries, not full customer records. An unavailable source must not be inferred from a missing table as an empty workload.

Operator screen design recommendations

Keep the current section cards but split the screen into “Urgent exceptions,” “Service workload,” and “Restricted management signals.” The header shows effective scope, a shift-window selector, overall fetch time, and the number of degraded sources. Give each card its own source timestamp and observation interval; a single generated response time is insufficient to prove that the underlying information is current.

The urgent list should have domain, exception class, unresolved count, oldest age, latest change, incident linkage, and destination. Define delivery backlog as pending/processing events beyond their retry deadline, support breach as unresolved first-response/resolution breach, and webhook backlog as due uncompleted deliveries. Saved views should support delivery-command, customer-care, and incident-response presets with window and severity filters, not a fifteen-card wall for every employee.

A card drawer displays source coverage, active versus historical totals, recent exceptions, and the exact filter used in its “Open queue” action. The command center may assign an incident coordinator or attach a signal to an incident, after previewing linked systems; financial settlement/retry decisions stay in their existing specialist workflows. Bulk actions here should be limited to acknowledging selected signals with a shared incident reference, never changing domain records.

When a section fails, show its last successful sample and retry only that section. A schema/configuration absence says “Source not configured,” separate from “No unresolved work.” A restricted card offers the existing access-request route and no count. Collapse normal cards on narrow screens, announce changed exception counts without stealing focus, and use RTL-compatible arrows and accessible table headings.

Acceptance tests and owner

Operations command-center engineering owns the interface; domain leads own definitions; database/SRE teams own query budgets. A delegated finance grant must show the same authorized aggregate as an equivalent direct grant, while expired/revoked grants expose neither. Fail one relation and verify fourteen independent cards retain truthful state and correlation IDs. In an isolated six-million-user-equivalent dataset, execute a declared 250-viewer polling workload; pass only if API p95 is under two seconds and domain mutation latency remains within its established SLO. Agree that budget before implementation. Drilldowns must reproduce the counted exception predicate, include records beyond page one, and preserve incident linkage across a shift handoff.

3. Users

Route and business purpose

/admin/users is the customer/helper account operations directory. It should support finding a person, understanding their current lifecycle, and making a narrowly defined intervention with traceable consequences. It is not the employee-administration directory: workforce accounts belong in the existing corporate/admin workspaces. At six million users, staff need precise identity resolution and lifecycle context, especially before bulk suspension or recovery.

Verified current implementation

The server requires manage_users for listing, detail, name changes, and ban actions, while verification and privileged role/deletion operations use separate stronger gates (server/routes/admin.js:140-154, :194-202). Listing excludes admin/super-admin and internal placeholder accounts, permits bounded pages, and restricts SQL sort fields (server/controllers/superAdminController.js:546-585). It joins helper/subscription context and computes job/earnings summaries (:589-627). The client maintains URL-driven filters and clears page selection when filters change (client/src/admin/pages/UsersList.jsx:440-463). Ban writes compare the fetched active state in the update, returning conflict on concurrent change (server/controllers/superAdminController.js:854-874). Shared scope tests confirm regional grants cannot silently authorize this global list (server/__tests__/adminPermissionScope.test.js:23-40).

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: list-level financial data crosses the neighboring financial boundary.** A manage_users list request calculates and returns earnings_total from paid payouts without requiring view_financials (server/controllers/superAdminController.js:616-627; compare server/routes/admin.js:137). This is a server response issue even if a table omits that column. Separate finance field authorization is required; no public access is alleged.
2. **CONFIRMED DEFECT, P1: bulk “suspend/reactivate” uses a toggle contract.** The UI chooses eligible users from a stale list, then calls /ban without the desired state (client/src/admin/pages/UsersList.jsx:540-553). The server recomputes a toggle from current state. If another operator already suspended a chosen active user, the bulk suspension can reactivate them. The server's comparison guard protects its own read/write race but cannot recover the client's omitted intent.
3. **DESIGN GAP, P2: account suspension has competing meanings.** The list's active/banned filter is is_active only (server/controllers/superAdminController.js:562-563), whereas Enforcement writes is_suspended/suspended_until (server/controllers/enforcementController.js:48-55). A person operationally suspended through Enforcement can appear active here. Bulk failures are swallowed and summarized as successful-count feedback (client/src/admin/pages/UsersList.jsx:548-553), losing actionable per-user recovery.

Expansion failure scenario

A trust team suspends compromised accounts while customer-care runs a bulk suspend view. Toggle requests accidentally reverse some decisions. The directory calls enforcement-suspended users active and discloses earnings to staff whose task is identity support. Customer360, Enforcement, financial reporting, and account restoration must use one explicit lifecycle vocabulary, while employee access stays in workforce administration.

Required changes and operator design

Add an intent-based account-state endpoint accepting desired state, expected version, reason, and operation ID. It must reject stale conflicting intent and return individual bulk results. Apply financial projection gates before constructing list SQL. Return effective access state with causes, including inactive account, active enforcement, scheduled deletion, and restoration eligibility. Keep existing exclusion, pagination, URL filters, and privileged gates. Extend list filtering with exact user ID and country, and make summary cards explicitly global or filter-scoped. No password or credential workflow is audited or exercised in this assignment.

Operator screen design recommendations

Preserve the current searchable directory and URL filters. Its header should identify “Customer and provider accounts,” effective geographic access, snapshot time, and matched versus total counts. Replace the ambiguous active badge with a compact effective-access badge plus cause chips: account disabled, timed enforcement, or deletion scheduled. KPI cards must state whether they describe all accessible accounts or the current search, and link to the exact corresponding predicate.

Use columns for stable user ID, name, role/persona, masked contact, country/region, effective access, verification evidence age, subscription, last activity, and pending lifecycle action. Provide exact-ID lookup beside the existing text/location search; saved views should include deletion-due, verification-followup, and enforcement-active with named scopes. Reveal sensitive contact or earnings only in authorized drawer sections.

The drawer should show identity references, access causes, jobs/business links, Customer360 link, deletion timeline, notes, and action history. Row actions specify “Suspend account” or “Reactivate account,” never “Toggle.” Require a reason and preview active work/deletion implications; use existing stronger authority for deletion and role changes. Bulk preview lists selected IDs, desired state, exclusions, and expected versions; finish with applied/conflicted/failed rows and retry only explicitly selected failures.

Conflicts display the current access cause and preserve the operator's reason without reapplying automatically. No results offers clear filters and exact-ID entry; no access explains customer-directory permission; unavailable retains cached rows labeled stale with mutations disabled. Mobile uses an identity summary card and full drawer, keeping action confirmation above the keyboard. Contact identifiers use bidi isolation, and row selection/check-state announcements work without color cues.

Acceptance tests and owner

Account operations engineering and Trust & Safety jointly own lifecycle intent; Security owns field projections. A manage_users-only API response must omit payout totals. Race a bulk suspension with another operator's completed suspension: no selected user may become active. Every failed bulk row must retain its ID, reason, retry eligibility, and current state. Seed an active is_suspended enforcement with is_active=true: directory and Customer360 must display the same effective restriction. Verify internal/admin accounts remain excluded, filters survive navigation, exact-ID lookup is deterministic, and duplicate timestamps do not duplicate or skip users in the eventual cursor-based list. Current guarded toggles must remain covered while the intent endpoint replaces them.

4. Jobs

Route and business purpose

/admin/jobs and /admin/jobs/:jobId support investigation and intervention in marketplace work. Staff must distinguish hiding objectionable content from cancelling an accepted commercial commitment. At global scale, the page should expose the job's lifecycle, participant obligations, and financial dependencies before offering a destructive action.

Verified current implementation

Listing/detail/action require manage_operations; hard deletion requires delete_jobs (server/routes/admin.js:77-81). List pagination is bounded and search/status/category parameters are bound (server/controllers/superAdminController.js:1033-1045). JobsList preserves prior data on refetch failure (client/src/admin/pages/JobsList.jsx:47-61), and delete requires a reason in both UI and API (:80-89; server/controllers/superAdminController.js:1097). Deletion rechecks permitted statuses in its SQL, returning conflict if eligibility changes (:1101-1108). Migration 188 adds terminal-state constraints and a locking transition function (server/migrations/188_job_state_transition_safety.sql:11-24, :30-64). The customer cancellation path already has a specialized transactional service with payment checks (server/services/jobCancellationService.js:99-108).

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: administrative cancellation bypasses the established cancellation workflow.** Force action changes status directly, without cancellation timestamp/reason, bid cleanup, or payment prerequisite checks (server/controllers/superAdminController.js:1060-1064). The existing service records cancellation and handles related obligations. This is a path discrepancy; no production money loss is asserted.
2. **CONFIRMED DEFECT, P1: successful job mutations can lack audit evidence.** Force action and hard deletion catch and ignore audit failure after mutation (server/controllers/superAdminController.js:1087, :1110-1119). The existing cancellation service explicitly supports fail-closed transactional audit (server/services/jobCancellationService.js:34). A staffing mistake can become untraceable.
3. **DESIGN GAP, P2: budget display lacks its financial contract.** The list omits budget_type/currency but the renderer branches on budget_type and prints $ (server/controllers/superAdminController.js:1043; client/src/admin/pages/JobsList.jsx:175-178). Detail formatting also assumes USD (client/src/admin/pages/JobDetail.jsx:24-29). International monetary interpretation is unsupported; actual non-USD job population is **UNVERIFIED**.

Expansion failure scenario

A moderation employee cancels an accepted job whose payment remains held. The job disappears from active operations without the participant/financial cleanup used by normal cancellation. Another employee exports all jobs while believing the export matches a filtered queue: current export sends no filters (client/src/admin/pages/JobsList.jsx:70). Support and finance then reconcile different records.

Required changes and operator design

Route administrative cancellation through the existing cancellation service with explicit actor/reason. Treat content visibility separately. Preview live commitments and payment dependencies; do not perform automatic refunds from this screen. Make audit success part of the state transaction. Return currency/budget type and export-filter receipts, and replace deep offset traversal with stable cursors when measured plans require it.

Operator screen design recommendations

Retain the list/detail route pairing, but put an effective territory, snapshot time, and active-work interval in the header. Define queue KPIs as published without accepted bid, accepted not started, overdue in progress, and disputed awaiting response. Every KPI opens its matching filter. Extend existing text/status search with exact job ID, category, scheduled-date range, assignee, payment dependency, and administrative flag; save “Unassigned accepted,” “Overdue work,” and “Financially blocked cancellation” views.

List columns should show job ID/title, city/country, customer, assigned helper, scheduled local start with timezone, lifecycle, age in current state, price with ISO currency, and exception chips. Detail becomes a drawer or existing detail page with sections for work description/evidence, participant commitments, accepted bid, payment/escrow references, dispute history, and chronological administrative decisions.

Preserve stable links to Support and Payments rather than copying their mutation controls.

Row actions are open, flag/unflag, hide content, and request cancellation. A cancellation preview shows present state, affected participant/bid, notification plan, and blocking payment IDs; confirmation requires reason and expected version. Deletion must preview retention/foreign-key impact and remain restricted to eligible states. Bulk work is limited to flagging or assigning review, with individual receipts; no bulk cancellation/deletion at launch.

On conflict refetch the job and show which prerequisite changed. On timeout recover by operation ID before retrying. Empty says no jobs match the selected lifecycle; unavailable keeps stale rows with actions disabled. No access distinguishes operations read from delete authority. Mobile presents job/exception cards, RTL uses logical column alignment, and confirmation dialogs trap focus and return it to the acted row.

Acceptance tests and owner

Marketplace operations engineering owns lifecycle; payments engineering owns cancellation prerequisites. Seed an accepted job with pending payment: admin cancellation must reject with the same actionable block as normal cancellation. Successful cancellation must atomically record reason/timestamp/audit and participant effects. Inject audit failure: neither cancel nor delete commits. Verify open budgets and explicit foreign currency display without $ substitution. Export receipt must match applied filters or clearly declare global export. Existing migration constraints must remain effective under concurrent completion/cancellation.

5. Reports

Route and business purpose

/admin/reports handles individual content complaints, with reporter evidence and a review decision. Its purpose differs from the job-centric Moderation queue and the user-behavior reports dashboard. Operators need to see what was reported, decide what rule applies, and route an escalation to a named reviewer without prematurely clearing the case.

Verified current implementation

List/review require manage_operations (server/routes/admin.js:72-73). The API returns bounded oldest-first pages (server/controllers/adminController.js:55-80). Pagination now exists in the browser (client/src/admin/pages/ReportsList.jsx:233-253); the old unreachable-page finding is fixed. Review locks the report and rejects already-reviewed cases (server/controllers/adminController.js:101-118). Unsupported removal targets are rejected (:121-126), and the UI uses shared confirmation (client/src/admin/pages/ReportsList.jsx:46-53). Job/user deep links exist (:163-176). Schema allows job/user/bid/review targets (server/migrations/005_admin.sql:84-101); report idempotency coverage exists in server/__tests__/communicationEdges.test.js:196.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: job removal directly cancels commercial work.** Report review writes status='cancelled' with a generic reason (server/controllers/adminController.js:143-151) rather than the transactional cancellation service. This path has the same obligations problem as Jobs, but is reachable from a complaint decision.
2. **DESIGN GAP, P1: escalation becomes a status without an executable followup.** Review writes escalated, while eligibility accepts only pending/open (server/controllers/adminController.js:113, :137). The UI exposes action buttons only on pending (client/src/admin/pages/ReportsList.jsx:188-199). There is no assignment or escalated-case decision path in these handlers. Escalation can move work out of the actionable queue rather than to a reviewer.
3. **DESIGN GAP, P2: decisions lack durable rationale/delivery context.** Client action_taken is the translated button label (client/src/admin/pages/ReportsList.jsx:58-61), not policy evidence. Warning notification occurs before report commit, failure is tolerated, and audit is subsequently best-effort (server/controllers/adminController.js:198-224). The user can receive a warning without durable review if commit fails; the browser ignores the returned delivery result.

Expansion failure scenario

Hundreds of complaints during a regional abuse campaign are escalated across timezones. No reviewer owns the escalated backlog. A reported accepted job is cancelled without financial handoff, and failed warning notifications appear as generic success. Reports, Enforcement, Content Removals, and job cancellation need a linked case ID and explicit next decision.

Required changes and operator design

Persist policy code, evidence snapshot, review owner, escalation destination, due time, and decision history. Extend the existing locked decision endpoint with expected case version and a supported escalated-review transition. Use post-commit notification outbox receipts. Delegate removals to domain services; retain the current target validation and review conflict response. Add evidence detail for bids/reviews rather than relying on unrelated navigation.

Operator screen design recommendations

Replace long report cards with a triage table and a case drawer. Header shows content-report scope, last successful queue load, pending count, escalated awaiting decision, and oldest unreviewed age. These KPIs use complete server counts, not visible-page length. Filters include target type, complaint reason, country/language, submitted interval, owner, escalation destination, and evidence completeness; save “Urgent harm,” “My escalations,” and “Missing target evidence.” Preserve existing pagination and status tabs.

Columns are report ID, target link/type, rule allegation, brief description, reporter trust/context indicator, age, owner, status, and next deadline. Mask reporter contact until required for outreach. The drawer separates allegation/evidence, current target snapshot, prior related reports, policy decision, and notification/audit receipts. Links to Jobs and Users remain, while bid/review evidence is rendered directly with source IDs.

Allowed decisions are dismiss, warn, domain-safe removal request, or escalation. Show only target-supported actions. Require policy reason and rationale; removal preview must identify live-work/payment blockers before confirmation. Escalation requires a destination and acknowledged handoff, not only a new status. Bulk actions may assign cases or combine duplicate reports after preview; bulk punitive decisions should be unavailable.

On 409 retain the rationale and show the other reviewer's decision. After a response timeout look up the decision receipt before repeating. Empty pending means no pending reports in this scope, with escalated backlog still visible. A missing target becomes “Evidence unavailable/target removed,” not dismissible by assumption. Permission denial hides evidence; source outage preserves stale case IDs. On mobile prioritize allegation and deadline; provide keyboard case navigation, focus-safe confirmation, bidi-isolated IDs, and text severity indicators.

Acceptance tests and owner

Trust & Safety operations owns policy and escalation, with marketplace engineering owning domain effects. More than 25 pending reports must remain reachable. Two decisions on one case yield one commit and one 409. An escalated case must have a named queue and be decidable there. Warning outbox failure must display retryable delivery status without duplicating the decision. A job-removal decision with a pending payment must not silently cancel. Each decision must retain machine policy code and operator rationale independent of display language.

6. Moderation

Route and business purpose

/admin/moderation is a consolidated queue of flagged or reported jobs. It should reduce repeated investigation by grouping complaints around a target, while keeping report adjudication and commercial cancellation separate. A moderator must know whether they are resolving allegations, clearing an internal flag, or changing marketplace availability.

Verified current implementation

The page is grant-guarded for moderate_content (client/src/admin/AdminApp.jsx:239); its list API uses the same grant (server/routes/admin.js:74). Current loading is abortable and errors are distinguished from a clear queue (client/src/admin/pages/Moderation.jsx:17-20, :64-77). SQL includes either metadata.admin_flagged or pending content reports, excludes cancelled jobs, and sorts by report count/creation age (server/controllers/adminController.js:265-276). Job deep links exist (client/src/admin/pages/Moderation.jsx:86-90). Actions call the shared job action endpoint (:25-39), whose conditional flag updates already return conflicts (server/controllers/superAdminController.js:1069-1084). The content-report schema and independent moderation queue are distinct (server/migrations/005_admin.sql:84; server/migrations/016_phase5_create_moderation_queue.sql:9-22).

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: list and mutation authority disagree.** A moderator with only moderate_content can load this queue, but action requests require manage_operations (server/routes/admin.js:79). The screen presents dismiss/flag/remove without reflecting that extra requirement. This is a broken least-privilege workflow, not a public endpoint.
2. **CONFIRMED DEFECT, P2: “Dismiss” cannot dismiss report-only items.** It sends unflag (client/src/admin/pages/Moderation.jsx:109-115), while SQL also selects pending-report targets. A job without an admin flag receives “not flagged” conflict (server/controllers/superAdminController.js:1080-1084); clearing a flag still leaves pending reports. The button label suggests adjudication that does not occur.
3. **DESIGN GAP, P1: queue retrieval is unbounded and evidence is thin.** The complete grouped query has no limit/cursor (server/controllers/adminController.js:250-277). Cards show a two-line description and report count, without reasons or complaint evidence (client/src/admin/pages/Moderation.jsx:92-105). “Remove” routes to direct cancellation, coupling moderation to the Jobs financial-path gap.

Expansion failure scenario

A mass-reporting attack floods the queue with tens of thousands of jobs. An authorized moderator loads a large payload, repeatedly tries Dismiss on report-only rows, and cannot resolve anything without broader operations grants. Senior staff cancel jobs to clear the queue, potentially affecting accepted obligations. Reports should supply allegations; Content Removals supplies visibility decisions; Jobs supplies safe commercial intervention.

Required changes and operator design

Add bounded target-case pagination, total count, and per-target reason aggregates. Separate clear_admin_flag, review_reports, and hide_listing capabilities; attach commercial cancellation as an explicit handoff with existing prerequisites. Rename Dismiss to Clear internal flag unless it genuinely adjudicates reports. Persist claimed review ownership and an expected target/case version. Reuse report decisions rather than create another incompatible moderation case store.

Operator screen design recommendations

Create a target-based triage workspace with header scope, sample time, distinct counts for internally flagged targets, reported targets, and targets awaiting evidence. Current subtitle uses loaded item count; replace it with complete scoped totals and separately display rows loaded. Search by job ID/title; filters cover flag source, reason cluster, report volume, oldest unresolved report, job lifecycle, region, and assigned reviewer. Saved views should distinguish “Reported only,” “Internal flag only,” and “Accepted work requiring commercial review.”

Columns show job identity, lifecycle, country, report reasons/count, first report age, flag source, reviewer, and evidence state. The drawer contains original listing/media, grouped reports with reporter privacy protected, prior decisions, participant/accepted-work context, and policy rationale. Display both active allegation count and previous dismissed allegations to make mass-reporting patterns visible.

Row actions must be explicit: claim review, clear internal flag, open grouped-report adjudication, hide public listing, or request safe cancellation. Preview lists the exact reports or visibility effect being changed. A moderator-only grant should allow authorized moderation effects and show why cancellation requires another function. Bulk actions may assign review or clear selected internal flags with individual versions; no bulk commercial cancellation. Require rationale for punitive visibility and link its receipt back to Reports.

After conflict refresh only the affected target and retain investigation notes. Outages retain a stale queue and disable effects. Empty must say no targets matching selected flag/report predicates; missing category/evidence should appear as an exception, not vanish silently. Mobile collapses target evidence into ordered sections; media has alt text where available, keyboard review navigation preserves focus, and RTL layouts avoid English-only reason alignment.

Acceptance tests and owner

Trust & Safety engineering owns target review; marketplace operations owns cancellation handoff. A moderator-only account must complete permitted flag/visibility decisions without gaining general operations access. A reported-only unflagged job must never offer a misleading Dismiss action. Seed 10,000 targets: response page is bounded, totals accurate, and every target is reachable. Clear a flag with pending reports: row remains until those reports are decided. Evidence failures must show a recoverable case state. Concurrent reviewers must get one successful versioned effect and one conflict, with audit receipts linked to the underlying reports.

7. Enforcement

Route and business purpose

/admin/enforcement records warnings, feature restrictions, and account suspensions. It must let trust staff reason about overlapping decisions and reversals without accidentally removing another team's protection. Enforcement differs from the Users account-disable toggle and from report dismissal; all three need linked evidence and consistent effective-access state.

Verified current implementation

Issue/list/deactivate use manage_users; audit reads require view_audit_log (server/routes/enforcement.js:14-18). Issuing an action, applying its effect, and writing enforcement audit are transactional (server/controllers/enforcementController.js:125-177). Restriction deactivation now targets only the rows tied to that enforcement ID; suspension deactivation checks other active suspensions (:388-405). These safeguards exist and should remain. The screen has action/active filters and a deactivate confirmation (client/src/admin/pages/AdminEnforcementDashboard.jsx:143-169, :380-382). Appeals already have a persistent schema (server/migrations/173_enforcement_appeals.sql:4-30); it would be incorrect to recommend building appeals storage from scratch. The relevant current shared authentication regression tests include TOTP enforcement (server/__tests__/adminAuthSecurity.test.js:54); overlap behavior still requires targeted isolated tests.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: temporary suspension destroys underlying restriction effects.** Issuing suspension calls liftUserRestrictions for all active restrictions (server/controllers/enforcementController.js:150-154, :63-68). Lifting suspension clears user flags but does not restore those previously active restrictions (:75-83, :396-405). The original restriction actions can remain active while their enforcement rows are inactive.
2. **CONFIRMED DEFECT, P1: invalid restriction input broadens to “all.”** Unsupported restrictions.type silently defaults to all (server/controllers/enforcementController.js:24-31). A typo in “bidding” can block checkout and job posting rather than produce validation failure. Operators need explicit effect validation.
3. **DESIGN GAP, P1: overlapping suspension changes lack serialization.** Deactivation reads the action before its transaction and queries other suspensions without a user-level lock (server/controllers/enforcementController.js:365-405). Concurrent issue/deactivate can produce inconsistent derived user flags. Actual race outcome is **UNVERIFIED** until reproduced. Audit rationale on deactivation is a constant string (:413), preventing review of why a sanction was reversed.

Expansion failure scenario

Different timezone teams apply bidding restriction and temporary suspension. The temporary suspension ends, unexpectedly removing the bidding safeguard. A reversal lacks an operator rationale. Users may show active while other enforcement records say restricted. Trust staff need one effective restriction calculation, linked to appeals and underlying reports.

Required changes and operator design

Keep restrictions as independent active decisions; derive their effective union rather than deleting them when a stronger sanction applies. Lock the user's enforcement state during issue/reversal and recalculate suspension end from remaining decisions. Reject unsupported restriction types, require explicit expiry/reversal reason, and connect existing appeals to the case drawer. Persist expected version and original policy/evidence references.

Operator screen design recommendations

Use a two-pane sanction workspace. Header states effective enforcement scope, snapshot time, and policy-version reference. KPI tiles are active warnings, active feature restrictions, active suspensions, and appeals awaiting review; distinguish decisions from unique affected users. Filters should include exact user/action ID, sanction type, affected feature, expiry window, policy code, issuing team, and appeal status. Saved views include “Expires this shift,” “Overlapping sanctions,” and “Appeal awaiting evidence.”

The table shows sanction ID, user identity reference, action, exact features blocked, effective/expiry times, issuing reviewer, linked report, appeal indicator, and current effect. Clicking opens a drawer with allegation/evidence, user enforcement timeline, effective-access union, live work/payment implications, and existing appeal history. A visual effect preview should show which restrictions survive a proposed reversal;

do not imply that deactivating one action restores unrestricted access.

Issue warning/restriction/suspension and deactivate are explicit row actions. Require validated feature selection, policy code, reason, expiry or deliberate indefinite choice, and preview of impacted capabilities. Reversal requires rationale and expected version. Bulk actions should be limited to assigning reviews or exporting permitted case references; sanctions need individual evidence at launch. Route appeals to a separate reviewer and show the next required decision.

On conflict reload the user's full sanction set and preserve draft rationale. Retry timed-out issue requests only after operation-receipt recovery. Empty means no sanctions match filters; no appeal data must be distinguished from an appeal-service outage. Permission denial names sanction/audit access separately. Mobile stacks the effect preview above confirmation; keyboard users can inspect feature chips and deadlines, and RTL preserves chronological meaning with explicit timestamps.

Acceptance tests and owner

Trust & Safety policy engineering owns effect composition. Issue a bidding restriction, then temporary suspension, then reverse suspension: bidding must remain blocked. Invalid feature strings must return 400 without rows/effects. Race two reversals with a new suspension: final flags equal the remaining active decision set. Reversal audit must retain actor, policy rationale, versions, and appeal reference. Existing per-action restriction reversal must remain isolated from unrelated sanctions.

8. Content Removals

Route and business purpose

/admin/content-removals lets moderation staff remove bids and hide/restore reviews. A bid is also an offer in a work agreement; a review affects public reputation. The operator must see those consequences, distinguish reversible visibility from contractual cancellation, and retain the reason for each restoration.

Verified current implementation

List/remove/restore all require moderate_content (server/routes/admin.js:84-87). Search is debounced, pages are bounded, and failed reads retain previous results (client/src/admin/pages/ContentRemovals.jsx:104-124). The removal dialog displays target evidence and requires a reason (:46-75). Bid/review effects and audit rows are committed together, with conditional state predicates protecting duplicate effects (server/controllers/adminController.js:467-493, :529-555, :589-613). Restore errors now surface (client/src/admin/pages/ContentRemovals.jsx:148-156). The base audit schema exists (server/migrations/005_admin.sql:11-30), while other review systems have separate tables; this page is not a universal content console. Shared admin authentication tests cover role/TOTP boundaries (server/__tests__/adminAuthSecurity.test.js:28-54), not every content consequence.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: accepted bids can be marked removed without agreement handling.** The update accepts every state except already removed (server/controllers/adminController.js:463-475). No job assignment, accepted agreement, payment, or cancellation prerequisite is checked. Existing job cancellation coordinates these obligations; this visibility action does not.
2. **DESIGN GAP, P2: restoration has no rationale or case reference.** The client posts an empty request (client/src/admin/pages/ContentRemovals.jsx:150), and audit stores rating/reviewer but no restoration reason (server/controllers/adminController.js:605-611). A mistaken hide can be reversed, but an appeal or coordinated restoration cannot be explained from that receipt.
3. **DESIGN GAP, P2: audit before-state can be stale.** Bid/review details are fetched before the transaction; update rechecks only removed/public state (server/controllers/adminController.js:463, :474, :525, :536). A concurrent edit can make recorded evidence differ from removed content. Reputation-cache effects are **UNVERIFIED**: these handlers do not recalculate ratings, but an independent trigger/consumer may do so.

Expansion failure scenario

A provider's accepted bid is removed during abuse review while their assigned job remains active. Another team restores a hidden review after an appeal without recording why. Support cannot reconcile agreement status and trust staff cannot reconstruct evidence. Content Removals must link to Jobs, report decisions, reputation recalculation, and enforcement appeals.

Required changes and operator design

Lock and snapshot the target inside the transaction. Restrict bid removal to noncontractual states or route accepted bids into safe job/financial handling. Require restoration rationale, original decision/case reference, and expected version. Verify and document reputation propagation with a targeted test before claiming correctness or a defect. Add media-review queue links to the new restaurant moderation API (server/routes/adminRestaurantMedia.js:20-29); reuse its existing flag/DMCA workflow rather than rebuilding it here.

Operator screen design recommendations

Keep Bids and Reviews as explicit tabs, with a header that says these are marketplace entities and links to restaurant-media moderation. Display last successful load, matched count, hidden-review count, and accepted-bid exceptions; define counts server-side. Search supports target/job/user ID and text; filters include bid lifecycle, review visibility, reason, removal date, reviewer, and linked appeal. Save “Hidden awaiting appeal,” “Accepted bid requires job review,” and “My recent removals.”

Bid columns include job, provider, offer amount/currency, status, accepted commitment, and removal decision. Review columns show reviewer/reviewee, rating, snippet, visibility, original decision, and pending appeal. The detail drawer contains the full content snapshot, linked job and participants, prior decisions, original report/appeal evidence, and reputation propagation receipt. Avoid forcing staff to infer commercial risk from the current simple amount-only modal.

Row actions are preview removal, hide review, preview restoration, or open contractual review. Before removal show exact visibility effect and accepted-work blockers; before restoration show the original rationale and current policy checks. Both require reason and expected version. Bulk actions may assign review or hide independently eligible spam reviews after individual evidence preview; accepted bids and restoration with appeals remain single-case decisions.

Conflicts show fresh content alongside the original draft and require renewed confirmation. Failed mutations remain in the modal with retry guidance; timed-out decisions recover their receipt first. Empty means no targets under selected filters, unavailable preserves stale rows, and no access withholds evidence. Mobile presents content first and agreement warning second; use focus-managed dialogs instead of the current generic modal shell, accessible rating text, and bidi-isolated monetary/user identifiers.

Acceptance tests and owner

Moderation engineering owns visibility; marketplace engineering owns contractual bids. Removing an accepted bid must block or coordinate a documented safe transition. Audit failure must roll back content state. Concurrent content edit/removal must record the exact removed snapshot. Restoration requires a rationale and points to its prior hide. Hide/restore a rated review and verify the public reputation pipeline's defined result before release. Restaurant media must remain on its existing correctly gated workflow.

9. Skills

Route and business purpose

/admin/skills maintains the searchable skills vocabulary used in provider discovery and job requirements. It is reference-data governance, not provider qualification approval. At worldwide scale, a rename or deletion must preserve historical meaning and downstream matching rather than merely changing a row in a lookup table.

Verified current implementation

The page supports name/category search, paging, edit, active toggle, and super-only hard-delete UI (client/src/admin/pages/SkillsManager.jsx:16, :91-129, :247-258). API gates are manage_operations; hard deletion checks super role (server/routes/admin.js:666, :753-762). SQL limits and offsets are bounded (:669-670). The initial schema has unique names, category, and active state (server/migrations/037_skills_and_licenses_lookup.sql:5-12). Sanitization has a later trigger (server/migrations/197_skill_input_sanitization.sql:3-20). Dependencies differ: business skills have a restrictive FK (server/migrations/079_business_skills.sql:7), jobs store skill IDs in an array (server/migrations/080_job_required_skills.sql:6-13), and user skills also use free-text names (server/migrations/039_user_skills_and_tool_rentals.sql:10-18). Pagination contract tests cover bounded admin handlers (server/__tests__/adminPaginationContracts.test.js:8).

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: schema/query mismatch in listing.** The list selects created_at (server/routes/admin.js:672), but migration 037 creates no such column and the inspected skill-altering migration 197 adds only sanitized name. A migration-built schema cannot satisfy that query. Actual production schema drift is **UNVERIFIED**; this is a repository contract defect, not a claimed live outage.
2. **CONFIRMED DEFECT, P2: creating an existing name overwrites it.** POST uses ON CONFLICT(name) DO UPDATE to change category and active state (server/routes/admin.js:709-723). An employee expecting a duplicate warning can reclassify/reactivate an established term, with no preview or expected version.
3. **DESIGN GAP, P1: taxonomy change lacks dependency and audit workflow.** Update/delete handlers write directly without audit/version receipts (server/routes/admin.js:732-772). Restrictive business FKs may block deletion, but job arrays and free-text user skills cannot provide equivalent referential protection. Renaming does not define aliases or historical display semantics.

Expansion failure scenario

Regional editors create translated variants of the same skill, reclassify an existing name through POST, or delete a term still referenced by jobs. Business-linked terms error; array-based job requirements become unresolved. Direct Catalog and provider qualification use related vocabularies but cannot safely assume this lookup is their canonical identity.

Required changes and operator design

Repair the migration/read contract with a new migration or remove unsupported fields. Make create reject duplicate canonical names and add a distinct merge/alias workflow. Introduce stable canonical IDs, translated labels, aliases, deprecation replacements, and versioned audited changes. Dependency-preview endpoints must count business FKs, job arrays, and free-text mappings before deprecation. Do not equate a skill label with license verification.

Operator screen design recommendations

Frame the screen as a taxonomy editor. Header shows vocabulary version, editing scope, last successful load, and pending change requests. KPI chips are active canonical terms, deprecated terms awaiting replacement, unmapped legacy names, and duplicate-label candidates. Extend current name/category controls with active/deprecated state, locale, usage count, and unmapped dependency filters; save “Unused candidates,” “Missing locale,” and “Replacement required.”

The list shows immutable skill ID, localized label, canonical name, category, active/deprecated status, downstream usage, replacement ID, and last reviewed change. A drawer contains translations/aliases, dependency counts by module, sample linked jobs/businesses, qualification notes, and revision history. Current category autocomplete can remain, but category clearing must have an explicit remove option rather than silently preserving COALESCE values.

Actions are propose create, edit labels, reclassify, deprecate with replacement, merge aliases, and inspect deletion eligibility. Preview duplicate names and affected modules before confirmation. Publishing a rename must show before/after labels and expected version. Hard delete requires zero dependencies and a typed identity confirmation; default deprecation preserves IDs. Bulk operations may recategorize or deprecate after per-term previews, with excluded dependencies and receipts.

Conflict recovery compares saved versus edited fields and keeps translation drafts. Duplicate create opens the existing term instead of overwriting it. Empty means no vocabulary matches, while migration/query failure says taxonomy unavailable with editing disabled. No access describes taxonomy authority without offering broad user-management grants. On mobile place term identity and dependency count above the editor; provide labeled locale tabs, Arabic text direction within fields, keyboard autocomplete selection, and screen-reader warnings for irreversible deletion.

Acceptance tests and owner

Marketplace taxonomy/data governance owns vocabulary; discovery engineering owns mappings. Apply migrations to an isolated empty schema: list must succeed. Duplicate POST must leave category/active state unchanged. Renaming preserves job/business references and legacy aliases; deprecation removes new selection while historical jobs remain readable. Hard delete with any dependency must return a specific blocker. Every change records actor/reason/version; conflicting edits produce 409 and no partial taxonomy change.

10. Support Tickets

Route and business purpose

/admin/support handles asynchronous cases, public submissions, internal notes, and customer replies. It must give each obligation an accountable handler and recoverable communication history. This ticket workflow is separate from Live Support conversations, although linking and escalation should carry context between them.

Verified current implementation

Reads/claim/status use manage_support; replying uses message_users (server/routes/admin.js:280-288). Claims now reassert assignment and nonterminal status in the update, and release rechecks ownership (server/controllers/supportController.js:592-616, :650-664). The former claim race is fixed and covered by server/__tests__/supportClaimT10Repro.test.js:35-90. Migration 600 prevents closed/resolved cases from jumping directly into active work (server/migrations/600_support_ticket_terminal_integrity.sql:10-26). Reply handling supports optional UUID idempotency and checks ownership/terminal state (server/controllers/supportController.js:764-822). The screen has priority/owner/status filters, detail actions, realtime refresh, and pagination controls (client/src/admin/pages/SupportTickets.jsx:829-859, :640-645, :901).

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: pagination controls send the wrong contract.** The browser sends page and limit (client/src/admin/pages/SupportTickets.jsx:591-592); the controller consumes offset, defaulting to zero, and never reads page (server/controllers/supportController.js:417-422, :502). Clicking subsequent pages reloads the first page despite the newly added controls. This is a current integration defect, not the old absence of controls.
2. **CONFIRMED DEFECT, P1: reply idempotency is not used by this UI.** The composer sends only content/internal flag (client/src/admin/pages/SupportTickets.jsx:297-300). A committed reply with a lost response can be resent as a second message because the server permits a null client ID (server/controllers/supportController.js:764-767, :801-808).
3. **DESIGN GAP, P1: status changes and reply effects are not one ownership/versioned transition.** Status updates target ID alone (server/controllers/supportController.js:692-697). Reply checks owner, inserts message, and updates case separately (:778-850). A transfer/close between those steps can produce misleading ownership or partially recorded reply effects. Terminal DB protection exists but does not make these operations atomic.

Expansion failure scenario

A team sees 1,000 urgent cases but can navigate only the first fifty. A network timeout prompts a duplicated reply; another employee closes the case while the original handler is composing. Support SLA, Live Support handoffs, and Customer360 now disagree about ownership and last customer response.

Required changes and operator design

Align page-to-offset conversion or adopt a cursor response end to end. Generate a stable client-message UUID for each draft and retain it through retries. Lock or version case status/owner through message creation and case effects; return separate durable-send and external-delivery receipts. Keep existing claim/release guards and terminal trigger. Make assignment/closure actions enforce the owner's authority and require a resolution reason.

Operator screen design recommendations

Keep the current inbox/detail arrangement, but header scope and counts must distinguish active company obligations from waiting-customer cases. Show last queue refresh separately from connection status. KPI tiles are unassigned urgent, mine due today, waiting-customer followups due, and unresolved beyond target; clicking applies their exact predicates. Existing status/priority/owner filters should gain category, source, last customer reply, due interval, and exact ticket number, with saved “My due cases” and “Unowned urgent” views.

Table columns include ticket number, subject/category, customer, priority, status, accountable agent, last customer/admin event, response deadline, and communication failure. The drawer separates customer-visible thread, clearly marked internal notes, linked conversation/Customer360 context, assignment history, next promised action, and resolution summary. Display whether a message is saved, externally queued, delivered, or failed rather than collapsing these into sent.

Row actions are claim, release, request transfer, reprioritize, reply, wait for customer, resolve, and explicitly reopen. Preview transfer/closure with current owner and outstanding followups; resolution requires reason/summary. Bulk actions may assign or reprioritize with per-case versions and receipts; customer replies/closure remain individual. Preserve drafts and UUIDs on timeout and recover by receipt before retrying.

On 409 show new owner/status and offer save draft or request handoff. Empty distinguishes no matching cases from all selected work completed. A statistics failure must not erase an otherwise usable inbox, unlike the current combined load. No access differentiates read/claim from reply permission; unavailable retains stale cases with actions disabled. Mobile opens a full-height drawer, internal-note mode has a persistent text warning, keyboard users receive live send status, and RTL isolates ticket IDs and customer contact.

Acceptance tests and owner

Customer support engineering owns the workflow. Seed 125 cases: pages 1-3 contain distinct complete records. Lose a reply response and retry: one message and one notification receipt remain. Race transfer/close/reply: stale actions return 409 without partial messages or owner overwrite. Existing claim-race tests stay green. A stats outage must preserve a loaded list and display degraded statistics; resolution/reopen must satisfy migration 600 and retain their decision history.

11. Live Support

Route and business purpose

/admin/support/chat is the live conversation workspace for platform support, embedded around the shared conversation component. It must let agents accept work within capacity, respond through the right channel, and hand over conversations without losing context. Availability preference, websocket presence, and actual staffed capacity are different facts.

Verified current implementation

The wrapper already distinguishes live-ready, connecting, and offline preference (client/src/admin/pages/SupportAgentChat.jsx:8-19). Platform-wide list now requires manage_support or eligible platform employment, with customer fallback for grantless admins (server/routes/support.js:282-303); the historical pure-role list defect is fixed. Claims lock agent capacity and conditionally acquire an unassigned conversation (server/controllers/conversationAssignmentController.js:50-101). Persistent message idempotency and lifecycle history exist (server/migrations/394_support_message_idempotency.sql:4-9; server/migrations/405_support_conversation_lifecycle_history.sql:1). Tests cover lifecycle transitions and audited/replayed email delivery (server/__tests__/supportConversationStateMachine.test.js:27-36; server/__tests__/supportConversationEmail.test.js:23-119). Customer360 mock repair is historical resolved work, not a new defect.

Concrete flaws and gaps

1. **DESIGN GAP, P1: a bounded conversation count still includes entire transcripts.** Global list aggregates every message into JSON before limiting conversations (server/routes/support.js:343-374). Fifty long-running cases can produce a large sensitive payload. List metadata should not require all message content; actual payload/capacity impact is **UNVERIFIED**.
2. **CONFIRMED DEFECT, P2: delegated support authority is omitted from list resolution.** Inline grant lookup considers only direct grants (server/routes/support.js:291-299), unlike common permission middleware's validated delegations (server/middleware/adminAuth.js:143-156). An authorized delegate can fall into a customer-only view and mistake it for a clear operations queue.
3. **DESIGN GAP, P2: supplementary evidence failures become empty state.** Customer360/lifecycle rejected requests become null/empty; reminder/template/knowledge failures also become empty arrays (client/src/components/support/SupportConversationWorkspace.jsx:284-299, :307-318, :384-400). These are production UI error states distinct from the fixed mock. An agent cannot tell “no followup/history” from “source unavailable.”

Expansion failure scenario

Hundreds of agents reconnect during a provider incident. Conversation metadata calls retrieve full histories; some context requests fail silently. An agent misses an existing promise and sends an inappropriate followup. Support Tickets, SLA monitoring, channel-delivery recovery, and workforce shifts must share the handoff evidence.

Required changes and operator design

Return metadata-only lists with cursor, last-message excerpt, unread count, SLA deadline, and source status. Fetch transcript pages only for the selected conversation. Reuse the common authority resolver and expose per-context loading/failed state. Preserve existing transactional capacity claim, channel recovery, idempotency, and lifecycle services. Measure reconnect behavior with the new isolated scale harness rather than treating its existence as a capacity result.

Operator screen design recommendations

Use a three-column desktop layout: queue, selected transcript, and context/action panel. Header should show employer/queue scope, staffed availability, socket connection, current chats versus capacity, and last successful queue sync. KPIs are waiting beyond first-response target, mine awaiting agent reply, channel sends failing, and handoffs awaiting acceptance. Preserve channel search and paging while adding department, language, age, agent, and response-due filters; saved views are “My replies due” and “Unclaimed language match.”

Queue rows show conversation/customer reference, channel, language, subject excerpt, unread count, wait age, SLA deadline, assignment, and failure chip. The context panel contains Customer360, related ticket/job references, lifecycle/assignment events, promises/reminders, and channel-delivery receipts, each with its own unavailable status. Transcript loading should use newest-page-first with earlier-history controls rather

than entire histories in list payloads.

Actions are claim within capacity, reply/internal note, transfer with summary and destination, schedule followup, request escalation, resolve with reason, and retry a specific failed channel send. Preview recipient/channel before sending; transfer remains pending until another agent accepts or a supervisor explicitly overrides. Bulk actions only triage/assign queue cases, never send mass transcript replies. Maintain a stable message ID and visible saved/queued/delivered/failed states.

On disconnect retain draft and show reconnecting; recover transcript and receipts before resending. Capacity/ownership conflict displays the new assignment, not generic failure. Empty states distinguish no eligible work, no queue access, and missing employment; unavailable context must never read as no history. Mobile becomes queue→transcript navigation with a context sheet. Provide keyboard composer shortcuts, announce new messages without stealing focus, and render customer Arabic text with appropriate direction independently of interface locale.

Acceptance tests and owner

Support platform engineering owns live staffing and channel evidence. Two concurrent claims at capacity must yield one eligible assignment. A 50-case list with 10,000 messages per case must stay metadata-bounded. Direct and delegated authority must produce equivalent eligible views. Fail Customer360/reminders independently: labeled unavailable panels remain retryable. Reconnect and replay sends must create one message/delivery; unresolved transfer cannot remove accountability. Exercise browser keyboard/RTL use and the isolated reconnect workload with declared payload/latency limits.

12. Quote Requests

Route and business purpose

/admin/quote-requests is intake triage for customer service requests. It records contact preferences, requested service, review state, and notes before Direct dispatch/quotation. Staff need a reliable lead queue and a clear handoff to pricing and scheduling; financial markers must not be mistaken for provider-confirmed payment.

Verified current implementation

List, detail, and update now share manage_operations (server/routes/quoteRequests.js:10-12). Detail loading guards rapid selection (client/src/admin/pages/QuoteRequestsInbox.jsx:82-104). Omitted currency now stays null so update preserves the stored value (server/controllers/quoteRequestsController.js:648-660, :846); the prior USD-reset defect is fixed and tested (server/__tests__/quoteCurrencyT12Repro.test.js:29-67). Money/status validation and confirmed-amount guard exist (server/controllers/quoteRequestsController.js:772-807; server/__tests__/quoteRequestsAuditGuards.test.js:154-231). Scheduling has a DB advisory-lock overlap guard (server/migrations/420_direct_schedule_conflict_guard.sql:17-36). The original persistent intake schema is migration 374 (server/migrations/374_quote_requests.sql:4-33).

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P2: KPI counts mix total scope with visible-page scope.** Total is server-provided, but new/active tiles count only the current twenty requests (client/src/admin/pages/QuoteRequestsInbox.jsx:198-213). Paging changes apparent intake workload without changing the underlying queue.
2. **CONFIRMED DEFECT, P1: confirmed price currency remains mutable.** The existing guard checks amount only when amount/payment is supplied (server/controllers/quoteRequestsController.js:775-807). Explicit currency alone bypasses that guard and writes through COALESCE (:846). A confirmed 10,000-cent price can be relabeled another currency. This changes a quote record, not proven movement of money.
3. **DESIGN GAP, P1: general PATCH is not versioned and can erase omitted notes.** It requires status, defaults notes to empty text, and writes admin_notes=$2 unconditionally (server/controllers/quoteRequestsController.js:639-640, :813, :843-844). Direct scheduling edits using the same controller can clear notes; concurrent intake editors overwrite state. Audit is limited to schedule/assignment changes (:877-885), leaving notes/status without equivalent history.

Expansion failure scenario

Regional intake agents triage the same request while dispatch changes its schedule. Notes disappear, last writer wins, and the new-request tile understates the backlog on later pages. Pricing may accidentally relabel confirmed value. The shared record ties this tab to Direct Ops, Catalog eligibility, Service Areas, and customer communications.

Required changes and operator design

Add complete status-summary counts matching filters, versioned partial-field semantics, decision history, and intake ownership. Lock price currency alongside confirmed amount; route approved monetary corrections through a separate explicit amendment with evidence. Preserve the fixed omitted-currency behavior and DB scheduling guard. Quote approval/payment confirmation should be distinct domain actions with clear receipts, not a broad status dropdown masquerading as settlement.

Operator screen design recommendations

Keep the two-pane inbox, adding effective intake scope, last list/detail refresh, and a visible dispatch-handoff status in the header. KPI definitions should be all matching new requests, requests awaiting staff response, and overdue promised contacts. Replace page-local calculations with server aggregates. Search supports request/reference/email/service and exact ID; filters add country/service area, contact method, intake age, owner, assigned dispatch member, and catalog linkage. Save “New unowned,” “Contact promised today,” and “Ready for dispatch.”

Rows show reference, customer, service/category, location, preferred contact, received age, intake owner, status, and next promised contact. Drawer sections are original request/photos, customer/contact consent, service-area/catalog evidence, staff notes/history, quotation terms, and Direct handoff. Existing notes/status editing should show changed fields and current version; a read-only price block includes ISO currency and confirmation provenance.

Actions are claim intake, record contact outcome, add append-only note, request quotation, and hand off to dispatch. Preview the fields and notification effect before state change. Currency/amount corrections after confirmation require a dedicated authorized amendment, rationale, and before/after terms. Bulk actions may assign leads or mark contact attempts with per-request receipts; pricing/payment states remain individual.

Conflict recovery compares versions and preserves unsaved notes. A list failure must show stale results explicitly, and a detail failure cannot leave another person's data under the selected identity. Empty means no requests match; no-access means intake grant absent; unavailable catalog/coverage means eligibility unknown rather than service approved. Mobile makes list/detail separate navigable views, RTL uses contact bidi isolation, and photo evidence/notes remain keyboard-accessible with clear form labels.

Acceptance tests and owner

Service intake engineering owns partial edits; Direct pricing owns financial terms. Notes-only edits preserve currency, and currency-only edits on confirmed quotes fail. Schedule-only PATCH preserves notes. Two editors receive a deterministic conflict instead of silent overwrite. New/active KPI counts remain unchanged across pagination and match server predicates. Every contact/handoff creates history and a named next queue; notes/status changes must be auditable without relying on calendar-only events.

13. Direct Ops

Route and business purpose

/direct/admin is the Direct service-operation portal, with inbox, calendar, work-order, payment-review, customer, and management subroutes. It coordinates company service delivery using shared quote-request records. Staff need to keep a customer commitment, dispatch assignment, payment proof, and actual completed work distinct.

Verified current implementation

Routes are mounted in the main SPA (client/src/App.jsx:357-369), rendering DirectAdminPortalPage. Admin APIs use manage_operations, with settings/export/audit separately gated (server/routes/direct.js:80-112). Calendar range, pages, and page sizes are bounded (server/controllers/directController.js:1883-1895). Scheduling uses persistent blocks/history and a database overlap guard (server/migrations/416_direct_calendar_blocks.sql; 418_direct_calendar_audit_history.sql; 420_direct_schedule_conflict_guard.sql:17-36). Slot capacity/hold recheck tests already exist (server/__tests__/directSlotEngine.test.js:45, :131). The browser has paginated calendar loading and export (client/src/pages/DirectAdminPortalPage.jsx:1047-1069). This is a substantial dispatch foundation, not an empty portal.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: customer-facing Direct routes bypass admin grant/MFA handling for role-based staff access.** /requests/:id and message routes have ordinary authentication only (server/routes/direct.js:70-72), while canAccessRequest permits any admin/super-admin role (server/controllers/directController.js:527-530). These paths serve details/messages to staff denied equivalent admin endpoints. Common requireAdmin checks are absent on this alternate staff branch; anonymous access is not alleged.
2. **CONFIRMED DEFECT, P1: payment-proof confirmation creates completion evidence.** Confirming proof sets completedAt to now if absent (server/controllers/directController.js:1767-1779). A receipt review can therefore produce a work-completion timestamp without completing the service. The operator's commercial record becomes misleading.
3. **DESIGN GAP, P1: manual payment markers have weaker decision controls than their significance.** Proof confirmation requires operations authority and a note is optional (server/routes/direct.js:78; server/controllers/directController.js:1767). It checks cancellation but not submitted proof/evidence reconciliation. This is marking a request, not proof of Stripe capture or settlement. Quote PATCH shares the currency/partial-edit gaps documented in Tab 12.

Expansion failure scenario

A dispatch-only employee reviews payment through an alternate authenticated route, marks it confirmed, and the customer history appears to contain completed work. Multi-country crews rely on a calendar with versionless shared quote edits. Direct Ops depends on Catalog capacity, Service Areas, Quote Requests, customer messaging, and financial evidence reconciliation.

Required changes and operator design

Apply one staff access resolver to both customer and admin routes, retaining customer ownership. Separate proof accepted, funds verified, and work completed with independent timestamps/authority. Require evidence references and explicit reconciliation outcome before financial confirmation. Keep scheduling guards, assignee pools, blocks, and history; use versioned amendments for shared quote/dispatch data.

Operator screen design recommendations

Treat the portal as a dispatch workspace with header territory, dispatch timezone, date range, last schedule refresh, and capacity-source status. KPI definitions are unassigned accepted jobs, today's scheduled work, overdue arrivals, and unresolved payment evidence; calculate them over displayed scope rather than all lifetime requests. Provide saved views for “Today by crew,” “Unassigned tomorrow,” and “Payment evidence exception.” Search/filter by reference, city/service area, skill/catalog service, assignee, status, time range, and financial blocker.

Calendar lanes represent assignees or crews, with explicit timezone, duration, travel buffer, and unavailable blocks. An adjacent queue shows reference, service, customer, local start, crew, commercial state, proof state, and next dispatch decision. Selecting work opens sections for customer/access instructions, scope/accepted quote, assignment/capacity, scheduling history, messages, and payment evidence. Keep

financial records accessible by authorized links rather than turning calendar actions into settlement controls.

Actions are assign, reschedule, acknowledge arrival, start, complete with work evidence, contact customer, or request cancellation. Preview conflicts and affected customer commitments before rescheduling; require reason/version and confirm notification. Proof review shows amount/currency, uploaded evidence, provider reference, reviewer, and separate verification result. Bulk actions may assign eligible unassigned work after capacity preview; no bulk payment confirmation or completion.

A schedule 409 shows competing booking/block and retains the draft. Recover uncertain writes by operation receipt before retrying. Empty calendar explicitly states no scheduled work in the range, with unscheduled queue still visible. Unavailable capacity disables booking; no access distinguishes settings/audit/financial denial. Mobile uses an agenda instead of narrow lanes, offers accessible time inputs, and renders RTL customer text without reversing chronological order.

Acceptance tests and owner

Direct operations engineering owns dispatch; financial operations owns evidence verification. An admin lacking operations grant or TOTP must not access another customer's Direct details through alternate routes. Proof confirmation must never create work-completion time. Concurrent booking/block changes must return precise conflicts and preserve one assignment. Customer notifications must reference committed schedule versions. Test calendar range/pagination beyond 100 jobs and currency/notes preservation through dispatch-only updates.

14. Direct Service Areas

Route and business purpose

/admin/direct-service-areas configures where Direct service can be requested through geographic radius checks. This is an offer-of-service boundary, not evidence that a crew is available. A worldwide launch requires country/postal semantics and a controlled activation decision connected to service-specific capacity.

Verified current implementation

Admin list/create/update require manage_operations (server/routes/direct.js:101-103). The page supports editing radius/anchor coordinates, ZIP lookup, active state, notes, and filtered coverage cards (client/src/admin/pages/DirectServiceAreas.jsx:174-210). Public listing projects area data separately (server/controllers/directController.js:1146-1155). Availability uses haversine distance over active areas (server/services/directServiceAreas.js:197-223). The persistent table and geocode cache are migration 377 (server/migrations/377_direct_service_areas.sql:5-33). Tests exist for the service and public page (server/__tests__/directServiceAreas.test.js, directServiceAreaPage.test.js); their existence is not worldwide coverage proof.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: accepted coordinates are not geographically bounded.** Validation requires finite latitude/longitude but no ±90/±180 bounds (server/services/directServiceAreas.js:227-239). Schema stores numeric values without corresponding range checks (server/migrations/377_direct_service_areas.sql:9-11). A typo can activate meaningless coverage.
2. **DESIGN GAP, P1: address model is US-specific.** ZIP geocoding uses /us/; address geocoding forces countrycodes=us (server/services/directServiceAreas.js:106-126). The input strips nondigits and truncates to five (client/src/admin/pages/DirectServiceAreas.jsx:179). This cannot represent many worldwide postal systems; the limitation is scope support, not an outage in supported US locations.
3. **DESIGN GAP, P1: publishing coverage has no version/impact/audit control.** Update reads a row then rewrites all normalized fields by ID (server/services/directServiceAreas.js:273-305); it has no expected version or transactional audit. Active input is boolean-coerced, so string false becomes true (:248). Coverage availability is geometric only, distinct from Catalog/Slot Engine capacity.

Expansion failure scenario

A regional editor activates an oversized radius or unintended true state while another narrows coverage. Customers request service outside staffed territory, while foreign postal addresses fail or are mis-geocoded as US. Quote intake and Catalog advertise inconsistent eligibility, and operators cannot reconstruct the launch decision.

Required changes and operator design

Validate geographic bounds and strict boolean types in API and schema. Add country, postal format, timezone, supported services, effective dates, and versioned activation history. Introduce preview-impact and point-check endpoints with provider/confidence information; preserve explicit supported-US mode until international geocoding is verified. Keep radius checks, but distinguish covered, requestable, and currently bookable.

Operator screen design recommendations

Replace the coordinate-first form with a coverage editor alongside map and textual area list. Header identifies country/territory, editing version, published-versus-draft coverage, and last geocode/capacity refresh. KPI chips show active areas, uncovered demand, pending activation changes, and areas without staffed services; geometric area count must not be labeled available crews. Search by area name/postal anchor; filters include country, active/draft state, supported service, and pending review. Save “Launch pending,” “Capacity missing,” and regional coverage views.

List columns are area ID/name, country/postal anchor, shape/radius with units, supported services, activation dates, version, and last approver. Selecting opens boundary/coordinate details, provider/confidence, sample address checks, overlapping areas, linked catalog capacity, affected open requests, and change history. Supply a text equivalent for every map geometry so operators can review without mouse/map access.

Actions are draft boundary, test addresses, preview expansion/contraction, request activation, activate/deactivate, and restore a prior version. Preview includes affected scheduled commitments and newly allowed/denied sample locations. Require reason/version and explicit confirmation before publishing. Bulk changes may deactivate selected areas only after per-area impact review and named responsible queue; editing an anchor must not silently re-enable coverage.

On geocoder failure preserve existing coordinates and show provider unavailable; never treat it as out-of-area. Conflicts show the newer boundary and allow draft merge. Empty means no configured areas under the chosen country, while no matches invites filter reset. No access is operations configuration denial; unavailable coverage prevents launch decisions. Mobile uses text-first boundary cards, keyboard-accessible address testing, localized kilometer/mile display, and RTL-compatible coordinate fields with signed values isolated.

Acceptance tests and owner

Regional launch operations and geo-platform engineering jointly own coverage. Reject latitude 91, longitude 181, and string booleans without writes. US ZIP and approved non-US postal fixtures must resolve only within their explicit country. Two edits yield version conflict rather than lost boundaries. Activation audit records before/after geometry and sample effects. Deactivation must preserve existing scheduled commitments through an explicit operations handoff; failed geocoding must never overwrite valid coordinates.

15. Direct Catalog

Route and business purpose

/admin/direct-catalog, also /direct/admin/catalog, manages service/tool categories, translated content, pricing, lifecycle, and staffing preferences. It determines what customers can see, request, or book. Catalog publishing must keep marketing claims consistent with actual availability and prevent a shortcut archive from bypassing lifecycle safeguards.

Verified current implementation

APIs require manage_operations (server/routes/direct.js:88-100). The UI has five-language validation, review issues, lifecycle filtering, and bulk-transition confirmation (client/src/admin/pages/DirectCatalog.jsx:121-153, :252). Capability derivation already distinguishes visible/requestable/bookable and checks lifecycle, operational flag, pool, and capacity (server/services/directCapability.js:47-87). Single-save transactions lock the row, optionally compare expectedUpdatedAt, and block retiring categories with live children (server/services/directCatalogService.js:663-708). Bulk transition locks selected rows and validates translations (:817-835). Migration 414 supplies lifecycle foundations; capacity/pool migrations 589 and tests server/__tests__/directCapability.test.js / directCatalog.test.js cover existing behavior. Do not recommend rebuilding lifecycle or translation guards.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: archive bypasses category retirement prerequisites.** The archive route calls removeEntity (server/controllers/directController.js:688-693), which directly retires the entity without the live-child check or transition validation (server/services/directCatalogService.js:763-778). It can hide a category containing live services even though ordinary save refuses that transition.
2. **CONFIRMED DEFECT, P2: the UI does not use the existing edit-conflict mechanism.** Single save supports expectedUpdatedAt, but the form/payload does not submit it (client/src/admin/pages/DirectCatalog.jsx:150-152; server/services/directCatalogService.js:668-677). Serialized writes still overwrite another editor's full record when no expected value is provided.
3. **DESIGN GAP, P1: admin listing is whole-catalog and staffing failures look empty.** It selects all three entity tables and enriches every service (server/services/directCatalogService.js:514-548). Pool load failure becomes zero pools (:523-524); browser assignee failure becomes empty (client/src/admin/pages/DirectCatalog.jsx:104-113). This can look like configuration removal rather than a dependency outage. Measured scale impact is **UNVERIFIED**.

Expansion failure scenario

Regional editors publish services while another archives the category. Live children become invisible, staffing enrichment fails, and an editor overwrites translations from an old form. Quotes and Slot Engine may still reference those services. Service Areas, Skills, translated marketing, dispatch capacity, and open bookings must remain linked in publishing previews.

Required changes and operator design

Route archive through the existing guarded transition service; require expected version for edits and selected-version receipts for bulk changes. Add server-paged admin search/review summaries and explicit staffing-source availability. Show derived public capability reasons alongside preferences. Preserve existing AI preview/apply separation, translation checks, transactional audit, and capacity engine.

Operator screen design recommendations

Keep category navigation, list, and editor, but header shows editing territory/locale, catalog version, last source refresh, and pending publication changes. KPI tiles are items needing translation/review, live but not requestable, bookable preference without capacity, and retirement blocked by dependencies. Filters should use server search with category, lifecycle, locale issue, operational state, pool/capacity reason, and service-area association. Save “Translation blockers,” “Live without crew,” and “Retirement review.”

List columns show immutable ID/slug, localized title, category, price with ISO currency, lifecycle, derived visible/requestable/bookable badges, staffing status, and revision. Drawer sections cover names/descriptions, pricing/terms, coverage, crew pool/slot rules, linked requests/bookings, publication history, and AI proposal diff. Preserve current accessible locale tabs, while Arabic editing direction is independent from tab order. A zero crew count must be visibly different from crew-service unavailable.

Actions are draft/save, preview public card, transition lifecycle, manage crew pool, and request guarded archive. Preview must show customer-visible changes, affected open bookings, missing locales, and blocked prerequisites. Bulk transitions show each selected version and exact effects; provide separate confirmation for cascading child pauses. AI apply requires selected field diff review and cannot implicitly publish.

A 409 retains translation drafts and opens side-by-side current/edited values. Failed staffing data disables capacity-dependent publishing decisions, not text drafting. Empty means no items match filters; no access names catalog authority; source outage retains a stale list with publication disabled. Mobile makes list/editor separate views, exposes capability explanations as text, and supports keyboard row selection and focus-safe transition confirmation.

Acceptance tests and owner

Direct catalog engineering owns lifecycle; regional service operations owns launch readiness. Both ordinary retire and archive must reject categories with live children unless an explicit guarded cascade applies. Two UI edits must send versions and yield one conflict. Failed pool lookup must display unavailable, never confirmed zero staffing. Paginated admin review reaches all items without full catalog transfer. Verify published five-language cards, capability invariants, and existing bookings after pause/retirement using isolated fixtures.

16. Messages

Route and business purpose

/admin/messages is staff-to-user correspondence, distinct from Live Support and private marketplace chat. Operators need continuity across staff handoffs, complete message history, and truthful delivery/read states. A supervisor inspecting another employee's conversation must not accidentally start a different conversation under their own identity.

Verified current implementation

APIs require message_users (server/routes/admin.js:656-658). Regular admins list their own conversations; super admins can list all (server/controllers/adminMessageController.js:36-80). Thread loading is abortable and sequence-guarded (client/src/admin/pages/AdminMessages.jsx:79-99). Sending sanitizes content, validates optional UUIDs, rejects admin recipients, and transactionally finds/creates a thread (server/controllers/adminMessageController.js:195-249). Message idempotency exists and has replay tests (server/__tests__/adminMessageIdempotency.test.js:99-144). Migration 060 prevents duplicate jobless participant pairs (server/migrations/060_conversations_null_job_unique.sql:11-13). Thus duplicate-thread protection and server retry support already exist; browser integration is the residual problem.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: displayed thread identity differs for supervisor inspection.** Super admins list another admin's conversation, but click loads by user ID (client/src/admin/pages/AdminMessages.jsx:102-104). Server resolves a thread for the current admin/user pair (server/controllers/adminMessageController.js:126-132). The selected row can show another employee's identity while the transcript is empty or belongs to the supervisor's own conversation.
2. **CONFIRMED DEFECT, P1: unread is cleared beyond what was displayed.** GET marks all unread messages read before returning oldest-first limited history (server/controllers/adminMessageController.js:146-163). Client requests no history offset and ignores total (client/src/admin/pages/AdminMessages.jsx:88-90). A 200-message conversation can mark recent unseen messages read while displaying only its first fifty.
3. **CONFIRMED DEFECT, P1: the browser omits retry identity and loses failed draft text.** Send submits content alone, clears composer before response, and removes the optimistic bubble on failure (client/src/admin/pages/AdminMessages.jsx:119, :133-145). A response timeout can duplicate an already saved message on resend. List also hard-limits 100 conversations without exposing total/paging (:56-57).

Expansion failure scenario

A supervisor responds to an escalated correspondence but receives a different thread. Recent customer replies are marked read without being visible, and timeout-driven resends create duplicates. Support handoffs and case audit cannot explain which employee actually communicated. This workflow must link into ticket/case history without giving ordinary employees global private-message surveillance.

Required changes and operator design

Resolve inspection by immutable conversation ID with participant/supervisor authority, and make send identity explicit. Use newest-first cursor pages plus a separate acknowledge-through-message-ID endpoint. Integrate existing client-message UUID support and recover receipts before retries. Add list pagination, employee handoff/ownership state, and delivery-state linkage without replacing the established messaging service.

Operator screen design recommendations

Keep inbox/transcript layout, but header shows “My correspondence” or an explicitly authorized supervisor scope, current sender identity, connection state, and last inbox sync. KPI chips are unread threads, customer replies awaiting staff response, failed deliveries, and handoffs awaiting acceptance. Existing recipient search should gain conversation ID, unread/reply-needed state, assigned employee, linked case, and last-contact range; saved views include “My followups” and “Supervised escalations.”

Inbox rows show conversation ID, customer, responsible staff member, linked case, latest excerpt, latest time, unread count, and delivery exception. Selecting opens that exact conversation, with newest messages visible and “Load earlier” preserving scroll position. A context drawer contains recipient/contact preference, participant/sender identities, case history, handoff notes, and delivery/read receipts. Viewing another employee's thread must display a prominent supervisor banner and require an explicit choice before replying as self or using an approved shared sender.

Actions are send, save draft, create/link case, request handoff, acknowledge displayed messages, and retry a failed delivery by message ID. Bulk actions may assign threads or mark only explicitly displayed items acknowledged; no bulk customer message sends. Before sending show recipient/sender/channel, enforce length, and preserve a stable UUID with optimistic pending bubble. Failure keeps editable draft and a retryable bubble; timeout first checks persistence.

Empty means no threads in the selected scope, while a thread with no messages invites a clearly identified first contact. No access withholds transcript; unavailable history keeps the selected identity and disables sending until context is recovered. Mobile separates inbox and transcript with a stable back route. Keyboard composer use, live delivery announcements, bidi-isolated IDs, and per-message language direction prevent RTL confusion.

Acceptance tests and owner

Messaging engineering owns identity/history; customer operations owns handoff policy. Supervisor selection must return the exact listed conversation. With 200 messages, newest unread content must display and only acknowledged IDs become read. More than 100 conversations must remain reachable. Lost send response/retry must yield one message/audit/notification. Failed sends preserve draft and UUID. Unauthorized staff cannot inspect unrelated conversations; existing participant-pair uniqueness and replay tests must remain valid.

17. Search

Route and business purpose

/admin/search locates entities across users, jobs, messages, disputes, payments, tickets, reviews, and bids. It should be a safe discovery interface into domain workflows, not an alternative route around their permissions. At six million users, operators need exact identifier lookup, truthful per-source completeness, and controlled investigative access.

Verified current implementation

The API requires view_search_logs (server/routes/admin.js:295). Search validates query length, escapes wildcards, and bounds per-type pages (server/controllers/adminSearchController.js:38-83). Payment/message type grants now exist: explicit unauthorized types return 403, default types exclude restricted domains (:17-20, :55-71). Historical unqualified payment-search access is fixed and covered by server/__tests__/federatedSearchT17Repro.test.js:28-80. The UI has abort/sequence handling and per-type pages (client/src/admin/pages/AdminSearch.jsx:414-479). Migration 657 adds message trigram indexing (server/migrations/657_message_search_trgm_index.sql:12-13); index presence is not measured multi-domain performance.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: current UI defaults contradict restricted-type behavior.** All types start selected and the browser always sends them explicitly (client/src/admin/pages/AdminSearch.jsx:404, :440). An ordinary search-only employee gets 403 on their initial search rather than automatic safe exclusions. The server fix is correct for explicit requests, but the UI is not permission-aware.
2. **CONFIRMED DEFECT, P1: residual message scope remains broader than correspondence authority.** manage_support permits searching the generic messages table with no conversation-participant predicate (server/controllers/adminSearchController.js:18, :143-156). Ordinary Messages APIs restrict threads to the employee's own correspondence (server/controllers/adminMessageController.js:75-76, :129). Adding a type grant did not align record scope. Other entity types also lack equivalent domain gates in the requirements map.
3. **CONFIRMED DEFECT, P2: failed sources become zero matches.** Rejected queries are extracted as empty/zero (server/controllers/adminSearchController.js:306-312). Response has no failed-source flags. Empty later pages also erase totals because count comes from the first returned row. An operational search can appear complete while a domain failed.

Expansion failure scenario

A search-only employee cannot use initial defaults. A support grantee with search access reads unrelated private marketplace messages. During a database slowdown, failed domain searches appear as no results and the investigator wrongly closes a case. Users, financial records, disputes, participant chat, and purpose-limited investigation access must use matching authority and evidence boundaries.

Required changes and operator design

Return permitted entity types and field scopes from one shared authority resolver, including valid delegations. Scope records before querying, with elevated private-content search tied to approved investigation purpose. Separate exact-ID lookup from broad substring search. Return per-domain success/unavailable/denied status, stable counts/cursors, and search receipt. Keep query bounds/escaping and indexes; benchmark realistic cardinality rather than calling output limits bounded query cost.

Operator screen design recommendations

Header should show effective search scope, approved investigation purpose where required, submitted query, and source completion time. Type chips come from server-permitted domains; disabled chips explain required authority without issuing failing requests. Replace global “results” totals with per-domain matched/returned counts and failed-source indicators. Filters include exact ID/reference mode, domain, country, created range, lifecycle, and limited field choice. Saved views retain non-sensitive filters and purpose templates, not customer search text by default.

Results remain grouped by entity with domain-specific columns: identity/state for users, job/assignment for work, ticket/owner/deadline for support, amount/currency/status for permitted payments, and case/outcome for disputes. Sensitive message snippets require approved purpose and bounded context. Each result drawer shows identity, match reason, permitted summary, source/version, related references, and a link to its authoritative screen. Search itself performs no punitive or financial mutation.

Allowed actions are open entity, link result to an existing case, request investigative access, and export permitted references with explicit filter/purpose receipt. Bulk selection should only attach references to a case; do not bulk contact users or alter financial state. Preview count, fields, and scope before export. Maintain independent domain pagination so a large user result set does not hide a small ticket match.

When one source fails, label that group unavailable and allow source-specific retry; never call the overall search complete. Stale results retain their query label. Empty means successful zero matches in listed authorized sources; no-access explains excluded domains. Mobile uses expandable domain groups and full-height summaries, keyboard users can traverse chips/results without losing query focus, and bidi-isolated identifiers/RTL snippets prevent identity confusion.

Acceptance tests and owner

Search platform and Security jointly own discovery scope. A search-only employee's first query must succeed for permitted domains. Restricted message search must fail without participant or approved investigation authority. Fail one domain query: result identifies failure, retaining successful groups. Empty later pages preserve total or return a valid cursor endpoint. Tests must cover delegated grants, revoked investigation access, output-field redaction, and exact-ID lookup. Run the isolated scale harness plus representative query plans; no live capacity certification is implied.

18. Businesses

Route and business purpose

/admin/businesses is the account moderation workspace for finding businesses, inspecting ownership and activity, changing availability, recording concerns, and soft deletion. At enterprise scale it should produce defensible business decisions and investigations, with clear distinctions between a business, its owner, and the owner's other businesses. It is already implemented.

Verified current implementation

client/src/admin/pages/BusinessModeration.jsx:243 loads paged, cancellable results and preserves previous data on error. Importantly, :282 now sends {reason, confirmed: true} for deletion: the earlier missing-confirmation defect is repaired. server/routes/admin.js:94 protects the list and mutations with current admin authentication and manage_users. server/controllers/businessModerationController.js:90 conditionally changes active state and returns 409 on a competing change; :446 performs soft deletion. server/migrations/161_business_moderation_enhancements.sql:4 supplies moderation/deletion fields. server/__tests__/businessDeletionT18Repro.test.js:27 covers confirmed deletion and :44 rejects an unconfirmed request. These are source safeguards, not a live deletion trial.

Concrete flaws and gaps

T18-1 - CONFIRMED DEFECT, medium: the activity filter is accepted but never used. The UI sends it at BusinessModeration.jsx:247; the controller destructures it at businessModerationController.js:9 but predicates at :13-22 address only search, active state, and deletion. A dormant-business saved investigation would therefore include businesses outside the intended criterion.

T18-2 - CONFIRMED DEFECT, medium: commercial indicators are not business-specific measurements. The projection at businessModerationController.js:37 returns a null category, :43 hardcodes earnings to zero, and :42 counts jobs assigned to the owner. Two businesses sharing an owner inherit the same job count regardless of actual business attribution. Zero cannot truthfully mean no earnings when no earnings aggregation was performed.

T18-3 - CONFIRMED DEFECT, high: flagging can report success without retaining the flag. businessModerationController.js:368 writes only an admin audit action, catches failure at :377, and returns “flagged for review” at :379. No durable investigation row is created in this path. The operator receives affirmative confirmation even if the only attempted record failed.

T18-4 - DESIGN GAP, medium: note editing has no stale-version protection. Despite its safeguard comment, the UPDATE at businessModerationController.js:405 predicates only on ID. A row lock or version comparison is absent, and deletion does not predicate on deleted_at IS NULL at :448. Repeated deletions and competing notes need explicit outcomes. The existing conditional active-state update solves a narrower race and should remain.

Expansion failure scenario

A regional fraud team investigates 800 interconnected businesses after a campaign. Owner-level job counts imply activity at every linked business, the dormant filter fails to narrow the queue, and one flag request loses its audit record. Another team overwrites investigation notes. Staff can mistakenly close a case as unprofitable or inactive without reliable evidence or a persistent case to hand over.

Required changes and operator design

Implement an explicit business activity predicate and business-attributed aggregates, or return null plus a measurement-status field. Replace flag-as-log with a transactionally persisted moderation case carrying reason, assignee, severity, status, and next review date. Add expected versions to note/state commands, retain deletion history, and make audit failure visible. Regional delegation needs a scoped query/target policy: current requirePermission deliberately accepts global grants (server/middleware/adminAuth.js:126), so a country dropdown alone cannot authorize regional operations. Link the case to Business360, ownership changes, active jobs, and settlements before suspension.

Operator design recommendations

Keep the business table but place a jurisdiction/team scope selector, result snapshot time, and “business measures” explanation above it. KPI cards should count matching non-deleted businesses, active businesses, open moderation cases, and cases overdue for review. Replace the misleading earnings zero with “not measured” until a business ledger aggregate exists; label owner job totals explicitly during migration.

Rows need business ID/name, owner, operating location, membership count, verified business activity, active/deleted state, case severity, and latest moderation decision. Search should run on the server across business ID/name and authorized owner identifiers. Add activity period, open-case, assigned-team, and review-due filters; save “Dormant, no open work” and “Flagged, awaiting evidence” views with query scope.

Open a drawer with Identity and memberships, Commercial dependencies, Moderation timeline, and Case tasks. Flag creates an assigned case; notes append investigation entries rather than replacing one paragraph. Suspend/reactivate and soft-delete remain individual actions. Confirmation must preview affected active jobs, memberships, and unresolved money, require a reason, and submit the current version. Permit bulk case assignment after a selected-ID preview; exclude bulk deletion.

On 409 show the competing decision beside the draft and require rereading dependencies. Audit persistence failure leaves the action unresolved, with a trace ID and retry status. No matches suggests clearing activity/date filters; no access names manage_users; unavailable data retains the last snapshot with actions disabled. Mobile cards prioritize identity/state/case due date, with the full dependency drawer accessible by keyboard. Use logical alignment and isolated IDs/emails for RTL; focus returns to the reviewed row after closing.

Acceptance tests and owner

Business Trust Operations should own decision policy; the Business API function should own data attribution and case integrity. Test two businesses belonging to one owner with disjoint jobs: each receives its own counts, and unknown earnings remains null. Activity filters must change both list and total. Inject audit/case persistence failure: no false flag success. Competing notes must yield one committed edit and a 409. Preserve existing deletion tests and add a browser payload assertion. A scoped operator must be unable to mutate a foreign target by ID; load-test representative indexed searches before agreeing a sub-second queue target.

19. Tool Rentals

Route and business purpose

/admin/tool-rentals moderates peer equipment listings. The business purpose is to stop unsafe or misleading supply while preserving rental obligations and evidence. Listing visibility, the owner's willingness to rent, existing bookings, and safety enforcement are different states; this tab currently represents most of them through a single availability flag.

Verified current implementation

server/routes/admin.js:123 requires admin authentication and manage_operations. server/controllers/businessModerationController.js:132 returns listing, rate, condition, location, and owner fields; :167 uses a conditional availability update with a 409 conflict response. The client client/src/admin/pages/ToolRentalModeration.jsx implements search, availability filtering, pagination, and a reason-confirmed toggle. The wider owner API already validates money and peer attestation at server/routes/toolRentals.js:245 and :281. server/migrations/208_tool_rental_peer_agreement.sql establishes agreement-related data; booking and review schemas exist in migrations 075b_tool_booking.sql and 074b_tool_reviews.sql. server/__tests__/toolRentals.test.js:33 checks required attestation; :48 checks availability updates preserve prices.

Concrete flaws and gaps

T19-1 - CONFIRMED DEFECT, high: an owner can undo an administrator's hide by updating the same is_available field. Moderation changes it at businessModerationController.js:168; the authenticated owner update accepts availability at toolRentals.js:272 and writes it at :523. There is no independent administrative hold predicate in that write. The race safeguard does not protect a later valid owner request. This proves a reversible enforcement mechanism, not that a live owner has exploited it.

T19-2 - DESIGN GAP, high: moderation decisions cannot distinguish safety suspension from voluntary unavailability. The list at businessModerationController.js:133 projects one flag; the toggle at :179 logs enabled/disabled actions with an optional reason. It creates no durable hold classification, expiry, evidence attachment, or assigned case. “Hidden” is insufficient information for a return-to-service decision on damaged equipment.

T19-3 - CONFIRMED DEFECT, medium: success can survive lost moderation evidence. Audit failures are swallowed at businessModerationController.js:184, after the state update. The response at :185 confirms hiding even when its historical reason was not retained. Because this action can affect income and safety, a mere application log is not an adequate substitute for a committed decision record.

T19-4 - DESIGN GAP, medium: the admin projection at businessModerationController.js:132 excludes deposit/replacement value, current booking exposure, and agreement evidence, although the owner API validates those fields. Its raw page/limit parsing at :120-143 lacks bounds. Visibility controls therefore operate without showing the financial and contractual dependencies already represented elsewhere.

Expansion failure scenario

An equipment recall affects 1,200 listings across regions. Moderators hide the affected equipment, but routine owner edits make some available again. Staff cannot see which tools are currently rented or require safe return instructions. A six-million-user marketplace amplifies each resumed listing's reach; the failure is enforcement-state ownership, not an absence of rental infrastructure.

Required changes and operator design

Add an administrator-owned moderation state, separate from owner availability, and require both to permit browsing or new bookings. Expose a command such as POST /api/admin/tool-rentals/:id/moderation-decisions with desired state, reason code, case ID, expected version, and review date. Preserve the current conditional-update protection. A hold should block new rentals without silently cancelling existing bookings or releasing deposits. Reuse booking, agreement, dispute, and refund services for those consequences. Bound pagination and make audit plus hold persistence atomic; unauthorized owner writes must never clear a moderation hold.

Operator design recommendations

Replace the binary availability header with “Rental listing safety” and a visible market scope and last successful refresh. Four cards should measure publicly rentable listings, administrative holds, unresolved safety cases, and active bookings on held equipment. Do not calculate those values from the visible page; request scoped aggregates. The current Hide/Show control becomes a state-specific “Place hold” or “Review release” action.

The queue should include tool ID/name, category, owner availability, administrative status, city, rate currency, deposit exposure, active rental count, complaint count, and next safety review. Add brand/model, category, market, hold reason, booking exposure, and report-age filters beside server search. Saved views should include “Recall: active rentals” and “Release requested: evidence missing.”

A drawer should show images/description, attestation and agreement version, booking timeline, deposit/refund dependencies, reports, and the moderation case. Releasing a hold requires required repair/inspection evidence and a preview of publication eligibility. Holding equipment previews affected bookings and produces a communication task. Allow bulk recall holds only after an exact selected-ID/model preview; bulk release requires individual evidence and should be unavailable initially.

If the owner changes availability during review, display both changes and preserve the independent hold. Retry a timed-out command by command ID, then read the decision rather than toggling again. A true empty queue says no listings meet the recall criteria; no access names the operations grant; failed booking exposure shows “dependency unavailable” and disables release. Mobile cards keep tool/state/exposure first; a full-screen detail view supports image descriptions, labeled controls, keyboard focus, and RTL text without reversing serial numbers or rate amounts.

Acceptance tests and owner

Marketplace Safety should be accountable, with Rentals Engineering implementing enforcement. Owner reactivation of a held tool must leave it excluded from browsing and new booking, while existing booking records remain readable. Competing moderation decisions yield one result and 409. A recall bulk preview must list exactly the applied IDs and preserve per-item evidence. Failed audit writes must roll back the hold command or return an explicit unresolved result. Test deposits and active bookings in the drawer; retain existing price-preservation and attestation tests. Evaluate bounded pages against a million-listing synthetic dataset, including oldest unresolved reports.

20. Planned Needs

Route and business purpose

/admin/planned-needs supervises future work and recurring activation. Operators must know whether a need is waiting, intentionally paused, ready to publish, or already linked to a live job. The correct intervention depends on which lifecycle has started; stopping automation is not equivalent to cancelling an accepted job or returning money.

Verified current implementation

client/src/admin/pages/PlannedNeedsAdmin.jsx:24 parses date-only values locally, avoiding a common display shift. server/routes/admin.js:127 protects supervision with manage_operations. The earlier pause defect is repaired: server/controllers/businessModerationController.js:245 now sets is_paused = TRUE. server/jobs/plannedNeedsScheduler.js:44 locks and rechecks status, linked job, and pause before publication; :176 and :198 exclude paused candidates. server/__tests__/plannedNeedsT20Repro.test.js:30 covers durable pause and later cases exercise locked eligibility. server/migrations/156_planned_needs_pause.sql supplies pause state. Force cancellation at businessModerationController.js:284 remains conditional on nonterminal state.

Concrete flaws and gaps

T20-1 - CONFIRMED DEFECT, medium: the admin list cannot reveal the repaired pause state. Its SELECT at businessModerationController.js:208-213 omits is_paused, while a cancelled auto-publish returns to planned. Operators see apparently ordinary planned work that the scheduler will deliberately skip. The success message at :261 also describes only a reset, without exposing the durable pause consequence.

T20-2 - DESIGN GAP, high: force cancellation and published-job handling remain disconnected. The handler reads published_job_id at businessModerationController.js:275, updates only the need, and returns the previously read ID at :299. Its broad nonterminal predicate can accept a state that changed to published after the initial read. The linked job may therefore survive as intended, but the returned dependency can be stale. The UI warning about surviving jobs is useful and should remain; a canonical refreshed dependency is still needed.

T20-3 - DESIGN GAP, medium: publication uses a locked eligibility record but candidate content from an earlier SELECT. The lock at plannedNeedsScheduler.js:45 reads only lifecycle fields; title, description, cost, and preferred helper are used from need at :60-68. An edit between candidate collection and lock acquisition can publish superseded commercial content. This is a current stale-content window, not a claim that the existing duplicate-publication guard is absent.

T20-4 - DESIGN GAP, medium: the scheduler loads all due candidates at plannedNeedsScheduler.js:185-201 and iterates sequentially at :204. There is no LIMIT in this candidate query. At global volume the admin queue needs publication latency and failure visibility; it currently returns due-date-sorted rows without activation lag, last attempt, or attempt owner. Whether today's worker volume is problematic is UNVERIFIED.

Expansion failure scenario

A fleet operator imports 30,000 recurring maintenance needs. Staff pause suspicious records during review and edit estimates as prices change. The admin queue hides the pause distinction, older candidate data can become published descriptions, and due items accumulate behind sequential processing. A support agent then cancels a need while a separate team continues its published job.

Required changes and operator design

Include pause state and published-job state in list/detail responses. Read publication content under the existing row lock, not from the earlier candidate snapshot. Add bounded scheduling batches, idempotent attempt records, error classification, and activation lag metrics. Implement separate commands for pause, resume, cancel-future-recurrence, and linked-job escalation; resume should preview the next activation date and require the latest version. Persist reasons and refresh dependency IDs after the mutation. Integrate Job Operations and payment cancellation policy without having this tab directly refund funds or erase completed work.

Operator design recommendations

Use an “Activation supervision” header containing market/timezone, scheduler heartbeat, and queue snapshot time. KPIs should count paused needs, eligible needs already past activation time, attempts failed in the selected window, and published needs requiring a separate job decision. Define activation time as due date minus lead time in the agreed service timezone, rather than equating due date with publication deadline.

Add columns for need ID/title, customer, due date, activation date, recurrence, pause reason, automation state, linked job state, last attempt, and activation lag. Search title/customer/need ID on the server; filter pause state, due range, recurrence type, failed attempt, and published-job existence. Saved views should include “Paused: review due” and “Activation overdue: no job.”

The drawer should show schedule and recurrence chain, latest editable content, reserved/estimated amounts labeled by meaning, preferred-helper eligibility, publication attempts, and linked-job actions. Existing cancel-autopublish becomes “Pause activation”; resume previews immediate publication if activation is already overdue. Force-cancel confirmation must explicitly say whether a live job continues and offer a linked escalation task. Bulk pause can be allowed with a reason and selected-ID preview; prohibit bulk linked-job cancellation.

On a publication race, reread the canonical job ID and switch to job follow-up. A failed resume keeps the pause visible and retains the draft reason. Empty means no needs in the chosen activation window; no access names manage_operations; missing scheduler health marks automation unknown and suppresses “all clear.” On mobile use a dated agenda with pause and lag badges. Announce state changes and conflicts, keep date-only fields stable in RTL, and provide keyboard access to recurrence/history sections.

Acceptance tests and owner

Service Operations owns activation policy; Scheduling Engineering owns publication correctness. After pause, two scheduler passes must create no job and the admin row must show paused. Editing cost/title between candidate selection and locking must publish the latest committed values. Simultaneous publication and cancellation must return the actual linked job and a truthful continuation warning. Process a 30,000-item synthetic workload in bounded batches with no duplicate jobs; every failure remains discoverable. Validate recurrence cancellation leaves historical jobs intact and an overdue resume preview accurately predicts immediate activation.

21. Payments

Route and business purpose

/admin/payments is the marketplace job-payment read workspace, exposing payer, payee, job, gross amount, platform fee, payout, and status. It is one financial domain, not a consolidated view of Consulting Pay, Orders, logistics costs, restaurant wallets, or Credit. Staff need traceable financial evidence and a clear route into exception handling.

Verified current implementation

server/routes/admin.js:137 requires view_financials; client/src/admin/pages/PaymentsList.jsx:44 has cancellable loading and a dedicated permission state at :77. server/controllers/paymentsAdminController.js:6 now clamps page/limit and :15 includes the entire date-to day using a half-open boundary. Its list and count use the same participant joins at :34 and :42, so the older count-mismatch recommendation is not current. Refunds are separately protected by issue_refunds at server/routes/payments.js:33. server/services/paymentRefundService.js:28 requires idempotency; :56 reserves attempts transactionally. Migration 597_payment_refund_reconciliation.sql and server/__tests__/paymentRefundReconciliation.test.js:38 support partial-refund accounting.

Concrete flaws and gaps

T21-1 - DESIGN GAP, high: refund and reconciliation state are absent from the table contract. paymentsAdminController.js:22-24 returns status and terminal timestamps but not cumulative refunds or outstanding refund attempts. The refund service updates cumulative value at paymentRefundService.js:182; partial refunds intentionally need not set full refunded status. A captured/released row therefore cannot establish its remaining refundable amount or whether an external operation is unresolved.

T21-2 - DESIGN GAP, medium: financial discovery is limited to status and creation dates. paymentsAdminController.js:8-19 has no payment ID, provider ID, participant, job, refund-attempt, or exception search. Although provider identifiers are selected at :23, the main table at PaymentsList.jsx:172-179 does not provide an investigation workflow for them. A user presenting a provider receipt cannot efficiently locate the authoritative payment.

T21-3 - DESIGN GAP, high: the page and projection have no currency contract. Formatting at PaymentsList.jsx:28 always uses USD and the selected payment fields at paymentsAdminController.js:22 omit currency. This may be a legitimate USD-only domain today; foreign-currency support is not assumed. Worldwide expansion must explicitly preserve USD restrictions or carry currency/minor-unit metadata rather than interpreting a global amount as USD by accident.

T21-4 - UNVERIFIED, medium: inner participant joins at paymentsAdminController.js:34-35 exclude rows whose payer/payee is absent. The corrected count faithfully follows the same rule, but completeness of historical finance evidence depends on deployed retention/FK behavior. Source inspection does not establish that such orphaned rows exist. Audit a synthetic deletion/retention fixture before deciding whether participant joins should be left joins with retained identity snapshots.

Expansion failure scenario

Support receives hundreds of refund inquiries after an external-provider timeout. Some payments remain captured with partial refunds and reserved attempts. Staff using this table see normal statuses and totals, search by date alone, and may hand cases to the wrong financial domain. Repeating a refund from another workspace must reuse the original intent, not infer eligibility from the row color.

Required changes and operator design

Add a finance detail endpoint with payment identity, currency policy, provider references, immutable accounting history, cumulative refund, remaining capacity, and attempt/reconciliation state. Reuse the durable refund service and the separate refund grant; do not embed an unguarded refund write in the list API. Provide scoped indexed search and keyset ordering (created_at, id). Link Job Operations and reconciliation exceptions by payment ID, while explicitly linking to other payment domains. Return a server snapshot timestamp and a completeness indicator if participant retention affects results.

Operator design recommendations

Title the screen “Job payments” and show the ledger/domain scope, selected creation/settlement window, currency policy, and last successful snapshot. KPI cards should report gross captured, net retained after settled refunds, refund amount awaiting resolution, and exception count. Define each metric by settlement state and currency; counts and amounts must use the complete filtered query rather than the current page.

Rows should retain payer/payee/job but add payment ID, currency, captured amount, cumulative refunded amount, refundable remainder, provider attempt state, and exception link. Search by exact payment/job/provider identifiers plus authorized participant fields. Offer creation-versus-settlement date basis and saved views “Provider outcome unknown,” “Partial refund inquiry,” and “Released, reconciliation exception.”

A drawer should separate Payment identity, Amount breakdown, Provider events, Refund attempts, and Job/dispute dependencies. Copying identifiers, opening a case, or exporting selected read evidence are row actions. Refund is visible only with issue_refunds, then fetches fresh remaining capacity and previews amount, fee reversal, reason, and original intent ID before confirmation. Permit bulk case assignment; prohibit bulk refunds from this list.

An unresolved provider result displays the existing attempt with “check outcome,” not a fresh refund button. A 409 refreshes capacity and preserves the entered reason. Empty means no payments for this domain/window; lack of financial authority shows its dedicated access panel; provider evidence unavailable leaves ledger amounts visible with investigation actions restricted. Mobile cards lead with ID/net/refund state and expose breakdown on tap. Keep monetary columns and provider IDs directionally stable in RTL and announce refreshed outcomes without clearing keyboard selection.

Acceptance tests and owner

Payments Operations should own exception triage; Payments Engineering owns monetary projections. Test a partial refund, full refund, reserved attempt, and timeout: each must render distinct truthful remaining capacity. Replay the same refund intent and verify one provider operation. Test date-to boundaries and participant-retention fixtures; assert no cross-domain totals are mislabeled consolidated revenue. If USD-only, non-USD records/commands must be rejected explicitly; if supported, currencies must never aggregate without conversion provenance. Retain current refund tests and add detail/list contract tests, including denied financial read versus denied refund write.

22. Orders

Route and business purpose

/admin/orders manages product marketplace fulfillment, order detail, shipping, cancellation, and refunds. Its operational record is a product order with line items and shipment evidence, not a service job payment or logistics purchase order. Fulfillment and financial state must agree without preventing investigation of resolved orders.

Verified current implementation

server/routes/adminOrders.js:8 requires manage_operations; :23 additionally requires issue_refunds. server/controllers/adminOrdersController.js:13 bounds list size and :67 applies matching search to totals. Shipping and cancellation now correctly invoke withTransaction(pool, operation) at :229 and :339; the prior transaction-signature defect is repaired. Refunded details remain readable at :130. Refunds call the durable marketplace refund service at :294, return 202 for unsettled outcomes, and include cumulative refunded amount at :316. server/migrations/603_marketplace_order_state_integrity.sql:26 guards terminal transitions. Existing repro tests at server/__tests__/adminOrdersT22Repro.test.js:33 and :112 cover repaired transaction/read behavior.

Concrete flaws and gaps

T22-1 - CONFIRMED DEFECT, high: detail unconditionally parses a JSONB address as JSON text. adminOrdersController.js:174 calls JSON.parse(order.shipping_address), while server/migrations/129_marketplace_orders_transactions.sql:13 declares JSONB. A normal decoded object is not valid JSON text for this call and causes the whole detail request to fail. The refunded-read repair does not fix this independent address contract.

T22-2 - CONFIRMED DEFECT, medium: shipping state errors become server failures rather than actionable conflicts. adminOrdersController.js:239 rejects only cancelled status before forcing processing at :247. Migration 603 correctly rejects pending→processing and completed/refunded resurrection. Consequently those invalid requests hit the generic catch/error response rather than a prevalidated 409. The database prevents corrupt transitions; the API still misrepresents routine invalid-state requests.

T22-3 - DESIGN GAP, medium: carrier input is not persisted with shipment data. It is read at adminOrdersController.js:222; the order UPDATE at :247 stores tracking number and timestamp only. It enters the action-log details but does not become canonical carrier evidence returned by detail at :192. Shipment investigation cannot depend on reconstructing the most recent free-form event.

T22-4 - DESIGN GAP, high: detail selects one arbitrary transaction at adminOrdersController.js:145 and only the latest twenty action-log entries at :151. There is no continuation in that response. Even with durable refunds elsewhere, this drawer does not establish a complete financial/event history, or distinguish the canonical transaction from another attempt. That matters for partial refunds and long-running disputes.

Expansion failure scenario

A holiday batch produces 50,000 paid orders and thousands of address/shipping inquiries. Normal address-bearing orders fail detail retrieval; staff attempt shipping from stale statuses and see generic 500s. Refund recovery operates correctly in its service, yet truncated drawer history leaves operators unable to explain which transaction or shipment was acted on.

Required changes and operator design

Accept decoded JSONB objects and handle legacy strings deliberately. Validate shipping/cancellation transitions before writes, condition writes on expected state/version, and map database transition conflicts to 409. Require carrier/tracking evidence appropriate to the fulfillment method. Use an explicit canonical transaction or a paged attempt collection, and expose durable refund attempts plus complete paged action history. Keep terminal guards, transaction fixes, and paid-order refund-before-cancel logic (adminOrdersController.js:353). Add shipment tasks that link delivery/logistics references where present instead of assuming every product order shares a logistics lane.

Operator design recommendations

The header should say “Product order fulfillment,” with warehouse/market scope, order-versus-event date basis, and snapshot freshness. KPI cards should distinguish paid awaiting shipment, shipments overdue, unresolved refund attempts, and completed net sales. Current aggregate revenue by completed order is a fulfillment measure; label it accordingly until accounting reconciles refunds and currency.

Keep order/customer/status/amount rows and add fulfillment deadline, carrier/tracking, canonical payment state, refund remainder, and assigned exception. Search exact order/provider/tracking IDs and customer fields; filter shipment age, warehouse, paid-unshipped, refund state, and delivery exception. Save “Paid, dispatch overdue” and “Refund accepted, provider pending” views with server-side paging.

The detail drawer should contain customer/address, line-item quantities and fulfillment, totals/currency, payment attempts, shipping milestones, and a paged action history. Mark shipped requires eligible paid state and carrier/tracking preview. Cancel remains for unpaid eligible orders; paid orders route to a refund preview showing remainder and idempotency identity. Bulk assignment or export is reasonable; bulk shipment requires per-order tracking validation and a result report, while bulk financial writes remain unavailable.

On address decode failure show a recoverable address section error rather than lose all detail. On 409 reload the current order and display the proposed transition alongside it. For 202 refund results retain the attempt and poll its status, preserving the command identity on retry. Empty differentiates no orders from no matches; no access identifies operations versus refund capability; provider unavailable preserves fulfillment evidence. Mobile uses item cards and a full-screen drawer; keyboard actions have labeled buttons, and addresses/identifiers receive correct RTL isolation.

Acceptance tests and owner

Commerce Fulfillment owns this workflow; Commerce Payments owns refund linkage. Test decoded JSONB, legacy JSON text, and null addresses: all valid cases return detail. Ship pending, completed, refunded, and cancelled orders: each returns a typed conflict without mutation; paid shipment persists carrier/tracking. Concurrent shipping/cancellation produces one allowed transition. Detail must expose more than twenty events through continuation and choose the intended transaction. Preserve existing refunded-order and transaction tests, then verify timeout/replay produces one refund intent and a visible unresolved status rather than a second money movement.

23. Templates

Route and business purpose

/admin/templates administers profile presentation pricing and exceptional user access. It combines preview, pricing, grants, revocation, and statistics. Revenue-affecting pricing and time-bounded access need coherent policies: a grant start date must mean something, and ending one grant must not accidentally invalidate another valid entitlement.

Verified current implementation

client/src/admin/pages/TemplateManagement.jsx:94 provides a pricing modal with preview and retained save errors at :112. Current grant routes require manage_settings at server/routes/templates.js:35, :44, and the revoke route; the older missing-grant allegation is repaired. Statistics use manage_operations at :31. server/controllers/templateController.js:352 revokes only an active access record. :192 rejects new lifetime purchases. server/migrations/168_remove_lifetime_template_pricing.sql:5 zeroes historical lifetime prices. server/__tests__/permissionGatesT11T12T23T25.test.js:39 inspects grant middleware. Preview and guarded APIs already exist and should be retained.

Concrete flaws and gaps

T23-1 - CONFIRMED DEFECT, medium: the pricing UI submits is_active at TemplateManagement.jsx:108, but templateController.js:204-210 never accepts it and the UPDATE at :219-225 never writes it. An operator can “save” a listing activation change that the server silently ignores. Either remove the input or implement the field and return its persisted value.

T23-2 - CONFIRMED DEFECT, high: future-dated grants are immediately usable. Grant creation stores start_date at templateController.js:330, but both getMyAccess at :36 and paid-template activation at :130 filter only active status and expiry. Neither requires start_date <= NOW(). The start-date form therefore promises a restriction not enforced by the authoritative access path.

T23-3 - CONFIRMED DEFECT, medium: revoking one access row resets active business/helper templates regardless of other valid access. templateController.js:361-369 changes every matching profile to default after the row revocation, without recomputing remaining entitlement. Those writes are also separate from the initial update, so a failure can leave an inconsistent partially revoked presentation.

T23-4 - CONFIRMED DEFECT, medium: zero pricing is replaced by default paid values. The modal initializes monthly_price_cents || 999 and lifetime_price_cents || 4999 at TemplateManagement.jsx:95-96; table cells repeat this at :223 and :229. Migration 168 deliberately sets lifetime prices to zero, while lifetime purchases are removed. The editor can display an invented $49.99 legacy price and write it back during an unrelated edit.

Expansion failure scenario

Marketing schedules template entitlements for a launch next month, while support grants temporary access to resolve a complaint. Customers can activate the future grant today. When support revokes its temporary record, the customer's independently valid entitlement is ignored and public profiles revert. Meanwhile zero-price legacy fields appear paid in the editor, complicating revenue and promotion review.

Required changes and operator design

Centralize effective entitlement evaluation across listing, activation, revocation, and expiry using start/end bounds. Recompute entitlement and presentation atomically on revocation; preserve another qualifying grant. Validate finite nonnegative integer cents and start-before-expiry, use nullish defaults, and remove deprecated lifetime pricing from ordinary edits. Implement a versioned activity command or remove unsupported controls. Make pricing and grant decisions durable with reasons, affected-user preview, and effective date; use existing previews and grant guards rather than rebuilding them. Corporate permissions and ordinary admin grants remain separate responsibilities.

Operator design recommendations

Keep the Pricing, Grants, and Statistics subviews, with a header describing market/currency policy and the last catalog/entitlement snapshot. Pricing KPIs should count active paid templates, free templates, scheduled price releases, and release conflicts. Grant KPIs should count effective, future, expiring soon, and revoked records according to the same server evaluator.

Pricing rows show slug, preview, current monthly price, active state, next effective price, and last approved revision. Remove ordinary lifetime editing; provide a read-only legacy entitlement section when needed. Grant rows show user, template, source, starts/ends, effective access, and decision reason. Search users/templates on the server and save views “Starts next month” and “Expiry within seven days.”

A detail drawer should expose template preview, pricing history, every entitlement source, affected business/helper profiles, and the decision trail. Price edits preview old/new cents and affected subscriptions before scheduling a release. Grant creation confirms recipient, bounded dates, source/reason, and current entitlement overlap. Revocation previews whether effective access actually ends and which profiles would revert. Bulk operations may schedule a reviewed price release or export grants; mass entitlement revocation requires an exact impact preview and separate reviewed job.

Conflicts retain the draft beside the latest catalog revision. Partial failures show which entitlement/presentation transaction needs recovery; retry by decision identity. Empty grant results explain selected dates/status; no access distinguishes pricing/grants from statistics; catalog unavailable hides previews and disables release. On mobile place preview and effective price together, then open full-screen entitlement history. Label date semantics and use keyboard-accessible dialogs; isolate slugs and currencies in RTL while preserving localized labels.

Acceptance tests and owner

Profile Commerce owns pricing policy; Entitlements Engineering owns evaluation. A future grant must not appear usable or activate before its start. Revoke one of two overlapping valid grants: the profile stays active; revoke the last: it reverts transactionally. An injected failure rolls back both access and presentation writes. Saving zero cents must retain zero, and editing unrelated fields must not resurrect lifetime pricing. An active-state save must persist or reject explicitly. Preserve permission tests and add merged entitlement, effective-date, and pricing-preview contract coverage with Marketing approval for release semantics.

24. Consulting

Route and business purpose

/admin/consulting is the consulting intake and assignment dashboard. It should convert an inquiry into accountable contact, appropriate specialist assignment, and a real appointment. This screen is the intake workbench; Consulting Sessions owns delivery, Scheduling owns reservations, and Consulting Pay owns financial investigation.

Verified current implementation

client/src/admin/pages/ConsultingDashboard.jsx:374 uses abortable loading, and :404 confirms contact changes. server/routes/consultingAdmin.js:13 wires request listing; :18 wires assignment behind authenticated admin checks. server/controllers/adminConsultingController.js:16 implements search/status/paging; :101 lists active consultants with workload counts. server/migrations/103_consulting_all_phases.sql:2 defines consultants separately from users, and :37 adds the consultant-record reference. Status values are validated at adminConsultingController.js:534. These are existing capabilities; bounded queue navigation, assignment integrity, and delivery handover need further work.

Concrete flaws and gaps

T24-1 - CONFIRMED DEFECT, high: dashboard discovery covers only the newest 200 requests. ConsultingDashboard.jsx:376 hardcodes that limit, discards the API total/offset, and searches/filters locally at :391. A request outside that slice cannot be found through the page's search, although the server already supports searching the full dataset. This is an operator completeness error rather than an absent backend search feature.

T24-2 - CONFIRMED DEFECT, high: assignment marks an inquiry scheduled without creating a reservation or recording appointment time. adminConsultingController.js:69 sets consultant_id and status = 'scheduled', while Scheduling updates consultant_assigned_id at :553. They are separate references with different schema meanings, not interchangeable field spellings. consulting_sessions_scheduled is a separate table (103_consulting_all_phases.sql:46), used by payment and completion. Assignment can therefore produce a scheduled dashboard record without a usable appointment.

T24-3 - DESIGN GAP, high: active-consultant listing is stronger than the assignment write. Listing limits candidates to active at adminConsultingController.js:109, but assignment checks only existence at :65. It does not reserve capacity, validate current active state, or condition on request version/status. A stale dropdown or direct request can bypass the selection criteria. No claim is made that current staff have done so.

T24-4 - DESIGN GAP, high: requests, assignment, and dashboard statistics use admin-role checks without a corresponding capability grant (consultingAdmin.js:7-19). The same stats response includes financial totals (adminConsultingController.js:598) while invoice reads correctly use view_financials. This creates an inconsistent financial-read boundary within consulting, requiring an explicit intake-versus-finance response policy.

Expansion failure scenario

A global campaign creates 4,000 inquiries. Staff search only the newest 200, then assign a consultant who has stopped taking work since the dropdown loaded. The request becomes “scheduled,” but payment cannot proceed because no scheduled-session record exists. Other teams interpret scheduled counts as booked capacity and plan staffing against inaccurate demand.

Required changes and operator design

Connect existing server search/paging to the dashboard. Define one canonical consultant identity contract and an assignment command with expected version, active eligibility, capacity check, and audit. Assignment should create an assigned intake state; scheduling should use the reservation service. Separate operational aggregates from protected financial totals. Introduce contact tasks and next-contact deadlines, linked to appointment and delivery records without duplicating their services.

Operator design recommendations

Retain the current dashboard and detail modal, but put team/market scope, intake date range, and snapshot time in the header. Replace blended conversion figures with pending unassigned inquiries, first contact overdue, assigned awaiting booking, and confirmed appointments. Define contact overdue from an explicit contact deadline; “scheduled” must mean a canonical reservation exists.

The queue needs inquiry ID, client/business, tier, help areas, age, contact state, assigned specialist, eligibility warning, next contact date, and appointment link. Move search and tier/status filters to the server; add language, specialty, assignment, and contact deadline. Save “Unassigned: specialist needed” and “Contacted: booking missing” views across the full dataset.

The drawer should show intake answers, contact attempts, consultant eligibility/capacity, assignment history, and linked scheduling/payment/delivery status. Mark contacted requires a recorded attempt or reason; assign previews specialist fit and current workload. Archive previews unfinished contact and booking tasks and requires a reason. Allow bulk task assignment, not bulk scheduling or irreversible closure.

On a capacity/version conflict show refreshed candidates and preserve the operator's notes. A committed assignment followed by communication failure stays assigned with a delivery-retry task. Empty means no matching inquiries, with clear filter reset; no access identifies intake capability; unavailable consultant data leaves the inquiry readable but disables assignment. Mobile cards prioritize overdue contact and booking gaps. Replace clickable-only actions with labeled keyboard buttons, use logical drawer placement for RTL, and isolate client email and appointment IDs from surrounding translated text.

Acceptance tests and owner

Consulting Intake Operations is accountable; Consulting Workflow Engineering owns the record contract. Find an inquiry older than the newest 200 through search and saved views. Assignment must not increment confirmed appointments until booking succeeds. A consultant deactivated after list load cannot receive an assignment; competing assignments return a version conflict. An intake-only admin must not receive financial totals. Verify assignment-to-booking-to-payment-to-delivery identifiers remain consistent through the complete synthetic workflow, with contact and archive reasons retained.

25. Consulting Pay

Route and business purpose

/admin/consulting/payments combines package configuration, purchases, invoices, and manual billing. It should distinguish package price, purchased entitlement, collected money, invoice delivery, and refund outcome. These concepts currently share a screen but are not one interchangeable accounting record.

Verified current implementation

client/src/admin/pages/ConsultingPayments.jsx:205 has abortable loading. Current invoice generation and package writes require manage_operations at server/routes/consultingAdmin.js:24 and :30; financial reads require view_financials at :26-28. Consulting refunds now require issue_refunds at server/routes/consultingPayment.js:18, repairing the prior role-only allegation. server/controllers/consultingPaymentController.js:54 checks payment ownership; :107 uses a deterministic provider key. :186 validates paid amount/currency before completion. Migration 622_audit_remediation_ws3.sql:19 establishes the active-payment uniqueness backstop; 599_consulting_payment_terminal_integrity.sql:10 protects refunded state. server/__tests__/ws3-consulting-chat.test.js:90 covers intent idempotency and guarded insertion.

Concrete flaws and gaps

T25-1 - CONFIRMED DEFECT, high: manual invoice recipient identity is transient. server/controllers/adminConsultingController.js:664 accepts client name/email, but the INSERT at :689 stores neither those fields nor a linked user/purchase. They appear only in the immediate response/email. Later listing at :643-648 derives identity through ci.user_id; a manual invoice created by this path cannot recover its recipient from that relation. This is loss of billing context, not merely a cosmetic missing column.

T25-2 - CONFIRMED DEFECT, medium: purchase and invoice history are silently capped. adminConsultingController.js:627 limits purchases to 100 and :650 invoices to 200 with no continuation. ConsultingPayments.jsx:211-212 loads those collections as the screen's history. An older unpaid invoice or disputed purchase becomes inaccessible through this page.

T25-3 - CONFIRMED DEFECT, high: deactivating a package does not prevent payment by known package ID. Public selection checks activity at consultingPaymentController.js:13, but intent creation selects by ID alone at :80-83, then uses its price. The administrator's inactive toggle is therefore a discovery restriction, not an enforced sales restriction.

T25-4 - DESIGN GAP, medium: package writes lack server price/unit validation and synchronized field semantics. adminConsultingController.js:783-792 accepts base price directly and maps session_count into duration_hours. Migration 109_consulting_payments.sql:19 initially copies legacy name/price into newer columns, while purchase history selects legacy package name at adminConsultingController.js:622. Editing package name/base price does not establish that every consumer sees the same commercial revision.

Expansion failure scenario

Consulting sales issue thousands of manual invoices and retire an obsolete package. Older invoices drop off the page, recipient information cannot be reconstructed from invoice rows, and a cached package identifier still creates payment intents. Finance cannot reliably reconcile invoice-to-customer-to-entitlement across the enlarged program.

Required changes and operator design

Persist invoice recipient snapshots and package revision/currency; require an idempotency identity and reject same-key/different-payload reuse. Page full purchase/invoice history with customer/status/date search. Validate effective package activity and integer cents during intent creation. Define sessions versus hours separately, synchronize commercial fields or expose one canonical schema, and append package release history. Reuse existing provider settlement/refund safeguards and link invoice delivery failures to an accountable retry task.

Operator design recommendations

Retain the three working areas but give Packages, Purchases, and Invoices independent loading and access states; the current combined Promise.all makes one unavailable collection block everything. The header should declare consulting currency, billing period, and each collection's snapshot time. KPIs should distinguish collected net funds, unsettled provider attempts, unpaid invoices past due, and remaining purchased sessions; define them from authoritative records, not visible rows.

Package rows need ID/revision, name, active selling state, integer-cent price/currency, included session count, duration, and effective date. Purchase rows need client, entitlement, collected amount, usage balance, and refund state. Invoice rows need durable recipient, number, issued/due dates, balance, and delivery outcome. Add exact invoice/payment/customer search, package and due-age filters, and saved “Overdue: undelivered” and “Refund pending” views.

A drawer should join recipient snapshot, package terms, purchase allocation, provider events, invoice revisions, and delivery attempts. Package edits preview commercial changes; manual invoice creation previews recipient and amount before reserving its request ID. Refund confirmation uses freshly read provider settlement and remaining capacity. Bulk billing is a reviewed job with per-recipient preview; bulk refund is excluded.

On timeout, recover the original invoice/refund by request identity. On package conflict retain the draft against the new revision. An empty invoice filter explains the due-date window; no-access states separately identify finance-read and package-write grants; delivery/provider unavailable leaves persisted billing evidence readable. Mobile places recipient/amount/due state first, with labeled tab controls and keyboard dialogs. RTL keeps invoice numbers and currency amounts isolated while retaining translated billing labels.

Acceptance tests and owner

Consulting Finance Operations owns billing truth; Consulting Payments Engineering owns package and invoice contracts. Create a manual invoice and reload it: recipient and package terms must survive. Find invoice 201 and purchase 101 through paging. An inactive package must reject intent creation even by direct ID. Negative/fractional cents and ambiguous sessions-versus-hours writes must fail validation. Concurrent invoice replay produces one durable invoice and rejects changed payload. Retain existing refund-grant and provider amount/currency tests; add separate-screen unavailable-state tests so a package outage does not hide invoice evidence.

26. Scheduling

Route and business purpose

/admin/consulting/scheduler is the staff calendar and pending-booking workbench. It should reserve an eligible consultant's actual time, communicate a confirmed appointment, and preserve rescheduling history. A date placed on an intake record is insufficient to prove a calendar reservation or entitlement to paid service.

Verified current implementation

client/src/admin/pages/ConsultingScheduler.jsx:338 loads requests and consultants with cancellation; :109 converts selected local date/time to an ISO timestamp. Availability and booking APIs already exist under /api/scheduling (server/routes/consultantScheduling.js:7-19). server/controllers/consultantSchedulingController.js:251 checks session ownership for non-admin booking, :260 applies consultant platform approval for non-admin callers, and :280 checks available slots. Its conditional write at :304 checks overlapping scheduled sessions. Migration 107_consultant_availability.sql:68 defines available-slot calculation; 103_consulting_all_phases.sql:46 supplies scheduled records. These capabilities should be integrated, not rebuilt as another calendar.

Concrete flaws and gaps

T26-1 - CONFIRMED DEFECT, high: the admin scheduler bypasses the existing booking service. Its submit at ConsultingScheduler.jsx:110 PATCHes request status and scheduled_at, handled by adminConsultingController.js:551-557. That path performs no availability/overlap check and creates no scheduled-session record. The guarded /scheduling/.../book-slot path exists, but this screen does not call it.

T26-2 - CONFIRMED DEFECT, high: navigating weeks does not query the requested week. ConsultingScheduler.jsx:340 loads the newest 200 inquiries once; :356-358 merely changes local week state. An older intake with a current appointment can be absent from the calendar. There is no calendar-range response establishing completeness.

T26-3 - CONFIRMED DEFECT, medium: “Send calendar invite” sends the reminder handler. server/routes/consultingAdmin.js:41 routes /send-invite to sendSessionReminder; adminConsultingController.js:826-830 builds a plain HTML reminder, with no calendar artifact in that invocation. The button's outcome is misnamed. It also reads persisted session fields, not unsaved modal values (ConsultingScheduler.jsx:126-129).

T26-4 - DESIGN GAP, medium: time interpretation follows the operator's browser rather than an explicit appointment timezone (ConsultingScheduler.jsx:109, :247). Meanwhile booking defaults to America/New_York at consultantSchedulingController.js:315. Those are different contracts. A global dispatcher can schedule the wrong instant even when each conversion is locally valid. Real multi-transaction overlap safety requires database concurrency verification; the NOT EXISTS check alone is not certification.

Expansion failure scenario

Dispatchers in New York and Dubai schedule the same specialist during a peak week. Both admin writes succeed because they bypass reservation checks. Their local time inputs refer to different instants, and the calendar may omit a client whose original inquiry is older than the latest 200. Customers receive reminder emails marketed as invitations without a durable booking.

Required changes and operator design

Add a bounded calendar-range API based on canonical reservations, then route staff booking/rescheduling through the existing slot service with appropriate operational authority. Store timezone, UTC instant, duration, consultant user/record identity, and parent session ID explicitly. Enforce overlap invariants across simultaneous transactions. Implement versioned reschedule/cancel commands with communication outbox and a real calendar artifact; otherwise relabel the action reminder. Keep ownership, approval, and available-slot checks, extending documented admin overrides with reason and history.

Operator design recommendations

The header should expose calendar timezone, week/date range, consultant/team scope, and reservation snapshot time. Replace request-count cards with unbooked inquiries awaiting scheduling, confirmed hours/capacity, conflicts needing action, and invitations not delivered. A reservation counts confirmed only when the canonical record exists.

Use consultant columns or lanes in the week view, plus an accessible agenda alternative. Each event shows client, start/end, timezone, booking status, payment prerequisite, and invite outcome. The unscheduled queue shows inquiry ID, preferred dates, language, tier, eligible specialists, and waiting age. Server filters should cover consultant, region, booking status, date range, and missing payment/communication; save “Next 48 hours: invite missing” and “Unbooked: preferred date passed.”

The scheduling drawer should show client preference, consultant availability, eligibility, entitlement, proposed slot, and reschedule history. Pick an actual returned slot instead of a free-form time alone. Confirmation previews client/consultant timezone renderings, duration, replaced reservation, and invitation recipient. Bulk task assignment and reminder preparation are allowed; bulk booking without individual slot reservation is not.

On slot conflict refresh alternatives without discarding contact notes. A booking committed but email failed remains confirmed with a retryable invitation task. Empty weeks say no reservations in that range; no-access distinguishes calendar read from scheduling authority; availability unavailable disables confirmation while retaining existing appointments. Mobile defaults to agenda, not a compressed seven-column grid. Events become keyboard buttons with readable time ranges, translated weekday labels, and deliberate RTL chronological navigation.

Acceptance tests and owner

Consulting Dispatch owns schedule policy; Scheduling Engineering owns reservations. An intake outside the latest 200 must appear in its appointment week. Two concurrent bookings of an overlapping slot must yield one reservation and one typed conflict. Test DST gaps/repeated times and dispatchers in different browser zones: persisted UTC and client rendering must agree. A real invitation contains the confirmed slot and calendar artifact; failed delivery remains visible without cancelling booking. Verify staff scheduling creates the scheduled record required by payment and completion.

27. Business Launch

Route and business purpose

/admin/consulting/business-launch manages founder intake, implementation phases, local consultant referrals, notes, and customer communication. It should explain what setup work is committed, who owns each deliverable, and whether the founder is waiting for a consultant, a document, or a decision. Referral estimates are not proof of collected money or regulatory completion.

Verified current implementation

client/src/admin/pages/BusinessLaunchAdmin.jsx:105 performs abortable server paging with country, state, urgency, package, status, and search. server/routes/businessLaunch.js:57-82 protects list, detail, status, and assignment with manage_operations; messaging separately requires message_users at :111. server/controllers/businessLaunchController.js:364 locks the request during assignment; :404 checks referral capacity and :409 minimum package value. :451 records referral history. Status changes write an audit row transactionally at :558. Migration 248_consultant_contractor_network.sql:5 defines coverage, capacity, and revenue-cut fields. Existing notes and request detail are real workflow foundations.

Concrete flaws and gaps

T27-1 - CONFIRMED DEFECT, high: the joined list query uses ambiguous columns. businessLaunchController.js:204-216 selects unqualified email, created_at, and updated_at; the FROM at :217-219 joins users and contractor profiles, which share those names. Search at :185 similarly leaves first name/last name/email unqualified. PostgreSQL cannot resolve these against the joined relations. The separate count query can succeed while the actual list fails. Contractor timestamps are explicitly present in 248_consultant_contractor_network.sql:24.

T27-2 - DESIGN GAP, high: assignment locks one request, not shared consultant capacity. businessLaunchController.js:374-385 reads the consultant without a capacity lock, then counts active referrals at :396. Two transactions assigning different requests can both observe one remaining slot. The existing capacity check is valuable but does not serialize this shared-resource decision.

T27-3 - DESIGN GAP, high: status updates validate values but not transitions or content version. businessLaunchController.js:521 defines allowed statuses; :535 reads without locking and :551 updates by ID alone. Staff can advance to completed without checked phase evidence or overwrite a concurrently cancelled state. Audit is present; it records the command without guaranteeing the prerequisite business outcome.

T27-4 - CONFIRMED DEFECT, medium: CSV export covers the current page while the UI displays total matching requests. BusinessLaunchAdmin.jsx:146 maps only requests, a 20-row page from :106. Quotes are escaped at :157, but spreadsheet formula prefixes are not neutralized. This is a source-level export safety/completeness limitation, not an allegation about an actual exported malicious row.

Expansion failure scenario

A founder program opens in twelve countries. List retrieval fails on its joined projection; assignments arriving through other flows overfill a popular specialist. A stale quick-status action marks a request completed despite outstanding launch tasks. A manager exports twenty visible rows believing they represent the whole selected program.

Required changes and operator design

Qualify projection/filter columns and share predicates between count/list. Reserve consultant capacity transactionally with an agreed eligibility policy. Add phase prerequisites, expected versions, and explicit reversal/archival transitions. Provide safe paged/full export jobs with clear scope and CSV neutralization. Connect referral history, delivery tasks, consultant capacity, messaging outcomes, and package financial records; label cuts as estimates until settlement exists.

Operator design recommendations

Preserve the existing country/state filters but display authorized market scope, query snapshot, and program/package revision in the header. KPIs should count unassigned founders, first-contact deadlines missed, kickoff booked, blocked implementation deliverables, and ready-for-launch requests. Derive readiness from accepted required deliverables rather than a manually selected completed status.

Queue columns should include founder/business, request ID, market, package, urgency, phase, consultant acceptance, next milestone/deadline, blocking reason, and last customer response. Add assignment/capacity, overdue milestone, missing evidence, and language filters. Save “Kickoff pending: referral not accepted” and “Implementation blocked: founder response needed.” Export must offer explicitly labeled current page or complete filtered job.

The drawer should have intake and scope, phase checklist with evidence, consultant referral/cut preview, founder communications, internal notes, and decision history. Assignment previews live capacity, coverage fit, and estimated cut; phase completion previews required deliverables and unresolved blockers. Archive requires a disposition and previews live tasks. Bulk assignment of operations tasks is permitted; consultant referrals and final completion remain individually reviewed.

On capacity conflict return refreshed matched consultants. On stale phase changes show new evidence/status and preserve the draft note. Message failure creates a retry task without changing implementation completion. Empty says no founders match market/phase; no access distinguishes operations from message capability; unavailable referral matching leaves intake and notes readable. Mobile uses a milestone card and full-screen checklist. Ensure keyboard checklist navigation, clearly separate internal/customer notes, and isolate country codes and monetary estimates in RTL.

Acceptance tests and owner

Founder Program Operations owns phase acceptance; Contractor Operations owns capacity. Run actual PostgreSQL list/search fixtures with users and profiles: no ambiguous-column errors and matching totals. Concurrent assignment at one remaining slot must admit exactly one request. Completion without required accepted evidence must fail. A stale status update returns 409 and preserves audit truth. Export all filtered rows with explicit count, neutralize formula prefixes, and verify twenty-row page exports are labeled. These tests are proposed; no dedicated Business Launch end-to-end proof was established by inspected test source.

28. Consulting Sessions

Route and business purpose

/admin/consulting/sessions oversees consulting delivery, completion, no-shows, notes, and follow-up. Operators need a consistent view of promised appointment, actual attendance, deliverables, customer action items, and purchased entitlement consumption. The same client may also have founder-program work; that dependency should be linked rather than conflated with this session's completion.

Verified current implementation

client/src/admin/pages/ConsultingSessionsAdmin.jsx:223 provides abortable server paging/search. Completion and cancellation use confirmation dialogs at :244; failures are rethrown to keep confirmation open. client/src/admin/pages/ConsultingSessionDetail.jsx:27 resets state when sessions change, repairing cross-session note/consultant bleed. server/routes/consultingAdmin.js:33-40 exposes list/detail, completion, no-show, and communication. server/controllers/adminConsultingController.js:191 lists the main session table; :267 exposes detail. Scheduled attendance is separately stored in server/migrations/103_consulting_all_phases.sql:46, and follow-up tracking is extended by 106_consulting_session_followups.sql. Existing action-item integration should remain.

Concrete flaws and gaps

T28-1 - CONFIRMED DEFECT, high: completion changes a different table from the one displayed. adminConsultingController.js:403 finds a scheduled record, and :415 updates only consulting_sessions_scheduled. The list at :232 still reads consulting_sessions, so a successful complete action can reload a still-scheduled row. No-show at :460 has the same split. Sessions created manually at :501 without a scheduled record cannot complete through this handler at all.

T28-2 - CONFIRMED DEFECT, high: the detail assignment dropdown returns consultant record IDs but writes a user UUID field. Consultant listing projects c.* at adminConsultingController.js:104; ConsultingSessionDetail.jsx:50-51 sends the selected value as consultant_assigned_id. Its options must be reconciled with consulting_sessions.consultant_assigned_id, defined as UUID referencing users (102_consulting_sessions.sql:49). The intake assignment path uses the separate consultant_id reference. These identifiers require explicit normalization.

T28-3 - DESIGN GAP, high: completion/no-show lack current-state conditions and delivery evidence. UPDATE predicates at adminConsultingController.js:417 and :463 are only scheduled-record ID. Generic detail writes at :316 also accept status without the validated transition contract used by another handler. Confirmation guards user intent but cannot prevent stale attendance changes or prove delivered service.

T28-4 - DESIGN GAP, medium: follow-up claims have no delivery history in this command. adminConsultingController.js:845-860 sends a generic email without an idempotency identity or persisted follow-up attempt; no email still reaches the success response. Completion promises an automatic follow-up at :429, while this admin view exposes no accountable delivery outcome or retry task. Existing follow-up workers are not presumed absent; their integration into this screen remains unproven.

Expansion failure scenario

Several hundred specialists finish sessions after a global training campaign. Attendance writes succeed, but the main list remains scheduled. Supervisors mark the same items again, and support sends duplicate follow-ups. A manually created session has no scheduled record and fails completion. Purchased entitlement usage and customer communications cannot be reconciled from this screen's status.

Required changes and operator design

Use one versioned attendance/completion command that synchronizes canonical session and reservation state transactionally. Require permitted source state, actual attendance/deliverable evidence, and entitlement linkage. Normalize consultant record/user IDs in API responses and writes. Persist communication attempts with stable intent IDs and retry status. Keep the existing per-session state reset and confirmation dialogs, adding detail freshness rather than rebuilding them.

Operator design recommendations

The header should display delivery team, scheduled/completed date basis, and snapshot freshness. KPIs should count sessions due today, attendance undecided after end time, completed with outstanding deliverables, no-shows awaiting disposition, and failed follow-ups. All derive from reconciled reservation/delivery records rather than the main table's status alone.

Rows need session/client, consultant, tier, scheduled interval, attendance, deliverable completion, entitlement usage, next action, and communication outcome. Add filters for specialist, appointment date, attendance pending, deliverable overdue, and follow-up failed alongside current search. Save “Ended: attendance missing” and “Completed: action items outstanding.”

The drawer should separate appointment/payment prerequisites, attendance evidence, deliverables/action items, internal notes, customer-visible summary, and communication history. Completion requires an outcome and delivered-item checklist; no-show requires attendance evidence and customer disposition; cancellation previews appointment and financial dependencies. Consultant reassignment previews identity and reservation consequences. Allow bulk follow-up task assignment or preparation, while completion/refunds remain individual evidence-based commands.

A split-record conflict shows both authoritative states and a reconciliation task. A failed completion retains its evidence draft; a delivery timeout retrieves the previous communication intent rather than sending again. Empty means no sessions for the selected delivery window; no-access distinguishes viewing from attendance commands; payment/communication unavailable preserves delivery notes and blocks dependent actions. Mobile focuses on today's appointments and outstanding deliverables. Use keyboard-accessible tabs and confirmation focus management; keep time intervals and consultant IDs stable in RTL.

Acceptance tests and owner

Consulting Delivery Operations owns attendance/deliverable policy; Workflow Engineering owns reconciliation. Completion must update list, reservation, and entitlement view consistently, exactly once. A manual session must be scheduled or return an explicit prerequisite error before completion. Test consultant record-to-user mapping with realistic distinct IDs. Concurrent complete/no-show commands must yield one accepted transition and one 409. Follow-up without a recipient must not report delivery; replay must not duplicate email intent. Confirm existing per-session state reset still prevents notes from leaking between drawers.

29. Org Chart

Route and business purpose

/admin/corporate/org-chart should let leadership and People Operations understand accountable reporting relationships across countries. Its current implementation is a department/title directory, not a reporting-line graph. That distinction matters when finding the person responsible for a country incident or an unstaffed department.

Verified current implementation

The API already requires both an administrator and manage_admin_accounts; it is not an unguarded employee directory (server/routes/corporateRoles.js:44). The controller includes global assignments when a country is selected, returns reporting-manager IDs, groups assignments by department, and also returns the entire flat dataset (server/controllers/corporateRolesController.js:787, server/controllers/corporateRolesController.js:792, server/controllers/corporateRolesController.js:809). The UI instead groups by seniority (client/src/admin/pages/super/CorporateOrgChart.jsx:41). Migration 246 stores manager references and assignment start/end timestamps (server/migrations/246_corporate_hierarchy.sql:485). Corporate assignment data also feeds effective corporate permissions; this screen must not imply that a displayed title is equivalent to an admin grant.

Concrete flaws and gaps

- CONFIRMED DEFECT: country requests can resolve out of order. Every keystroke changes the query, but load has neither cancellation nor a request-generation check before setChart; an earlier country's response can overwrite the newer selection (client/src/admin/pages/super/CorporateOrgChart.jsx:116, client/src/admin/pages/super/CorporateOrgChart.jsx:133, client/src/admin/pages/super/CorporateOrgChart.jsx:153).
- DESIGN GAP: reporting edges are available but unused. Seniority groups do not establish who manages whom, and the page provides no manager traversal or orphan-manager exception view (client/src/admin/pages/super/CorporateOrgChart.jsx:41, server/controllers/corporateRolesController.js:792).
- DESIGN GAP: “active” means assignment active only. The query does not filter inactive users, titles, departments, or future assignment starts; total_assignments is not distinct employee headcount (server/controllers/corporateRolesController.js:784, server/controllers/corporateRolesController.js:790, server/controllers/corporateRolesController.js:831). Whether background lifecycle processes keep these flags synchronized is UNVERIFIED; the read path itself makes no such guarantee.
- CONFIRMED DEFECT: initial API failure can display both an error and the empty-directory state, because null chart data enters the same branch as zero assignments (client/src/admin/pages/super/CorporateOrgChart.jsx:159, client/src/admin/pages/super/CorporateOrgChart.jsx:169).

Expansion failure scenario

With 3,500 employees holding multiple country/global assignments, a director switches markets during an incident and sees a stale directory under the new country label. A seniority grouping suggests authority but cannot reveal that the responsible manager has left. Duplicate assignments inflate apparent coverage while an operational reporting gap remains hidden.

Required changes and operator design

Extend GET /org-chart with department/search/cursor options, explicit snapshot time, distinct-person and assignment counts, and lifecycle exceptions. Provide bounded manager-subtree retrieval rather than transmitting both full grouped and flat copies. Define whether inactive/future assignments belong in an exception view. Sequence client requests; label retained results stale after failure. Keep assignment mutation in the separately governed Titles workflow.

Operator design recommendations

Use a header containing selected country, whether global assignments are included, department scope, and the server's snapshot time. Offer two explicitly named modes: “Reporting relationships” and “Title directory.” This resolves the present seniority-only visualization without pretending that incomplete reporting data can produce a trustworthy tree. KPI cards should show distinct active people, active assignments, positions without a manager, and inactive-person assignments; define their denominators in tooltips.

Place a searchable country selector and department/person/title filters above a bounded directory. Columns should be person, title, department, assignment geography, named reporting manager, effective start, and

lifecycle exception. Save views such as “Country leadership including global roles” and “Manager references needing repair.” In graph mode, expand one manager subtree at a time and include a text-table equivalent, not a mandatory pan-and-zoom canvas.

Selecting a person opens a drawer with all assignments, reporting chain, subordinate counts, source validity warnings, and recent assignment changes. Row actions are view manager, open assignment history, and request a correction through Corporate Titles. Bulk actions may create reconciliation tasks for selected exceptions, with a preview of assignment IDs and assigned People Operations reviewer; they must not silently change managers or permissions.

If a country response is stale, freeze its scope label and offer reload. Empty means no matching valid assignments; unavailable means directory retrieval failed, never “no employees.” No-access should explain the restricted directory capability without revealing employee records. On mobile, show a searchable reporting-chain list. Keep names readable in either direction, use logical indentation in RTL, and expose department expansion state to keyboard and screen-reader users.

Acceptance tests and owner

People Operations owns reporting-data definitions; Corporate Identity Engineering owns implementation. Resolve country responses in reverse order and require the final selection's data and label to agree. Seed multiple assignments for one person and verify distinct headcount versus assignment count. Cover a future start, inactive user, missing manager, and cross-country/global manager. Initial 503 must render unavailable without an empty-workforce assertion. A 3,500-person fixture must support bounded directory/subtree retrieval and keyboard traversal. These are proposed acceptance tests, not measured production capacity.

30. Corporate Titles

Route and business purpose

/admin/corporate/assignments administers internal titles, their geographic scope, reporting managers, and permission overrides. It is an access-affecting personnel workflow, not merely an org-chart label editor. Corporate permissions and ordinary admin capability grants are explicitly independent (server/services/corporateRolesService.js:13); operators need both meanings explained.

Verified current implementation

Assignment writes and overrides require manage_admin_accounts (server/routes/corporateRoles.js:38, server/routes/corporateRoles.js:41). Creation checks an active internal-staff target, active title, C-suite restrictions, and caller seniority, then writes an audit record inside its transaction (server/controllers/corporateRolesController.js:413, server/controllers/corporateRolesController.js:449, server/controllers/corporateRolesController.js:504). Updates lock the current assignment (server/controllers/corporateRolesController.js:541). The current list uses abortable requests and explicit end-assignment confirmation; those features exist (client/src/admin/pages/super/CorporateAssignments.jsx:295, client/src/admin/pages/super/CorporateAssignments.jsx:321). Migration 246 supplies assignment and override persistence rather than a client-only role registry.

Concrete flaws and gaps

- CONFIRMED DEFECT, access-sensitive: the override anti-escalation check only examines entries whose effect equals exactly allow, but persistence converts every non-deny value into allow. An authorized lower-level admin can submit an unsupported effect for a known permission they do not hold, bypass the intended check, and persist an allow override, provided the other assignment-management guards pass (server/controllers/corporateRolesController.js:715, server/controllers/corporateRolesController.js:743). This concerns corporate permission escalation, not automatic possession of all ordinary admin grants.
- CONFIRMED DEFECT: assignment retrieval stops at 500, returns no continuation/total, and the page accepts that array as the complete list (server/controllers/corporateRolesController.js:364, client/src/admin/pages/super/CorporateAssignments.jsx:309). The 501st matching assignment cannot be reached through pagination.
- DESIGN GAP: manager validation checks existence only on creation; there is no self-reporting/cycle or active-staff validation, while update accepts a manager reference without repeating even that application-level check (server/controllers/corporateRolesController.js:480, server/controllers/corporateRolesController.js:578). The foreign key protects referential existence, not reporting integrity.
- DESIGN GAP: a row lock serializes writes but does not reject a stale editor. Update has no expected revision predicate and can overwrite a newer country, manager, or note (server/controllers/corporateRolesController.js:562, server/controllers/corporateRolesController.js:572).

Expansion failure scenario

Regional administrators concurrently reorganize several hundred positions. The capped list hides older assignments, stale forms reverse manager changes, and an unsupported override effect turns an intended anti-escalation boundary into a bypass. Org Chart then reflects a misleading structure while downstream corporate-gated functions use the altered permissions.

Required changes and operator design

Reject unknown override effects before any authorization or deletion, validate known codes atomically, and evaluate the validated canonical payload. Preserve existing seniority/C-suite safeguards. Add assignment revision checks, bounded cursor retrieval, manager-graph validation, and an effective-permission preview endpoint showing both corporate effects and separately managed admin grants. Do not silently replace unknown-code overrides with an incomplete set.

Operator design recommendations

The header should identify personnel scope, active-versus-ended mode, and last fetched revision. KPI cards should count distinct staff, active assignments, overrides expiring soon, and reporting-integrity exceptions, using server totals beyond the existing 500-row cap. Search staff identity alongside department, title, country/global scope, regional boundary, assignment status, and override presence. Saved views should include “Acting appointments expiring” and “Assignments with exceptional permissions.”

The list needs staff, title/seniority, department, geography, manager, start/end dates, override count, and latest actor. Open a drawer with personnel eligibility, assignment details, reporting chain, permission inheritance, and audit history. Its permission tab must show the proposed before/after effective corporate set and clearly separate ordinary admin grants. Conflicting assignments should be surfaced rather than hidden behind a single title badge.

Allow create, edit scope/manager, end assignment, and replace overrides only when the existing capability and seniority rules permit them. Every access-changing preview identifies newly allowed codes, removed denies, geographic changes, affected employee, and reason. C-suite controls should visibly require the super-admin path. Bulk manager transfer may be proposed with cycle validation and per-row results; bulk permission grants require a distinct reviewed workflow, not list checkboxes plus immediate save.

A stale drawer must present the competing revision and preserve the operator's proposed changes for comparison. Region-service failure must disable region selection with a retry notice, not look like no regions exist. Empty means no matching assignments; no-access must not reveal the staff catalog. Mobile should use labeled assignment cards with a full-screen permission comparison. Make allow/deny states textual, keyboard-selectable, and RTL-safe; never communicate privilege changes through color alone.

Acceptance tests and owner

Identity/Security Engineering owns the override fix; People Operations owns assignment policy. Submit ALLOW, null, and arbitrary effects as a non-super admin lacking the target permission: all must return 400 with no override or audit mutation. Retest valid allow denial and legitimate deny. Traverse 501+ assignments without loss. Reject self-manager, inactive external manager, and a two-person cycle. Concurrent edits must produce one success and a 409 requiring re-preview. Verify an ended assignment stops contributing corporate permissions after the documented cache-invalidation boundary. No dedicated corporate suite was identified in the inspected test filenames; these checks remain required, not claimed passing.

31. Logistics Ops

Route and business purpose

/admin/logistics is the operational entry point for sourcing, pooled procurement, purchase orders, shipments, warehouse receiving, claims, and compliance. It currently provides navigation and aggregate status counts, rather than a dispatcher work queue. Financial outcomes depend on the linked procurement, pricing, refund-case, and landed-cost workflows.

Verified current implementation

The server mounts this API behind authentication (server/index.js:525). Stats require the operations workspace, whose middleware rechecks active database approval and route-specific access for non-admin staff; admin roles explicitly bypass that workspace check (server/routes/logisticsAdmin.js:39, server/middleware/requireInternalLogisticsWorkspace.js:54, server/middleware/requireInternalLogisticsWorkspace.js:63). Pricing writes now require super-admin, and request admin flags derive from the authenticated role (server/routes/logisticsAdmin.js:28, server/routes/logisticsAdmin.js:122). The client has abortable loading and a retry state (client/src/admin/pages/logistics/LogisticsAdminDashboard.jsx:68, client/src/admin/pages/logistics/LogisticsAdminDashboard.jsx:97). Existing lifecycle validators and migration 605 protect purchase-order state transitions; server/__tests__/moneyStateIntegrityMigrations.test.js:67 checks the encoded graph. Buyer trade restrictions fail closed on evaluation error, but are not staff-scope controls (server/services/logisticsAccessService.js:39, server/services/logisticsAccessService.js:163).

Concrete flaws and gaps

- CONFIRMED DEFECT: “active receipts” includes every warehouse receipt, including closed ones. The displayed label is active, but the aggregate has no status predicate (client/src/admin/pages/logistics/LogisticsAdminDashboard.jsx:112, server/controllers/logisticsAdminController.js:43). Migration 247 explicitly gives receipts lifecycle status (server/migrations/247_oxsteed_logistics.sql:652).
- DESIGN GAP: all six stats queries are global, without country, lane, warehouse, time-window, or staff-assignment scope (server/controllers/logisticsAdminController.js:37). Workspace admission is present; regional segregation inside this endpoint is not. Whether global visibility is an approved business policy is UNVERIFIED, not an asserted unauthorized-data exploit.
- DESIGN GAP: status counts contain no deadline, ETA breach, exception age, owner, or recoverable next action. The dashboard cannot prioritize a customs hold over an ordinary in-transit shipment (client/src/admin/pages/logistics/LogisticsAdminDashboard.jsx:101, server/controllers/logisticsAdminController.js:40).
- DESIGN GAP: navigation hides only the super-admin control tile; operations versus compliance and admin-only downstream capabilities are not represented (client/src/admin/pages/logistics/LogisticsAdminDashboard.jsx:74, server/routes/logisticsAdmin.js:131). Existing server rejection remains important, but the landing page promises “full control” beyond some users' allowed workspace.

Expansion failure scenario

A regional dispatcher receives worldwide aggregate workload and navigates into inaccessible compliance screens. Closed receipts appear active while a shipment's customs deadline expires unnoticed. At six million users, procurement customer impact cannot be determined from status totals, and local operators cannot establish which incident deserves immediate escalation.

Required changes and operator design

Extend stats with approved staff scope, documented receipt/on-hand definitions, and snapshot time. Add a paged exception endpoint joining PO, shipment, compliance, warehouse, and claim references with an accountable team and deadline. Reuse existing state machines and compliance approval commands rather than introducing generic “set status” actions. Keep global administrative scope explicit, not accidentally inferred from buyer-policy code.

Operator design recommendations

The header should contain lane/market and depot scope, reporting interval, last refresh, and the operator's workspace. Replace the undifferentiated navigation grid with a compact capability-aware module menu and a primary “Needs intervention” queue. KPIs should count shipments past promised ETA, customs holds past review deadline, open receiving discrepancies, and unresolved claims past SLA. Label receipts as open receiving records or historical receipts according to the actual query; show on-hand inventory only from a stock model that accounts for outbound allocations.

Queue columns should include exception type, PO/shipment reference, supplier, origin/destination, current milestone, deadline/age, financial exposure currency, assigned operator, and blocking prerequisite. Filters cover lane, depot, exception category, supplier, priority, and assignment; saved views include “Customs deadline today,” “Receiving variance,” and “Supplier handoff overdue.”

A drawer should show the linked procurement-to-delivery chain, milestone evidence, customs documents, inventory variance, claim history, and customer impact. Row actions open the authoritative module, assign an investigation, request missing documents, or propose the next legal transition. Compliance release remains confined to the compliance route. Bulk assignment and reminder preparation require a selection preview and per-item outcomes; do not bulk clear customs or approve payments from aggregate cards.

On conflict, reload the milestone and explain which prerequisite changed. Partial module failure should mark that KPI unavailable rather than zero; retain unaffected queues with their freshness labels. Empty means no actionable exceptions in scope, while no-access explains the missing workspace. Mobile prioritizes deadlines and depot scanning links. Ensure status/priority text accompanies color, tables have readable headers, and RTL layouts preserve shipment references and origin-to-destination meaning.

Acceptance tests and owner

Global Logistics Operations owns queue definitions; Supply Chain Engineering owns delivery. Closed receipts must be excluded from the active count. Seed two regions and prove each scope's totals and queue agree with its approved access policy. Revoke a staff workspace while its session remains valid and require rejection. Operations-only users must see an accurate capability menu and be denied compliance release. A PO prohibited transition must remain blocked by existing guards. Mock one unavailable source and require an unavailable KPI, not zero. Measure exception-query latency on realistic shipment history; existing migration tests do not establish operational throughput.

32. Learn Submissions

Route and business purpose

/admin/learn/submissions reviews user-authored guides, community videos, and flagged videos. It is a publication and content-safety workspace: approved instructions and attached commercial tools reach the Learn catalog and connect to Product Links, AI Suggestions, and Tool Rentals. Guide review and video review have different current state models and must not be treated as equivalent.

Verified current implementation

List, detail, and decision APIs require authentication, admin access, and moderate_content (server/routes/learn.js:136). Guide lists have bounded pagination and totals, and a separate full-detail endpoint exists (server/controllers/learnController.js:417, server/controllers/learnController.js:436). The client already uses abortable list requests and retains data on retry (client/src/admin/pages/LearnSubmissionsAdmin.jsx:286). Submission persistence includes full instructions, license notes, and reviewer metadata (server/migrations/282_guide_submissions.sql:12). Video decisions have an existing conflict test, and a bounded 24-hour publication worker exists (server/__tests__/learnGuideVideos.test.js:297, server/controllers/learnController.js:1394); those features should be extended, not rebuilt.

Concrete flaws and gaps

- CONFIRMED DEFECT: the expanded guide receives the list summary, not the full-detail response. The list omits intro, steps, skills, tools, video, and license; expansion passes sub directly and never calls the available detail endpoint (server/controllers/learnController.js:405, client/src/admin/pages/LearnSubmissionsAdmin.jsx:94, client/src/admin/pages/LearnSubmissionsAdmin.jsx:724). Operators can approve without seeing the substantive instructions.
- CONFIRMED DEFECT: guide review neither locks/checks pending state nor requires a revision. It updates by ID alone, so concurrent or repeated approvals/rejections can replace a prior decision and approval can republish (server/controllers/learnController.js:473, server/controllers/learnController.js:488). The video conflict guard does not cover this guide path.
- CONFIRMED DEFECT: publication failure is swallowed behind a savepoint, yet approval commits and returns success with null published_guide_id; the client ignores that field and closes the review (server/controllers/learnController.js:502, server/controllers/learnController.js:508, client/src/admin/pages/LearnSubmissionsAdmin.jsx:305). An approved-but-unpublished recovery case is invisible.
- CONFIRMED DEFECT, latent detail-rendering path: tools iteration shadows translator t with a tool object, then calls that object for buy/rent labels (client/src/admin/pages/LearnSubmissionsAdmin.jsx:169, client/src/admin/pages/LearnSubmissionsAdmin.jsx:175). Once full tool data is supplied, linked tools trigger a render error.

Expansion failure scenario

At six million users, moderation reviews summary cards while unsafe steps remain unseen. Two reviewers race, a guide's decision changes after publication, and a publish failure looks completed. The 24-hour video deadline makes queue visibility urgent; adding employees without a reliable review/recovery model increases throughput of mistakes, not safe publication.

Required changes and operator design

Fetch full detail before enabling a guide decision. Add pending-state/revision enforcement, append-only decision records, and a durable publication intent with explicit failed/retry states and a stable submission-to-guide link. Correct the translator shadow. Preserve existing video and flag endpoints; add deadline and escalation fields to their queues, and test actual publication/decision concurrency rather than assuming mocked sequential tests prove it.

Operator design recommendations

Use separate Guide Review, Video Deadline, Video Flags, and Publication Recovery tabs. The header should identify review language/category and freshness. KPIs count pending guides, videos nearing the 24-hour publication boundary, unresolved severe flags, and approved guides lacking successful publication; “approved” must not be a synonym for “live.” Add filters for language, category, submitter, risk classification, review assignment, and deadline, with saved views for “Unreviewed videos due soon” and “Approved: publication failed.”

Guide rows should show title, contributor, category, submission age, assigned reviewer, current decision, and publication outcome. The detail drawer must fetch and display full ordered steps, introduction, video preview, tool destinations, attribution/license note, and prior review/publication events. A persistent decision panel should require the relevant safety/licensing review evidence and make the content revision being approved visible. Video and flag drawers should display the affected guide, report reason, current playlist placement, and publication deadline.

Allow individual approve/reject with reason, video block/approve, existing flag dispositions, and retry of a failed publication intent. Bulk assignment and escalation are suitable; bulk approval of unseen content is not. Preview destructive video deletion and guide movement with affected catalog references. Keep decisions disabled while detail retrieval fails.

On 409, show the new decision and preserve an unsent note; on publication failure, leave a recoverable case rather than closing as success. Empty queues should name the selected content type and filters. No-access explains moderation scope without exposing submitter identities; unavailable detail must not look like an instruction-free guide. Mobile uses a full-screen readable step viewer and sticky decision controls. Keyboard focus, caption/transcript access, and bidirectional step numbering are necessary for multilingual moderation.

Acceptance tests and owner

Content Safety owns review policy; Learn Engineering owns publication integrity. Assert expanding a summary fetches full detail and approval stays disabled until it succeeds. Render linked tool objects without errors. Race approve/reject and replay approve: one decision and at most one catalog publication must survive. Inject catalog failure and require a visible recovery case with successful idempotent retry. Preserve existing video conflict behavior and test worker/block overlap with a real transactional database. Accessibility acceptance includes keyboard review of a long RTL guide with captions and preserved draft notes.

33. Learn Product Links

Business purpose and observed implementation

This registry changes the buying and rental destinations attached to learning guides without publishing a new mobile application. It is a commercial-content control, not an inventory reservation or evidence that a partner still sells the item. Moderation permission gates its endpoints (server/routes/learn.js:162). The admin UI supports link types, activation, guide/global overrides, compliance indicators and, in the final source refresh, shared typeahead. The API currently returns the complete matching collection (server/controllers/learnController.js:616); its public counterpart returns active overrides and substitutes an empty collection on query failure (server/controllers/learnController.js:605).

Findings, expansion scenario and required changes

P1 confirmed validation flaw: affiliate-domain validation uses a regular-expression prefix rather than a parsed exact hostname (server/controllers/learnController.js:660). A hostname beginning with an allowed domain but continuing into an attacker-controlled suffix can satisfy that prefix. Update also validates selected changed fields, not the complete resulting record (server/controllers/learnController.js:701), so a type-only conversion can retain an incompatible old URL. Validate the merged record with URL parsing, approved protocols, exact host boundaries and a server-owned link-type contract. Do not equate a green client badge with legal compliance or availability.

P2 evidence and scale gaps: registry edits/deactivation/permanent deletion do not write an explicit admin audit event in these handlers. The permanent-delete option erases the record instead of preserving destination history (server/controllers/learnController.js:744). Bounded API search, pagination and revision-aware saves are required before thousands of guides and regional destination variants exist. The public empty fallback cannot distinguish a genuine empty registry from a failed refresh; clients need a versioned last-known catalog and an unavailable marker. A widespread bad override must be traceable to affected guides and reversible by a new version, not by deleting historical evidence.

Operator design recommendations

Use a destination-review queue, with saved views for invalid destinations, newly changed links, missing disclosures, inactive items still referenced and overdue partner verification. Filters cover guide ID, global versus scoped override, link type, merchant domain, country, locale, status and last verification time. Rows show the product name, exact parsed host, guide coverage, active state, disclosure requirement, destination version, verification age and accountable editor. Keep the fully expanded destination in a selectable text field; shortening the visible host can conceal the very error the reviewer needs to detect.

Selecting a row opens a two-column drawer: stored configuration on one side and the actual customer-facing guide card on the other. Include source guide, global fallback, both buy/rent destinations and disclosure text. Mark “destination valid,” “partner availability unverified” and “inventory availability” as separate facts. Test-link navigation must be an explicit external action with safe opener handling; it must not automatically fetch arbitrary destinations from the server.

The save preview compares old and new URLs, type, scope and activation, lists impacted guide IDs and requires a reason for destination replacement. Deactivation remains available during a partner incident, while permanent deletion is withheld where evidence retention applies. Conflicting edits display the committed version without overwriting the operator draft. A timed-out save shows its command reference and a reload/status action before resubmission. On mobile, prioritize the parsed host and impact count rather than hiding them behind an overflow menu. Announce validation errors and provide keyboard access to every disclosure check.

Acceptance, owner and closure evidence

Content Commerce owns the URL contract; Security owns adversarial hostname tests. Test suffix-host tricks, embedded credentials, unsupported protocols, invalid type-only conversions and conflicting edits. Test global/scoped precedence, old-version rollback, last-known refresh during outage and discovery of an item beyond the first page. Record one complete audited correction and verify web/mobile representations. Affiliate policy review belongs to the responsible business/legal owner; syntax tests cannot certify it.

34. Learn AI Suggestions

Business purpose and observed implementation

This is a human review queue for generated tool, material and service suggestions. It already provides approve/reject controls rather than allowing the model to publish everything. Lists validate status and return the newest 200 suggestions (server/controllers/learnController.js:956). Approval locks the suggestion and checks pending state; tools also lock the guide before merging (server/controllers/learnController.js:987). Rejection has a conditional pending predicate (server/controllers/learnController.js:1068). These are useful existing concurrency protections, not missing functionality to rebuild.

Findings, expansion scenario and required changes

P1 confirmed publication-contract gap: approved tool/material suggestions copy generated buy/rent destinations into the guide without the full Product Links URL/compliance validation (server/controllers/learnController.js:1034). Human approval does not make an unsafe protocol or misleading merchant destination safe. Apply the same server-owned destination rules to every publication path, including imported/generated records. For service suggestions, verify a real target guide and preview that approving the suggestion changes its category rather than merely attaching a service recommendation.

P2 queue/evidence gaps: there is no cursor or total in the 200-row response; an influx of recent suggestions can indefinitely hide old pending work. Stored reviewer/time is useful but not a complete revision/provenance history: retain model/config version, generating guide version, source record IDs, evaluated destinations, rejection reason and applied diff. Confidence is a model output, not a calibrated safety probability. Separate it from actual eligibility and evidence. Introduce duplicate clustering and revision staleness checks; a suggestion generated for an older guide must not silently overwrite a newer classification.

Operator design recommendations

Create a review station with three explicit views: pending review, blocked by validation and decided history. Queue columns identify guide, suggestion kind, proposed object, current guide revision, generation age, source provenance completeness and validation outcome. Filters include oldest first, category, language, model/config version, invalid destination and duplicate group. Show backlog totals and oldest age from the entire authorized predicate, not merely the newest 200 loaded rows. Route batches to named reviewers, but do not provide approve-all for mixed unverified suggestions.

The detail panel places the current guide beside the proposed exact change. For a service, show the category before/after and downstream discovery effect. For tools/materials, show the precise new item, normalized destinations, duplicate detection and whether it refers to Direct equipment or peer rentals. Expand model rationale as untrusted text; it cannot instruct the operator or application to bypass a check. Confidence is labelled “model-reported,” while independently checked facts appear in a separate evidence list.

Approve opens a preview with guide version, suggestion ID and permitted publication effect. Missing or unsafe destinations prevent publication and offer edit/reject/escalate paths. Reject requires a structured reason plus optional explanatory notes for future prompt evaluation. After another reviewer acts, disable stale commands and display the committed outcome and actor. A guide changed since generation remains reviewable, but approval requires a refreshed comparison. Mobile uses stacked current/proposed panels; screen readers receive named fields and decision results rather than color-only badges. Returning to the queue retains its cursor and skips only records actually resolved.

Acceptance, owner and closure evidence

Knowledge Products and AI Governance jointly own this queue. Prove competing decisions yield one publication, a failed guide update rolls back approval, unsafe generated URLs cannot publish, and an outdated guide revision conflicts. Seed more than 200 pending records and reach the oldest one. Test missing guides, duplicates, regeneration after rejection and five-language decision messages. Track false approvals, reviewer throughput and disputed recommendations; neither a confidence score nor a successful API response is a readiness certificate.

35. Super Dashboard

Route and business purpose

/admin/super/dashboard is headquarters' broad operating summary: user growth, job outcomes, revenue, support demand, activity, and geography. It should tell a director which domain needs investigation, with definitions clear enough to compare regions and periods. It is not a general ledger or evidence that financial providers are healthy.

Verified current implementation

The four summary/chart/activity/geography APIs require super-admin (server/routes/admin.js:174-177). Authentication checks current account activity, role, revocation, and enrolled MFA (server/middleware/adminAuth.js:43-71). The controller probes optional schema, counts users, queries jobs, and derives ledger revenue (server/controllers/superAdminController.js:177-199, :233-258, :268-284). The previous mixed-cohort completion-rate defect is fixed: completions and cancellations use the same created-at interval (:239-240, :358-363). The UI already provides refresh, a last-updated label, error retry, and attention cards (client/src/admin/pages/super/Dashboard.jsx:280-309). Those features should be extended, not rebuilt.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** failed subqueries become authoritative-looking zeroes. The core user query catches to {} at superAdminController.js:199; ledger errors are swallowed at :284, and support counts default to zero at :310-317. The page's green/current indicator follows the overall fetch error at Dashboard.jsx:287-292, so a successful degraded response can conceal an unavailable source.
2. **DESIGN GAP:** the corrected completion metric measures jobs created in the period that are currently finished, not all completions that occurred during the period. Its definition remains sensitive to old work finishing later (superAdminController.js:239-240). Worldwide operations need separately named creation-cohort conversion and completion-event throughput; this is not the historical denominator bug.
3. **CONFIRMED DEFECT:** revenue aggregation has no currency predicate or grouping (superAdminController.js:270-276). Money's later USD-key correction does not apply to this endpoint. A native multi-currency ledger is compressed into the dashboard's currency cards (Dashboard.jsx:317-348). Current live currency composition is **UNVERIFIED**.

Expansion failure scenario

A director reviews seven-day performance while a regional payment feed is delayed. Support-query failure appears as zero backlog, and native currencies increase a revenue total without an exchange-rate policy. Staff divert capacity based on a healthy-looking screen. The corrected job cohort does not prevent these separate interpretation failures.

Required changes and operator design

Return per-section status, asOf, definitionVersion, and source errors; represent unavailable measurements as null. Add currency-separated revenue and explicit event-time/cohort-time metrics. Connect attention items to Support, Payouts, Revenue, and incident records through stable filters.

Operator screen specification

Keep the current responsive card grid, but put period, territory, reporting currency, and refresh status in a sticky header. Each section needs its own timestamp and a definition popover: completion-cohort numerator/denominator, new-user exclusions, revenue source types, and support statuses. Display zero only for a successful empty aggregate. A failed ledger fetch should replace monetary cards with “Revenue source unavailable” and a retry action while user metrics remain usable.

Below the cards, replace count-only attention tiles with a bounded exception table: domain, severity, affected count, oldest age, responsible team, and linked work queue. Filters should include domain and threshold; saved views should include “Shift handover” and “Payment exposure.” Row actions open the authoritative domain with matching scope or create an incident; bulk actions may assign exception ownership, never change underlying payment/job states.

A KPI drawer should show the formula, selected cohort, numerator, denominator, exclusions, comparison period, and drill-down link. A source-error drawer shows a safe failure category and correlation ID. Preserve the last successful value with an explicit stale badge during retry, and prevent a failed period change from relabeling old data as current. On narrow screens, collapse geography/activity before priority exceptions; use logical RTL placement and keyboard-accessible card links. Announce refresh completion without

moving focus. No-access should explain the super-admin requirement without exposing counts; an empty attention queue should state which sources were successfully checked and which were not.

Acceptance tests and owner

Data Platform owns definitions; Operations owns triage. Inject a ledger or support query failure: its section must be unavailable, never zero/current. Add old cancellations to the corrected cohort fixture: seven-day conversion must stay unchanged. Seed USD/CAD rows: two native totals must appear. Exercise period-switch races and keyboard drawer closure. Benchmark bounded dashboard requests on a six-million-user synthetic dataset with a proposed p95 under two seconds, recording query plans and workload impact rather than inferring capacity from CI.

36. Money

Route and business purpose

/admin/super/money explains the platform's revenue channels and distinguishes actual revenue, commerce volume, affiliate outflow, and configured franchise economics. Finance and commercial leadership need this distinction before investing in a channel or treating pending sales as cash.

Verified current implementation

The read API uses view_financials (server/routes/admin.js:260). The component groups channels by category and suppresses obsolete fetch responses (client/src/admin/pages/super/MoneyPage.jsx:196-230). Ledger totals now group by source/currency, and headline ledger sums select USD (server/controllers/superAdminController.js:1693-1757). Subscription channel lookup also uses the corrected composite key (:2068-2071), covered by server/__tests__/moneyOverviewT36Repro.test.js:76-99. Marketplace is explicitly volume-only, affiliates are partner outflow, and franchise is configured economics (superAdminController.js:2103-2152); recommending that classification from scratch would ignore existing work.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** currency breakdown is calculated internally but not returned. The response at superAdminController.js:2176-2204 exposes neither ledgerBySource nor reportingCurrency. Ledger-only USD filtering fixes mixed headline ledger sums, but operators cannot inspect excluded CAD/EUR revenue from this screen.
2. **CONFIRMED DEFECT:** recent ledger movements omit currency from the SQL projection (:1710-1721) while the UI formats movement amounts through one cents formatter (MoneyPage.jsx:500). Native non-USD rows can therefore appear with the default money presentation even though the headline excludes them.
3. **DESIGN GAP:** period/lifetime slots carry incompatible meanings. API products repeat the current monthly snapshot in both fields (superAdminController.js:2120-2125); affiliates repeat the same commission flow (:2133-2138); pipeline combines pending consulting, commerce, and franchise commitments (:2186). Existing notes help, but these are not comparable earned-period or lifetime amounts.
4. **UNVERIFIED:** consulting's ungrouped sum is added to USD ledger totals (:1807-1813, :2040). This inspection does not establish that every consulting record is guaranteed USD, or that completed consulting receipts cannot also post to the ledger. Do not assert double counting without verifying those producers.

Expansion failure scenario

A country leader sees large marketplace volume and a recurring-product snapshot beside tracked revenue, then treats the pipeline as spendable income. Finance cannot explain excluded currencies or reconcile the latest movement's amount. This is a commercial decision risk even with correct ledger-key lookup.

Required changes and operator design

Extend /admin/super/money with native-currency channel rows, typed measurement kinds, coverage status, period boundaries, and source identifiers. Define revenue recognition with Finance; prove disjoint consulting/ledger inclusion. Link snapshots and payables to their owning modules instead of manufacturing a combined accounting balance.

Operator screen specification

Retain the existing four channel groups, but make their headers explicit: recognized receipts, transaction volume, partner liabilities, and contracted economics. Put period and native currency selectors above them with “USD only; other currencies excluded” when applicable. Each KPI must display its measurement type and source coverage. Pipeline should expand into three separate amounts with distinct qualification rules, rather than one undifferentiated spending signal.

The channel table needs source, current-period amount, prior comparable amount, count definition, pending amount, refund amount, and completeness. Provide search by channel/source and saved views “Recognized revenue” and “Commercial pipeline.” A row opens a drawer containing source tables, inclusion/exclusion rules, accounting timing, native-currency breakdown, and reconciliation links. Snapshot channels must say “current snapshot,” with no fabricated lifetime comparison. From a channel, allow opening its revenue, affiliate, subscription, or franchise workspace; configuration links should appear only

when the caller can use the destination.

Recent movements should include currency, source reference, signed amount, timestamp, and underlying receipt link. Allow a bounded export preview specifying rows, columns, scope, and completeness; no money-moving bulk action belongs here. A failed refresh retains the previous period label and shows “previous successful snapshot,” while a source unavailable state identifies the missing channel independently. Empty channels state “no qualifying records” only after a successful check. On mobile, turn each channel into a labeled card; use RTL-aware source/reference alignment, accessible category badges, and a keyboard drawer. Assign reconciliation discrepancies through the linked financial exception queue, with case status visible on return.

Acceptance tests and owner

Finance Analytics is accountable with commercial channel owners. Assert CAD ledger rows appear in CAD breakdown but never USD headlines; assert every movement preserves currency. Keep both existing subscription-key tests. Seed a monthly snapshot and verify it cannot display as lifetime revenue. Test consulting inclusion with producer-level fixtures before adopting a recognition rule. Verify pending franchise commitments remain distinct from cash, and read-only finance users receive accurate destination capabilities.

37. Restaurant Credit

Route and business purpose

/admin/super/restaurant-credit oversees merchant-specific customer wallets, pending loads, reservations, and account restrictions. Its central invariant is that a customer's balance at one restaurant does not become spendable at another. Operators need receipt-level investigation and safe holds/status decisions without conflating this rail with OxSteed Credit.

Verified current implementation

All reads/control routes require super-admin; writes now carry the Credit mutation limiter and responses are no-store (server/routes/admin.js:179-188). Overview wallets have bounded paging and request/hold counts (server/controllers/restaurantCreditAdminController.js:216-231). The backend now returns native currency aggregates (:300-325, :348-351), fixing the earlier absent API breakdown. Status changes lock the account, prohibit closed-wallet reopening, and make unchanged targets no-ops (:530-551). The page has customer/restaurant/status filters, keyboard-selectable rows, detail retry, approval/decline dialogs, and recent activity (client/src/admin/pages/super/RestaurantCreditAdmin.jsx:552-577). Existing overview tests are at server/__tests__/restaurantCreditAdmin.test.js:89-123.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** the new summary.currencyBreakdown is not consumed by the current page. The mixed-currency guard and summary cards remain at RestaurantCreditAdmin.jsx:521-544; there is no currencyBreakdown reference in that component. The backend improvement therefore has not yet supplied finance with a usable screen breakdown.
2. **DESIGN GAP:** history cannot be fully investigated. Detail queries independently cap ledger, holds, requests, transfers, audit, and payments (restaurantCreditAdminController.js:415-465); recent requests/activity are further sliced to eight/ten in the UI (RestaurantCreditAdmin.jsx:576-577). Wallet list pagination already exists, but it does not paginate these independent histories.
3. **DESIGN GAP:** locks serialize changes without checking the state the operator saw. updateWalletStatus accepts desired status and requires a reason only for suspension (restaurantCreditAdminController.js:516-526); its update has no expected version (:548-551). A stale reactivation can undo a newer investigative suspension.

Expansion failure scenario

A restaurant group requests an investigation of a receipt from nine months ago. Headquarters can locate the wallet but not navigate to the old entry. Meanwhile a second shift reactivates the same wallet from a stale drawer after Risk has suspended it. Accounting isolation may remain intact while the operator workflow still violates investigation intent.

Required changes and operator design

Expose the existing currency aggregates, add separate history cursors and reference filters to wallet detail, and require expected status/version on restriction changes. Require a reason for reopening, returning a 409 with refreshed state. Add restaurant-scoped review capabilities if this responsibility is delegated; current global super-admin control is a real boundary, not an implied merchant-scoped workforce model.

Operator screen specification

Keep the existing list/detail split. Place restaurant scope, wallet status, customer search, currency, and snapshot time in its header. Replace the ambiguous mixed-total cards with a per-currency liability strip: balance, reserved, available, pending-load amount, and active-hold amount. Clicking a currency applies the list scope; clarify that available is balance less reserved, not money available to headquarters.

Wallet rows should add reserved amount, pending-request count, and oldest active hold beside existing customer/restaurant/balance/status columns. Save views for “Expiring loads” and “Long-lived holds.” The detail panel needs separate Accounts, Ledger, Holds, Loads, Transfers, and Audit tabs, each with its own cursor and reference/date search. Opening an old receipt must keep the restaurant and customer visible; link related POS order and payment evidence.

Status dialogs should preview observed/current state, reason, affected active holds, and the precise wallet identity. Approval previews show original request amount/currency, requester, expiry, funding evidence, and whether reviewer independence is satisfied by the actual service policy. Release previews show what reservation will be released, not promise a refund. Allow row approval/decline/release/suspend/reactivate with server capabilities; initial bulk actions should only assign cases or export scoped history. A 409

refreshes the drawer and preserves the draft reason for reconsideration. Timeout recovery reloads the receipt before offering retry. Empty history should distinguish “no matching entries” from “recent sample exhausted”; no-access and unavailable detail must not show zero balances. Use a full-screen mobile drawer with focus return, logical RTL columns, and textual balance/status labels.

Acceptance tests and owner

Restaurant Payments owns invariants; Risk Operations owns review decisions. Render two currency aggregates from the existing API. Traverse 10,000 wallet entries to the earliest receipt through bounded requests. Race suspension and stale reactivation: require one success and one 409. Approval replay must create only one wallet effect. Validate restaurant A/B isolation through real service calls, and confirm inaccessible or failed detail loads never retain another account's actionable controls.

38. OxSteed Credit, overview and all thirteen subpages

Route and business purpose

/admin/super/oxsteed-credit and its thirteen children form the restricted stored-value control plane. They oversee issuance, customer balances, partner entitlements, reserves, disputes, payouts, cards, reconciliation, compliance, and financial evidence. Restaurant wallets remain a separate rail. A worldwide workforce needs responsibility-specific operational queues; globally promoting every reviewer to super-admin would defeat the current restrictive boundary.

Verified current implementation

The route inventory is client/src/admin/adminRouteManifest.js:113-126. Administrative APIs use super-admin, no-store, mutation limiting, and export limiting (server/routes/admin.js:359-419). Maker/checker decisions forbid self-approval and verify fresh checker step-up (server/services/oxsteedCreditPendingActionService.js:483-520); payout/withdrawal initiation also has maker step-up (server/routes/admin.js:426-491). Settlements carry expected status and reversal idempotency (client/src/admin/pages/super/oxsteedCredit/SettlementsPage.jsx:253-279). Append-only transactions, audit chains, settlement evidence, and governance are persisted in migrations 425_oxsteed_credit_transaction_immutability.sql, 467_oxsteed_credit_audit_hash_chain.sql, 486_oxsteed_credit_settlement_approval.sql, and 640_oxsteed_credit_governance.sql. These are substantial existing safeguards.

Current subpage coverage and specific residual work:

Subpage	Verified current source	Residual operator requirement
Overview	client/src/admin/pages/super/oxsteedCredit/OverviewPage.jsx:46-69; server/controllers/oxsteedCreditController.js:854-1013	Currency summaries and notification retry exist. Show the coverage time and link each failed notification to its receipt and retry history.
Approvals	client/src/admin/pages/super/oxsteedCredit/ApprovalsPage.jsx:41-107, :153-211	Financial and agent queues both request 50 rows. Add complete traversal, assignment, aged-first sorting, and decided/executed receipt history.
Disputes	client/src/admin/pages/super/oxsteedCredit/DisputesPage.jsx:71-81, :132-160	Paging, resolution, and account context exist. Add case ownership and evidence chronology; resolution should preview linked compensation and original transaction.
Settlements	client/src/admin/pages/super/oxsteedCredit/SettlementsPage.jsx:155-169, :253-279	Keep expected-state transitions and compensating reversal. Put recovery on the selected row rather than relying on the six-item recovery summary at :123.
Reconciliation	client/src/admin/pages/super/oxsteedCredit/ReconciliationPage.jsx:93-113, :132-154	Internal/external history paging exists. Add receipt-level finding ownership, progress/checkpoint visibility, and a resolved-versus-acknowledged distinction.
Compliance	client/src/admin/pages/super/oxsteedCredit/CompliancePage.jsx:60-93; server/services/complianceCaseService.js:211-240	The former fixed-zero verification/sanction offsets are fixed. Correct case filtering before paging and separate the three queues' cursors.
Reserves	client/src/admin/pages/super/oxsteedCredit/ReservesPage.jsx:33-53	Currency policies and maker/checker updates exist. Preview current/effective floor, partner override precedence, and affected payable coverage before submitting.

Subpage	Verified current source	Residual operator requirement
Partners	client/src/admin/pages/super/oxsteedCredit/PartnersPage.jsx:70-84, :97-103	Directory continuation and searched partner lookup exist. Separate directory, agreements, funding, and entitlements in detail; show scoped completeness for each.
Health	client/src/admin/pages/super/oxsteedCredit/HealthPage.jsx; server/services/oxsteedCreditWorkerHeartbeat.js	Existing rail/worker controls need run-to-incident links and a checklist of outstanding findings before unfreeze. Provider Health is separately main-owned.
Audit	client/src/admin/pages/super/oxsteedCredit/AuditPage.jsx:43-56, :78-82	Filtered continuation/export exists. Add an event drawer explaining before/after, actor, correlated financial receipt, and verification coverage.
Customers	client/src/admin/pages/super/oxsteedCredit/CustomersPage.jsx:53-61, :87-125	Account filters/paging and idempotent loads/adjustments/status exist. Require account-bound previews instead of relying on typed customer IDs.
Money Out	client/src/admin/pages/super/OxsteedCreditMoneyOut.jsx:76-110; server/routes/admin.js:462-545	Withdrawals/conversions are fetched at offset zero. Add navigable queues and execution receipts, preserving current step-up and dual-approval fee changes.
Cards	client/src/admin/pages/super/OxsteedCreditCards.jsx:62-85; server/routes/admin.js:362-366	Filters/offset paging and ephemeral issuance output exist. Show fulfillment events, shipment failures, and active/revoked counts independently of the current page.
Ledger	client/src/admin/pages/super/oxsteedCredit/LedgerPage.jsx:69-100, :130-149	Transaction continuation/detail and idempotent refund exist. Show original/refund linkage and exact refundable remainder before opening the existing confirmation.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** compliance case totals and page membership disagree. listComplianceCases applies LIMIT/OFFSET without an open-state predicate when no status is supplied (server/services/complianceCaseService.js:172-189). getComplianceQueue then filters out closed cases after paging (:219, :228-238), although its count includes only actionable cases. Closed old cases can occupy every slot; UI page count based on open totals can prevent reaching actionable later cases. The recent offset fix is real but does not fix this ordering/filter contract.
2. **DESIGN GAP:** approval and money-out work is sampled. Approvals requests newest 50 pending actions (ApprovalsPage.jsx:41); its API returns no continuation metadata (server/controllers/oxsteedCreditController.js:839-846), although the service supports offset (oxsteedCreditPendingActionService.js:568-584). Withdrawals/conversions always request offset zero (OxsteedCreditMoneyOut.jsx:78-89). Older financial work can become practically invisible.
3. **DESIGN GAP:** one global super-admin persona owns every subpage (server/routes/admin.js:377-459). The restriction prevents ordinary unauthorized reads but supplies no country/entity/partner-scoped reviewer roles. The residual concern is scalable delegation and conflict-of-duty enforcement, not absent authentication.
4. **UNVERIFIED:** provider settlement, reserve custody, human compliance decisions, and long-period reconciliation capacity have not been observed. External reconciliation explicitly fails when Stripe is unavailable (server/services/oxsteedCreditExternalReconciliation.js:534-551); report that correct unknown outcome, not “all matched.” Relevant tests include server/__tests__/oxsteedCreditGovernance.test.js, oxsteedCreditCompliance.test.js, oxsteedCreditExternalReconciliation.test.js, and oxsteedCreditSettlementCycleLeaseFencing.test.js; inspecting them is not executing them.

Expansion failure scenario

A weekend surge creates more than 50 withdrawals and many resolved compliance cases. Older withdrawal requests vanish behind the newest sample, while actionable cases are displaced by closed records yet the count suggests few remaining pages. Two countries escalate to a global super-admin team, turning a sound journal into a bottlenecked operating process.

Required changes and operator design

Filter actionable cases in SQL before paging and count the same predicate. Return independent continuation/count contracts for cases, sanctions, verifications, pending actions, withdrawals, and conversions. Add bounded assignment/lease and execution-receipt states to existing resources rather than a shadow ledger. Define scoped read, maker, checker, compliance, and recovery functions; reauthorize the target on every execution. Retain journal immutability, reservation integrity, step-up, idempotency, and worker fences.

Operator screen specification

Use the existing Credit subnavigation as a persistent workbench. Every child header should show selected currency/partner scope, source freshness, and rail/freeze state. Overview's KPI drawer defines balance, reserved liability, partner payable, and unmatched exposure separately. Queue pages need resource ID, subject, native amount, status, oldest-event age, assigned reviewer, due time, and receipt reference. Saved views differ by purpose: approvals “Mine to check,” disputes “Evidence pending,” settlements “Approved awaiting payout,” compliance “SLA breached,” and money-out “Provider outcome unknown.”

Detail drawers should match the resource: approvals show frozen proposed payload plus maker/checker identities; disputes show evidence and compensation; reserves show coverage simulation; partners show agreement/entitlement/funding tabs; customers show balance and journal; cards show fulfillment history; ledger shows original and compensating entries. Health and reconciliation draw from run/finding records, showing checkpoint, error category, and incident links. Audit opens the correlated chain, not a raw JSON-only explanation.

Allow individually authorized decisions, funding, refunds, settlement transitions, freeze, notification retry, and card fulfillment. Bulk operations initially assign or export bounded selections; money movement requires per-resource previews and approval policy. Preserve stable idempotency across network retries. Show 202 as queued with a receipt link; show executed only after its stored result is confirmed. Conflicts refresh the resource without erasing evidence drafts. Empty queues must disclose filters and completeness; unavailable provider checks say “verification unavailable,” and no-access screens name the missing role. On mobile, use a full-screen drawer, fixed decision footer, RTL-safe amounts/IDs, focus trapping, and announced status changes.

Acceptance tests and owner

Credit Engineering owns accounting; Treasury owns payouts/reserves; Compliance owns case decisions; IAM owns delegated scope. Seed 120 closed cases preceding 60 open ones: all 60 must be reachable exactly once and match totals. Navigate 501 pending actions/withdrawals through bounded pages. Attempt self-approval, expired step-up, repeated refund, and lease-loss execution: require denied or single-effect outcomes. Validate scoped partner/entity isolation on direct IDs. Confirm provider outages produce failed/unknown reconciliation, not matched. Reconcile every proposed queue total against its source predicate before expanding reviewer staffing.

39. Revenue

Route and business purpose

/admin/super/revenue provides period revenue, subscriptions, job fees, refunds, growth, and a transaction export. Finance needs comparability and traceable receipt definitions; this page currently reads platform-ledger movements rather than a closed accounting period or bank settlement.

Verified current implementation

Summary and export use view_financials (server/routes/admin.js:261-262). The summary maps supported periods, calculates previous-period growth, and returns 50 recent rows (server/controllers/superAdminController.js:1528-1589). The component has period selection, retry, and export failure feedback (client/src/admin/pages/super/Revenue.jsx:45-74). Currency and sign are transformed on the server into major-unit amount and synthetic completed/refunded status (superAdminController.js:1570-1571). Export retrieves the selected interval into memory (:1609-1630). Schema currency is not hypothetical: the related financial source stores native amounts, and Money explicitly groups ledger currency at :1699-1707.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** the summary has no currency grouping and divides every amount by 100 (superAdminController.js:1545-1550). UI money presentation cannot recover currency omitted by the API. Money's USD-specific fix is not shared here.
2. **CONFIRMED DEFECT:** every negative entry counts as a refund, and every positive entry as total revenue (:1545-1548); recent status likewise depends only on sign (:1571). A negative correction or outflow is not necessarily a customer refund. The controller does not classify by its actual financial event.
3. **CONFIRMED DEFECT:** the dedicated export is unbounded (:1609-1619) and uses ad hoc CSV escaping which quotes only cells containing commas (:1622-1625). Quotes/newlines without commas and spreadsheet formula prefixes are not handled. This endpoint uses only the read grant and lacks the export limiter applied to generic export (server/routes/admin.js:262, :274). This is separate from existing safe-export code elsewhere.

Expansion failure scenario

During a year-end close, many finance users request a full-year CSV. A query materializes millions of rows and builds another large string while ordinary API traffic shares resources. A controller then interprets a negative adjustment as a refund, and spreadsheet output cannot reliably reproduce the reviewed report.

Required changes and operator design

Build summary and export from one versioned receipt classification with native currencies, half-open period bounds, and explicit recognition timing. Add bounded asynchronous export jobs with safe CSV serialization and dedicated capability/budget. Separate corrections/outflows/refunds and provide source receipt IDs; do not label sign as provider settlement state.

Operator screen specification

Replace the period-only header with period boundaries, accounting timezone, currency, scope, and last successful calculation. Keep the four cards but define total recognized revenue, subscription receipts, job-fee receipts, and actual refund events. Growth must identify its comparable prior interval; when the prior amount is zero, show “No comparable baseline,” not a misleading flat zero percent.

The transaction list should expose posted time, source classification, signed native amount, currency, receipt ID, linked customer/job, and reconciliation state. Add date, source, currency, and reference filters with saved views “Subscription close” and “Refund movements.” The existing 50-row table should disclose that it is recent activity and provide a cursor. A row drawer shows original event, ledger entry, recognition rule/version, any reversal, and settlement linkage. Allow opening Financials or the relevant payment; no row action should directly edit revenue.

Export becomes a preview dialog listing scope, columns, estimated rows, inclusion rule, and authorized sensitive fields. Submit creates a job receipt with queued/running/ready/failed status; downloads carry a completeness manifest. Bulk action is exporting selected authorized receipts or linking discrepancies to a reconciliation case. If generation fails, preserve the filter and job ID for retry/resume. Failed summaries retain clearly stale prior data; empty means successful zero qualifying receipts, while no-access does not display customer information. Provide a scrollable mobile table with a card alternative, RTL-safe IDs/amounts, keyboard-accessible export status, and announced job completion without automatic download or focus loss.

Acceptance tests and owner

Revenue Accounting owns classifications; Data Engineering owns delivery. Seed fees, negative corrections, refunds, and CAD receipts: each must land in its correct category/currency. Compare a fixed snapshot's paginated rows, export manifest, and totals exactly. Test commas, quotes, CR/LF, and formula prefixes using safe serializer criteria from server/__tests__/auditExportSafety.test.js. Load-test concurrent year-long exports in the synthetic harness; reject or queue oversized requests without affecting ordinary request SLOs.

40. Fee Config

Route and business purpose

/admin/super/fees edits transaction fees, subscription fees, and trial settings. It is a pricing publication workflow whose output influences quotes, helper proceeds, subscription provider prices, and commercial promises. A successful database change must not falsely imply successful billing publication.

Verified current implementation

Reads/preview require view_financials; update/reset require super-admin (server/routes/feeConfig.js:6-10). Strict numeric parsing rejects nonfinite and trailing garbage values, and server bounds/trial minima are checked (server/controllers/feeConfigController.js:13-19, :77-98). The UI converts percentages/cents, validates bounds, and preserves confirmation on failure (client/src/admin/pages/super/FeeConfig.jsx:43-89). History is stored in fee_change_log, with a no-delete rule (server/migrations/017_phase6_fee_config.sql:79-96). Subscription edits create a provider price and update the plan (feeConfigController.js:153-180). History and reset screens already exist (FeeConfig.jsx:190-239, :273-285).

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** history and value change are separate autocommit queries (feeConfigController.js:100-121), with no expected version. A failed update can leave a success-looking history event, and simultaneous operators can log the same old value while overwriting each other.
2. **CONFIRMED DEFECT:** provider synchronization errors are swallowed (:184-187) after the local value commits; the API still returns success (:129-134). Partial failure after archiving the old price but before updating the plan is particularly difficult to reconstruct.
3. **CONFIRMED DEFECT:** preview uses truthiness defaults, so supplied zero rates/minimums become nonzero defaults (:201-207). A zero-fee proposal cannot be previewed faithfully. Reset loops through autocommit writes and does not call subscription synchronization (:233-255), so its success is not atomic publication of billing prices.
4. **DESIGN GAP:** history is capped at 100 (:47-50), and the component displays five or the loaded sample (FeeConfig.jsx:191). Complete historical fee attribution needs searchable revisions.

Expansion failure scenario

Commercial teams stage a zero-fee market launch, but preview silently calculates default fees. An operator then updates subscription price; provider synchronization fails, yet success appears. Another shift resets fees and assumes billing prices now match the reset. Millions of users amplify that configuration ambiguity into disputed charges.

Required changes and operator design

Publish versioned fee revisions atomically with audit and expected revision. Use nullish defaults and the same calculation function as charging. Represent provider publication separately as pending/succeeded/failed with stable provider idempotency and recoverable jobs. Reset should prepare an explicit revision diff and the required billing sync, not sequentially overwrite keys.

Operator screen specification

Keep fee groups but add a publication header showing active revision, pending draft, provider-sync state, and last applied time. Each row must show key, business label, stored unit, operator unit, allowed bounds, current value, proposed value, and affected products. Search by key/product and save “Subscription publication failures” or “Transaction fee changes.” KPIs should count draft keys, blocked publications, and unsynchronized plans, rather than imply financial gain from an unapproved change.

A row drawer shows change history with older-page navigation, consumers, sample quote calculations, and the active provider price reference. Editing stages a draft. The preview panel must demonstrate customer total, platform/protection/broker deductions, and helper proceeds on representative amounts, including zero and minimum boundaries. Confirmation lists before/after values, effective time, scope, expected revision, reason, and billing publication effects. Reset is a multi-row draft with excluded keys clearly identified.

Allow saving drafts, requesting publication, canceling unpublished drafts, and retrying failed synchronization with its original receipt. Bulk actions are a reviewed fee revision or default-reset preview; no blind reset-all button should immediately publish. A stale revision returns a conflict diff and preserves proposed values for rebase. A provider timeout shows local revision status separately from provider outcome, assigning the sync job to Billing Operations. Empty history means no recorded changes for that key; unavailable history must not imply unchanged fees. No-access identifies read versus publish capability. Use full-width mobile editors, keyboard-linked labels, accessible numeric-unit descriptions, and RTL layouts that keep percentages and values together.

Acceptance tests and owner

Billing Engineering owns publication; Finance Pricing owns rate decisions. Inject failure between audit/value writes: both must roll back. Race identical starting revisions: one succeeds, one returns 409. Preview zero rates/minimums and assert exact zero deductions. Simulate provider failure at create/archive/plan-update stages: expose recoverable state and never claim full publication. Reset tests must reconcile fee config, plan amount, provider mapping, and revision audit after recovery.

41. Pricing Config

Route and business purpose

/admin/super/pricing edits public tier-price presentation. Its six fields are price/label pairs, not the complete subscription charging engine. Operators must distinguish changing advertised copy from changing actual plans, regional prices, and transaction fee rules.

Verified current implementation

The component loads public /config/pricing and sends its whole form to the update endpoint (client/src/admin/pages/super/PricingConfig.jsx:11-40). Server writes use authenticate plus requireRole('super_admin') (server/routes/config.js:6-10), a different authentication chain from requireSuperAdmin. The controller allowlists six keys and validates numeric price text before writes (server/controllers/configController.js:80-95); updates and audit now share a transaction with fail-on-audit-error (:96-123). Thus partial multi-field publication and nontransactional audit should not be reported as unchanged defects. Defaults remain string presentation values (:38-45), and a missing table or query failure returns defaults (:48-65). Migration server/migrations/021_create_platform_config.sql defines the persistent config.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** the default tier3_price: 'up to 17%' is incompatible with current update validation. The UI submits that unchanged default (PricingConfig.jsx:16, :40); server requires every _price value to match a numeric regex (configController.js:89-92). A legitimate edit to another field fails until the third-tier presentation is altered.
2. **DESIGN GAP:** the confirmation promises immediate platform-wide pricing effects (PricingConfig.jsx:109), but the update writes only platform_config (configController.js:101-104). Actual subscription checkout uses regional or plan provider IDs (server/controllers/subscriptionController.js:136, :174), and fee rules have another store. The screen should describe its actual publication boundary.
3. **CONFIRMED DEFECT:** successful defaults and failure fallback are indistinguishable (configController.js:50-65). Operators can review synthetic default copy during a database outage as though it were the persisted configuration; the API carries no freshness/provenance status.
4. **DESIGN GAP:** unconditional upserts have no expected revision (:101-104). Transactional atomicity does not prevent a later stale whole-form submission from replacing a newer editor's values. The exact MFA equivalence of this alternate middleware chain is **UNVERIFIED** here and must not be inferred from the page guard.

Expansion failure scenario

Marketing changes a label for a country campaign and cannot save because an untouched default violates validation. Once saved, staff assume the displayed amount equals what checkout charges, although regional Stripe prices remain unchanged. Two editors then overwrite each other's copy in a whole-form update.

Required changes and operator design

Create a typed presentation schema allowing an explicit percentage/message price type, separate monetary values from localized display labels, and publish revisions with expected version. Return persistence/fallback status. Add a read-only linkage to actual plan/fee/regional charge configuration, and test the server authentication chain directly before declaring parity.

Operator screen specification

Make the header read “Public tier presentation” with active revision, persistence status, language, and last publisher. Keep six fields grouped by tier, but show a live preview beside each: advertised amount/text, billing cadence, explanatory label, and actual linked billing configuration. A warning badge should identify a presentation/checkout mismatch without assuming they must always be identical; approved promotional copy may need explicit explanatory wording.

Replace generic text inputs with type selectors for free, monetary, percentage, and descriptive tiers. Attach visible labels and field-level validation. The detail drawer for a tier should show published revisions, related plan and regional overrides, fee references, and the public pages using that tier. Provide key/text search across revisions and saved views for “Unpublished copy” and “Billing mismatch review.” There is no meaningful revenue KPI here: counts should track mismatches, draft tiers, and publication failures.

Save creates a reviewable draft diff; publication requires scope, revision, reason, and exact public preview. Bulk publication means publishing all changed tiers as one atomic revision, not independently writing each field. A 409 shows the newer published text alongside the retained draft. Unavailable persistence should block publication from fallback data; “No custom presentation” should explicitly indicate defaults with provenance. No-access names the publication restriction and offers no editing controls. On mobile, stack editor and preview, maintain label associations, and provide translated preview direction independently from the operator shell. Return keyboard focus to the edited tier after confirmation and announce the actual published revision. Route any advertised-versus-charged discrepancy to Pricing Operations with the affected tier and country, not directly to refund processing.

Acceptance tests and owner

Product Pricing owns displayed claims; Billing owns charge linkage. Load default data and edit only a label: the proposed typed API must accept it without changing unrelated price semantics. Inject audit failure and verify existing transactional rollback. Race whole-form revisions and require 409. Test persistence outage returns unavailable/fallback provenance. Exercise direct update requests for ordinary user/admin, disabled super-admin, and missing MFA according to the adopted authentication contract.

42. Pricing Overrides

Route and business purpose

/admin/super/pricing-overrides manages country-specific Premium subscription prices. Correct operation requires a coherent mapping between native price, country, provider price ID, and actual checkout. Currency comparisons alone do not prove the charged price matches the configured price.

Verified current implementation

The page loads overrides and currency comparisons and edits country price/currency/reason (client/src/admin/pages/super/PricingOverrides.jsx:18-27, :162-187). Routes use super-admin (server/routes/pricingOverrides.js:5-8). Controller validation and floor conversion call updatePremiumPrice (server/controllers/pricingOverrideController.js:48-70). Service logs the old amount, upserts the override, and invalidates its classified local cache (server/services/pricingService.js:125-157). Migration 160_location_based_pricing.sql:6-35 stores currency, optional price bounds, provider ID, and change history. Provider sync already creates missing prices; tests at server/__tests__/pricingServiceStripeSync.test.js:10-63 cover that behavior. The new process-state registry classifies this cache (server/utils/processStateRegistry.js:64-67); it is not an unacknowledged process-state defect.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** changing amount/currency retains stripe_price_id because the upsert does not modify it (pricingService.js:142-153). Sync skips any row with an existing ID (:240-247). Checkout prefers that regional ID (subscriptionController.js:174), so an already-synced override can display a new price while checkout still uses the old provider price.
2. **CONFIRMED DEFECT:** audit and upsert run as separate queries (pricingService.js:125-153). A failed upsert can leave a change log; concurrent updates have no versioned old value. The history stores one currency value, insufficient to reconstruct a currency change's complete before/after state (160_location_based_pricing.sql:28-35).
3. **DESIGN GAP:** validation checks code length rather than supported country/currency membership and does not apply stored min/max bounds (pricingService.js:111-121; schema 160_location_based_pricing.sql:11-12). The editor divides all currencies by 100 and fixes two decimals (PricingOverrides.jsx:111-123, :169, :212), requiring an explicit minor-unit contract for zero/three-decimal markets.

Expansion failure scenario

A launch team changes a synced country's monthly price. Database and comparison cards show the new amount; sync reports already synced and checkout charges the prior price. Another region changes currency, complicating reconciliation because the history does not retain the previous currency/provider revision.

Required changes and operator design

Publish immutable country price revisions tied to provider amount/currency/recurrence and activation state. Create/reconcile provider prices with stable idempotency and preserve existing subscription policy explicitly. Use transactional old/new audit and expected revision. Validate against supported currencies, country packs, and minor-unit metadata; benchmark rather than assume cache TTL provides instant publication.

Operator screen specification

Keep per-currency comparison cards but add a country/activation search bar and publication-status filter. Header should state which subscription product and interval are being edited, snapshot time, and revision. Rows need country, native amount, minor-unit scale, provider amount/currency, effective date, synchronization status, active customers affected, and last publication. Save views for “Provider mismatch,” “Awaiting activation,” and “Country launch review.”

Open a country drawer before editing: current advertised/configured/charged prices, provider revision, change history, bounds, fallback behavior, and affected new-versus-existing subscribers. Draft edits show native-unit inputs and reason. Preview must compare old and new checkout outcomes and clearly state whether existing subscriptions are grandfathered or migrated. Confirmation includes country, currency, interval, provider mapping, expected revision, and activation policy.

Allow single-country draft/publication, approved batch draft import, and retry of an identified failed synchronization job. Batch preview lists every valid/invalid country and per-row result; avoid treating the existing batch API's top-level success as universal success (pricingOverrideController.js:163-167). A stale edit shows the intervening revision rather than overwriting it. A provider timeout creates an unknown/pending publication state until the original idempotent request is reconciled. Empty filters state no matching countries; absent country configuration identifies the current US fallback explicitly. No-access and unavailable comparison/source states remain separate. On mobile, preserve country and currency above editable amounts; keyboard labels must include the unit, RTL codes remain readable, and focus returns to the country row after publication. Assign mismatches to Billing with the provider receipt and next reconciliation action.

Acceptance tests and owner

Subscription Billing is accountable; Country Pricing approves economics. Extend existing sync tests: change a previously synced amount/currency and assert checkout cannot keep a mismatched provider ID. Inject log/upsert failures and demand atomic state. Verify zero/three-decimal fixtures end to end using the adopted currency metadata. Race revisions and provider retries: require one effective publication, complete per-country results, and a reconstructable old/new history.

43. Financials

Route and business purpose

/admin/super/financials lists platform-ledger entries with source filters, summary totals, and CSV export. It helps investigators locate receipts but is not the entity-scoped double-entry Treasury journal. The business must distinguish ledger movements, revenue recognition, refunds, and bank settlement.

Verified current implementation

The API requires view_financials (server/routes/admin.js:255). Controller paging is bounded, with source-type filtering and joined user context (server/controllers/superAdminController.js:1305-1308). Summary totals are separately queried (:1309-1318). The UI loads ledger pages and offers source tabs, fetch retry, and export feedback (client/src/admin/pages/super/Financials.jsx:15-75, :119-149). Treasury is already present with legal entities, accounts, period locking, and journal currency/entity validation (server/controllers/treasuryController.js:182-205); Financials should link to it rather than propose another ledger. Treasury reads require view_financials, and writes additionally require requireSuperAdminOrPermission('manage_financials') (server/routes/treasury.js:11-26). The route-matrix literal-pattern warning is not evidence that write gating is absent.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** positive movements become “gross revenue” and all negative movements become “total refunds,” irrespective of source (superAdminController.js:1311-1315). A signed operational ledger is not inherently a revenue/refund statement.
2. **CONFIRMED DEFECT:** totals aggregate all currencies and the UI renders dollar strings with a fixed divide-by-100 (Financials.jsx:83-91; superAdminController.js:1316). The source filter applies only to rows; totals remain lifetime/global (:1307, :1316), inviting mismatch between visible entries and headline scope.
3. **DESIGN GAP:** row queries have no date, currency, provider/reference, entity, or reconciliation filters (:1304-1308), and ordering lacks an ID tie-breaker. Bounded offset pagination limits payload size but does not provide a stable close snapshot under concurrent posting.
4. **UNVERIFIED:** cross-ledger reconciliation from legacy platform movements to Treasury journals and Credit dual-posting is not established by this page or treasury.test.js. Treasury's real entity/currency guards must be preserved, but their existence alone does not prove every legacy producer posts correctly.

Expansion failure scenario

A controller filters refunds while the cards still show lifetime totals across every source and currency. They attempt to reconcile that screen to an entity-specific Treasury close. New entries shift offset pages mid-review, and the export cannot explain whether it represented the same snapshot.

Required changes and operator design

Return separately labeled filtered and platform totals by currency, with typed movement categories and snapshot identifiers. Extend list queries with reference/date/entity where authoritative mapping exists; use stable cursor ordering. Connect immutable ledger entries to existing Treasury journals and reconciliation exceptions, explicitly indicating unmapped entries.

Operator screen specification

Place a scope bar above the existing source tabs: posting interval, currency, source, legal entity mapping, and snapshot time. KPI cards should show filtered debit/credit movement totals and separately defined recognized revenue/refunds; never relabel every sign as revenue. A toggle can expose global totals, clearly marked outside the selected filter. Saved views should include “Unmapped to Treasury” and “Refund close exceptions.”

List columns should be entry ID, posted time, source event, native signed amount, currency, user/job reference, journal linkage, and reconciliation status. Search by exact receipt/provider reference as well as description. The detail drawer needs immutable entry metadata, original financial event, related reversals, journal lines, entity/currency mapping, and audit history. Missing mapping is an actionable exception, not a guessed entity.

Row actions open the authoritative payment/refund page, open its Treasury journal, or create/attach a reconciliation exception. Bulk operations export a reviewed snapshot or assign unmapped exceptions; they must not edit ledger entries or close periods. Export preview includes scope and coverage, while status

displays the stored snapshot/job receipt. A stale snapshot notice offers refresh or completion against the original close snapshot. API failure must hide authoritative totals and retain only labeled stale data; a missing ledger table is “Financial source unavailable,” not an empty successful ledger. No-access explains the read grant; lack of journal-write authority hides Treasury posting actions. Mobile uses labeled entry cards and a full-screen evidence drawer; RTL amounts/codes use directional isolation and keyboard focus returns to the originating entry. Assignment belongs to the reconciliation record, keeping financial history immutable.

Acceptance tests and owner

Financial Systems owns source/journal linkage; Controllership owns close definitions. Seed fees, payout outflows, corrections, and refunds: totals must classify correctly. Apply source/currency/date filters and assert cards match rows or explicitly global labels. Verify equal-timestamp entries paginate once each while new posts occur. Test Treasury read-only users cannot post/close, writers without its required read grant follow the documented policy, and journal validation tests at server/__tests__/treasury.test.js:144-245 remain valid.

44. Payouts

Route and business purpose

/admin/super/payouts currently presents helper payment records as payout queues. Treasury staff must distinguish a customer payment's lifecycle from the helper's earned amount, transfer initiation, and actual settlement. Restaurant, affiliate, and Credit payouts are separate domains and cannot be assumed covered here.

Verified current implementation

Read access uses view_financials; retry is super-admin only (server/routes/admin.js:256-259). The controller maps tabs to payment statuses, joins helper/connect-account context, and bounds paging (server/controllers/superAdminController.js:1407-1432). Retry allows only failed records, treats released as no-op, and conditionally updates status to avoid duplicate racing retries (:1477-1503). UI confirmation, fetch errors, and a detail drawer exist (client/src/admin/pages/super/Payouts.jsx:17-48, :60-83, :201-207). Migration 014_phase4_create_payments.sql:13-28 distinguishes gross amount, helper payout, currency, status, transfer ID, and failure reason. A separate fenced affiliate settlement reconciliation worker exists (server/jobs/payoutSettlementReconciliation.js:3-11, :28-43), not proof of helper settlement.

Concrete flaws and gaps

1. **CONFIRMED DEFECT:** the UI displays p.amount, the customer's gross payment, as payout amount (Payouts.jsx:22, :178), although server stats sum helper_payout (superAdminController.js:1436-1440). A $100 gross/$83 helper record produces inconsistent queue and summary amounts.
2. **CONFIRMED DEFECT:** server returns database statuses authorized/captured/released/refunded/disputed, while UI badges expect pending/processing/completed/failed (Payouts.jsx:174-185). Retry appears only for literal failed, despite the failed tab including refunded/disputed (superAdminController.js:1412). Refunded/disputed are not failed transfer retries; preserve the controller's refusal to retry them.
3. **CONFIRMED DEFECT:** the drawer expects method/reference/failure information, but the list projection omits method and failure_reason (superAdminController.js:1418-1424; Payouts.jsx:23-27). It falls back to the payment ID without exposing selected transfer evidence. Currency is also omitted and dollar formatting hardcoded (Payouts.jsx:86-89).
4. **DESIGN GAP:** UI fetches only status without page/limit continuation (Payouts.jsx:61-64), and retry's local status reset does not itself prove a worker/provider will complete the transfer. That completion path is **UNVERIFIED**, so this review does not claim an actual duplicate payout or a working requeue processor.

Expansion failure scenario

A shift sees “failed” entries consisting of refunds and disputes, while successful released entries get a failure-colored badge. The operator reads gross customer value as money owed to a helper and lacks provider failure details. They retry a real failed item but cannot tell whether funds moved or only the local payment state changed.

Required changes and operator design

Define a payout-specific DTO with gross, fee, helper entitlement, currency, payment state, transfer state, and settlement state. Add continuation and failure/provider details. Retry should create a durable execution receipt with clear requeue versus provider outcomes and reconciled ambiguity handling, preserving its existing conditional update.

Operator screen specification

Replace the four overloaded tabs with queues derived from payout state: eligible/uninitiated, transfer processing, settled, retryable failure, and investigation required. Header shows rail, currency, accounting scope, and oldest successful refresh. KPIs must count records and sum helper entitlement separately; do not present counts as money. Add recipient/reference search, provider failure category, age, and payout-readiness filters with saved views “Disabled payout accounts” and “Unknown transfer outcome.”

Rows should show helper, job/payment ID, helper amount, currency, connect readiness, transfer status, age, and last attempt. Drawer sections are entitlement calculation, customer-payment history, provider transfer evidence, bank/settlement evidence, attempts, and audit. Refunds/disputes link to investigation without a retry button. Completed records show immutable receipt evidence rather than relying on a status string.

For retryable failures, preview exact helper amount, failure category, existing transfer reference, readiness prerequisites, and the original idempotent attempt identity. Confirmation explains whether it queues or executes. Initially bulk actions only assign investigations or export; bulk retry requires per-row eligibility preview and execution manifests. A timeout shows outcome unknown and offers receipt reconciliation before another attempt. A 409 refreshes state and explains why retry is no longer allowed. Empty failure queue means no qualifying failures after successful source checks; provider unavailable is distinct from no payouts. Read-only finance users should not see actionable retry controls. Mobile prioritizes helper amount/status over optional columns, with keyboard drawer focus, textual statuses, and RTL-isolated IDs. Assign every unknown outcome to Payments Operations with its next provider check.

Acceptance tests and owner

Payments Operations owns payout incidents; Payments Engineering owns execution contracts. Test $100 gross/$83 helper fixtures: both row and summary show $83 payout. Map every database status explicitly. Paginate 1,001 records without gaps. Retry races yield one state transition; released/refunded/disputed do not execute a transfer. Inject lost responses and verify reconciliation uses the original receipt before retry. Add integration coverage for the actual helper processor; adjacent mainAppPayouts.test.js:27-79 covers request/balance behavior, not this admin queue's settlement.

45. Conversations

Business purpose and observed implementation

Conversations is a privacy-sensitive investigation surface for exceptional staff access, not the ordinary customer messenger. The thread handler requires a normalized access reason and records access before fetching messages (server/controllers/conversationsAdminController.js:167). That ordering is important. Linked business/helper quotes, agreements and payment artifacts are loaded separately with individual bounds. List APIs support paging, while the current screen's search/typeahead operates over the loaded view; it does not make the entire corpus browsable automatically.

Findings, expansion scenario and required changes

P1 confirmed control gap: blocking/archiving uses a read followed by an unconditional status update, accepts an optional reason and ignores audit-write failure (server/controllers/conversationsAdminController.js:238). Two operators can overwrite each other's decisions without acknowledging the changed state. Move status transitions and required evidence into a version-aware audited command; blocking must not incidentally cancel financial or booking records. **P2:** message history is loaded without a bound (server/controllers/conversationsAdminController.js:193). Long-running corporate or support conversations can overwhelm memory and rendering. Implement chronological cursors and a dedicated evidence export budget. Linked-artifact failures are caught into empty collections, so the drawer must distinguish unavailable artifacts from proven absence.

At six million users, unrestricted investigation access is itself a material privacy risk. Bind access to a case, approved purpose, scope and short-lived viewing session; separately authorize exports and attachments. Existing reason logging is a sound foundation, not proof of case-based least privilege. Logs should contain identifiers and purpose rather than copying the private conversation into diagnostics.

Operator design recommendations

Start with an investigation index containing conversation ID, participant roles, linked engagement identifiers, last activity, case reference and current communication restriction. Hide message previews until the operator has obtained case access. Filters distinguish customer/helper, business/internal, guest inquiry, blocked and archived records, with bounded server search. A labelled access banner states why this operator can view the thread, when that permission expires and which downloads remain forbidden. A second investigator must supply their own purpose rather than inheriting another employee's open session.

The thread view uses a virtualized chronological timeline with load-earlier controls and explicit gaps when an evidence source is unavailable. A related-record rail shows quotes, booking, agreement and payment status individually; a failed payment-artifact panel says “not verified,” not “none.” Private staff channels remain separated from customer-facing exchanges. Attachment access is deliberate and checked per object; viewing a thread does not grant a bulk download entitlement.

Block/unblock/archive commands appear in a decision drawer with the current state, proposed restriction, affected communication channels, required reason and case owner. Explain that these commands do not automatically refund or cancel related work. A conflicting decision shows the intervening event and demands review. An unknown response offers command-status recovery, not repeated blocking. Closing the case removes viewing access but preserves evidence under its retention policy. On small screens retain the access banner and case owner above the message content; keyboard shortcuts cannot bypass the reason dialog. Restore focus after decisions and announce evidence-load failures separately from the chat timeline.

Acceptance and ownership

Trust and Privacy own access policy; Messaging owns bounded retrieval and transitions. Test denied/expired case access, pre-read audit failure, guest verification, private staff-channel separation, competing status changes and partial artifact outage. Demonstrate scoped export with retention and redaction. Prove a 100,000-message fixture stays usable without loading everything and that access revocation terminates ongoing reads/realtime subscriptions.

46. Admin Accounts

Business purpose and observed implementation

This tab manages privileged identities, distinct from the complete employee roster. The current list selects admin/super-admin accounts, login signals and audit counts without pagination (server/controllers/superAdminController.js:2885). Creation checks privileged-role authority and hashes the submitted initial password. Disable uses an expected-state predicate and invokes session revocation; these existing protections deserve credit (server/controllers/superAdminController.js:2973). They still need end-to-end tests across API, sessions and sockets.

Findings, expansion scenario and required changes

P1 confirmed misleading assurance: creation returns “must change their password on first login,” but its INSERT does not set a must-change flag (server/controllers/superAdminController.js:2950). The statement is not enforcement. Replace administrator-distributed passwords with expiring invitation enrollment, enforced credential rotation where applicable and MFA registration before privileged actions. **P2:** roster queries compute per-user audit count/max against growing history and load all accounts. Use bounded identity search and maintained activity summaries. An account disable is not the entire leaver process: transfer open approvals, queues, business responsibilities and emergency coverage without reassigning financial authority implicitly.

Not every one of 3,500 employees should become an admin. Corporate staff and workflow assignments need narrower permissions, while super-admin access should remain rare and reviewable. Verify protection against disabling the last recoverable privileged operator and against treating re-enabled accounts as automatically reauthorized for expired grants. No claim is made here that the current system lacks all MFA controls; the missing gate is demonstrated complete lifecycle enforcement.

Operator design recommendations

Use a privileged-identity roster, with tabs for enrollment pending, active privileged staff, disabled identities and recertification due. Display sponsor, employment link, privilege class, effective scope, MFA enrollment/compliance, last successful login, session count, earliest grant expiry and unresolved handoff count. The top row reports stale identities and access exceptions rather than a decorative count of all admins. Filter by country/team, sponsor, privilege, inactivity interval and enrollment blocker; expose total results and cursor navigation.

The identity drawer explains how each permission was obtained: role, direct grant, delegation or corporate assignment. Show overlapping grants and explicit denials separately. Invitation creation previews the approved role bundle, sponsor, expiry and enrollment requirements; it never places a reusable initial credential in a table, export or audit description. A recovery action requires fresh verification and its own audit reason, not a bypass toggle.

Disable opens an offboarding checklist listing active sessions, owned cases, pending decisions, expiring delegations and backup operator. Separate “disable identity now” from “transfer owned work”; the first must succeed safely even when a downstream handoff system is unavailable, with outstanding tasks tracked afterwards. Re-enable requires review of current role and MFA, not restoration of an old permission snapshot. A last-admin warning gives a verified emergency recovery path rather than trapping the operator. Unknown outcomes show the current authoritative status and revocation reconciliation. Mobile prioritizes security state and handoff actions; screen-reader announcements name the identity and the final result.

Acceptance and ownership

Identity Operations owns enrollment and leavers. Prove mandatory first-login/MFA gates, invitation expiry, competing disables, self-disable denial, last-operator recovery and revocation of refresh/realtime access. Re-enable a user whose old grant expired and prove it stays expired. Test a full offboarding with one failed handoff, showing access removed and a tracked unresolved obligation. Budget roster queries against a realistic privileged population and multi-year audit history.

47. Permissions

Business purpose and observed implementation

The permission catalogue now includes operations, financial management, moderation, support and visibility scopes (server/controllers/adminPermissionController.js:7). Grants have expiry and optional region; creation validates active regular-admin recipients and a maximum one-year term. Current self lookup loads at most 50 active grants, while the management view returns at most 200 without pagination (server/controllers/adminPermissionController.js:55, :114). Server authorization remains authoritative; a checked box in the UI is not proof of record-level regional access.

Findings, expansion scenario and required changes

P1 consistency gap: create/revoke writes the grant and then writes audit evidence separately (server/controllers/adminPermissionController.js:176, :216). An audit failure cannot undo the already committed authority change. Use a shared transaction or durable command/event record and return an unambiguous outcome. Revocation also needs conditional transition/idempotent final-state handling. **P2:** fixed self-list limits can omit valid grants and disagree with endpoint authority; management limits hide older active grants. Introduce bounded cursor retrieval and a server-computed effective-permission summary, including region and expiry, rather than relying on incomplete unions in the browser.

For worldwide staff, define role bundles and approval limits, separation of duties and periodic recertification. Keep region-only grants out of global endpoints until every list/detail/export/socket/action has a region predicate. A regional grant must not be silently promoted to global because a UI lacks a region picker. Delegation must not extend beyond its parent's authority or expiry; verify current grant/delegation composition as a complete journey.

Operator design recommendations

Organize the page around “who can do what, where, until when.” The roster shows staff identity, permission bundle, direct/delegated source, geographical scope, start/expiry, grantor, business reason, recertification owner and current effective status. Saved views include global money authority, expiring this week, orphaned sponsor, overlapping grants and overdue review. Search is server-bounded; a results footer states the full predicate count rather than presenting 200 rows as the whole company's access.

The grant drawer first selects the employee and authorized scope, then a role bundle or explicit capabilities. Display a plain-language impact preview: read customer messages, issue refunds within approved limits, change settings, or export data. Show incompatible combinations and routes not yet region-aware. Selecting a region must never make a global capability appear safe. Include exact effective and expiry times with timezone, requested reason and independent approver where policy requires it.

A comparison panel shows added authority, unchanged authority and authority removed by revocation. It includes dependent delegations and cached/session effects. Recertification presents evidence of actual use and sponsor confirmation; “keep all” is not the default. Revocation remains possible during staffing incidents and reports the command outcome plus downstream reconciliation. On conflict, display the new grant record and force renewed review. A permission denied state explains the required sponsor/escalation without exposing another staff member's private data. All labels use business language, while stable capability codes remain copyable for engineers.

Acceptance and ownership

Security/IAM owns the authority contract. Seed more than 50 simultaneous grants and prove the UI summary matches every server decision. Test expiry, parent revocation, region cross-access, denied exports and stale sessions. Inject audit failure and demonstrate no unrecorded privilege change. Run explicit route tests for each catalogue scope; fix checker blind spots without weakening middleware. Record a complete quarterly recertification batch with per-record decisions and an immutable evidence reference.

48. Regions

Business purpose and observed implementation

Regions defines staff territories used by grants; it is not automatically a delivery service area, a tax jurisdiction or a demand reporting region. Boundary validation supports postal ranges, city/state, geographic radius and country/region combinations (server/controllers/adminRegionController.js:17). Country definitions use canonical validation. Lists include active grant counts; deletion refuses a region with current active grants. Region middleware already has fail-closed semantics elsewhere, so this review is about registry quality, change control and coverage.

Findings, expansion scenario and required changes

P1 governance gap: editing an existing boundary changes what current region grants mean without an immutable boundary revision or expected-version predicate (server/controllers/adminRegionController.js:158). At global scale a casual radius edit may widen authority for many employees. Stage changed geometry, compute affected staff/data scope, and bind grants to an approved revision or explicit migration decision. **P2 confirmed validation gaps:** radius centers are finite but not constrained to valid latitude/longitude bounds, and postal ranges are numeric rather than country-specific postal identifiers. Postal codes with leading zeros or letters cannot be modelled reliably as generic numbers. The country-aware boundary types are preferable; do not assume every territory is a US ZIP range.

Registry audit calls currently catch failures (server/controllers/adminRegionController.js:140, :216). Make consequential scope changes durably auditable. Verify delete/grant concurrency and foreign-key behavior rather than relying only on a pre-delete count. Prefer retire over delete when historical grant/evidence records refer to a boundary. Add deterministic overlap precedence and “unassigned” treatment; geography must not silently grant access to records with missing location.

Operator design recommendations

Provide a territory inventory with canonical code, country, boundary type/revision, parent, active staff count, unresolved overlap count, unmapped records and lifecycle status. Separate labels for “admin access territory” and “service coverage” so dispatch personnel do not mistake this screen for an availability switch. Filters include country, boundary type, overlapping, unused and retirement pending. A map is optional evidence assistance; a fully accessible textual boundary and membership explanation must work without it.

The boundary editor uses country-specific input widgets: canonical subdivisions, postal strings or validated coordinates/radius. Before saving, show old/new geometry, staff whose scope changes and samples of records entering/leaving authority. Geographic precision is explicit, with missing-location records classified as unresolved. Let reviewers test a record ID or address against the proposed boundary using an authorized diagnostic, without exporting all customer coordinates.

Scope changes require a reason, version and approval where they widen privileged access. The decision drawer distinguishes editing a display name from changing authority. Retirement lists dependent active and historical grants, offers replacement mapping and keeps the old revision readable. Overlaps show deterministic rule application instead of assigning whichever region was loaded first. A conflict preserves the proposed geometry and reloads the approved revision. If mapping is unavailable, investigation continues but widening actions pause. Mobile replaces the map with a structured boundary summary, impact count and record-test control; keyboard users can inspect every coordinate and affected staff row.

Acceptance and ownership

IAM owns authorization boundaries; Geographic Operations owns data quality. Test invalid coordinates, international postal formats, leading zeros, overlaps, missing coordinates and records on an exact boundary. Run competing retirement/new-grant operations and verify no global scope is created accidentally. Prove an approved boundary revision propagates across two workers/caches and revoked territories cannot be used by stale sessions. Keep delivery service-area tests separate from admin territory tests.

49. Internal Access

Business purpose and observed implementation

Internal Access assigns additional route access to eligible internal staff. Existing code validates the employee/admin role, active identity and route-specific eligibility before saving flags (server/controllers/logisticsInternalAccessController.js:92). It hashes the supplied gate credential and projects only safe record fields. Operations, compliance and admin-portal flags are normalized against authoritative route rules. These are meaningful protections; the audit does not expose or rotate any live credential.

Findings, expansion scenario and required changes

P1 lifecycle/evidence gap: the save helper changes access with a read/upsert pattern, but its returned updated_by_user_id is constructed response metadata, not a persisted change history (server/controllers/logisticsInternalAccessController.js:175). Add expected version, durable actor/reason evidence and explicit revocation reconciliation. Trace caller-level audit/session invalidation before certifying the full flow; a changed response label is not sufficient proof. **P2:** the roster and eligibility logic must explain why requested route flags were normalized away. Silent normalization can leave an administrator believing access was granted when it was not.

At 3,500 employees, this extra gate should not become an alternate unmanaged identity system. Maintain the underlying employee identity, MFA, device/session policy and work entitlement as the authority. Enrollment, reset and emergency recovery need accountable sponsors, expiry and offboarding. Verify already issued internal sessions after disabling a route or the linked user; the presence of a 15-minute token does not by itself establish immediate revocation. Route access must remain narrower than broad admin access.

Operator design recommendations

Make the roster a route-entitlement matrix: employee identity, department, internal alias, underlying account health, operations/compliance/admin access, last verification, sponsor, expiry and pending revocation. Filters focus on inactive linked identities, missing sponsors, unused access and recently changed entitlements. Do not display gate credentials or reusable credential exports. A change history shows only that a credential was rotated, who authorized it and its effective time, never its plaintext value.

Selecting a staff member opens the underlying role and corporate assignment alongside requested route permissions. Each toggle has an eligibility explanation and an effective-state preview; if compliance access is not allowed for that assignment, disable the request with the correct staffing change path. Do not silently render it checked and then let the server drop it. Enrollment/reset is a separate verified workflow with deliberate handoff, not a routine row edit.

The revoke preview states exactly which routes and sessions will be affected, lists owned operational work requiring reassignment and records the reason. Access denial should happen immediately; handoff tasks may reconcile afterwards without keeping the identity active. Show revocation progress and remaining downstream checks separately. A conflict requires re-reading the employee's current role and record revision before applying changes. Recovery links take the operator to verified identity/support procedures rather than a bypass button. Keyboard and mobile use preserve effective-state labels, sponsor and expiry; safe masked contact details are enough for queue triage, with sensitive identity details available only in an authorized drawer.

Acceptance and ownership

Identity Operations owns enrollment/recovery; Logistics Security owns route policy. Test ineligible roles, inactive users, duplicate aliases, mismatched route requests and one-time credential reset. Disable underlying staff and route-specific access while sessions are open; verify API and realtime denials. Inject a timed-out save and audit failure and prove the final authority is discoverable. Record a staff-transfer case that removes old compliance access before new operational work is granted.

50. Gate Lockouts

Business purpose and observed implementation

This security queue lets authorized operators investigate failed internal-gate attempts and lift a verified lockout. The shared service persists rate-limit/attempt state, rather than relying only on one web process. The controller validates a positive lockout ID and passes actor, source and reason into the lift operation (server/controllers/logisticsInternalAccessController.js:673). Recent attempts have filters and offset controls. Active lockouts are capped at 200 (server/controllers/logisticsInternalAccessController.js:663). Verified email recovery is a separate flow and should remain separate from administrative incident handling.

Findings, expansion scenario and required changes

P1 policy gap: manual lift accepts an empty reason, weakening the investigation trail for a security override. Make purpose/case reference and identity verification mandatory for privileged recovery. Lifting a lockout must not enable a disabled staff account, restore expired authority or suppress fraud evidence. **P2 scale/design gap:** a fixed active-list cap without older-page navigation can hide affected staff during a broad incident. Add cursor/total/oldest-age metadata and safe grouped investigation, not a blanket “unlock everyone.” IP plus alias is an abuse signal, not a proof of a person's identity; shared workplace networks require careful interpretation.

Prove service behavior across workers and dependency outages. Database-backed lockouts do not by themselves establish that every gate path consistently applies them or that a recovery request cannot be replayed. Alert on unusual manual lifts, repeated lockout/lift cycles, distributed failures and provider-email delivery problems. Avoid exposing account-existence information to public recovery callers. Evidence-retention periods and access restrictions belong in a documented security policy, not ad hoc deletion buttons.

Operator design recommendations

Use an incident-oriented queue showing lockout reference, safe identity hint, route, first/last failed attempt, failure count, expiry, network indicator, linked case and current recovery state. Default ordering prioritizes active staff blocked from critical shifts and unusual repeated failures, while hiding full sensitive identifiers until the investigator opens a permitted record. Filters include route, time interval, repeated lifts, recovery sent/used and suspected shared-network incidents. The total and oldest age describe the whole authorized queue.

The investigation drawer presents a timeline of failed attempts, successful verification, prior lifts and recovery events. Compare staff account state and work entitlement; a locked gate on a disabled identity is not a reason to reactivate employment access. A network view can correlate suspicious patterns, but must not brand every employee behind the same office IP as an attacker. Provide links to the accountable Security case and current on-call owner.

“Lift verified lockout” requires case ID, purpose and completed identity checks, then previews the exact lockout scope and expiry effect. No bulk lift is offered by default. A second operator's completed lift becomes a resolved state, not an invitation to repeat it. During shared-storage outage the screen shows stale evidence and prevents unsafe recovery overrides. After lifting, display whether access verification succeeded; “lift recorded” and “employee can sign in” are different outcomes. Preserve investigation filters after returning to the queue and announce recovery errors accessibly. Mobile keeps the reason, scope and verification checklist visible above the confirmation action.

Acceptance and ownership

Security Operations owns this workflow. Test concurrent lifts, old recovery links, disabled/expired identities, enumeration resistance, shared-office IPs and distributed attempts. Kill one worker and verify counters survive. Simulate email and storage outages and prove no silent unlock. Complete a verified staff recovery while retaining attempt history and recording one lift outcome. Measure false-positive lockouts and time to restore legitimate access without using those targets to weaken abuse controls.

51. Create User

Business purpose and observed implementation

The account-creation workflow supports customer, helper, restaurant/business modes and internal employee identities with explicit role/default-mode mapping (server/controllers/superAdminUserController.js:48). It validates account fields and role authority, and creates the user plus helper profile and per-field action evidence through a transaction (server/controllers/superAdminUserController.js:298). Sensitive password fields are redacted from display/evidence helpers. This is a useful authoritative creation service, not a substitute for business membership, identity verification, paid entitlement or Stripe onboarding.

Findings, expansion scenario and required changes

P1 authority workflow requirement: distinguish an administrator asserting a verification/subscription field from actual provider-verified evidence or settled payment. Creation accepts verification flags and membership information; the operator UI must require appropriate authorization/evidence and explain which changes do not activate financial privileges. Do not allow a restaurant-manager default mode to imply membership in an arbitrary restaurant. **P2 onboarding gap:** operator-supplied passwords require a secure handoff and enforcement of initial credential setup. Prefer expiring invitation completion, duplicate-safe commands, consent provenance and a clear incomplete account state. Email uniqueness alone does not make a timed-out creation command recoverable.

The new entity-ID and ownership graph work added after the main audit snapshot is relevant here. Verify every admin creation path initializes required identifiers and attaches only explicit ownership; the existence of registration-path tests is not proof this separate controller is covered. Worldwide account import needs validation preview, per-row outcome and idempotent replay, not a mass “create admin” tool. No production identities were created during this audit.

Operator design recommendations

Make this a guided provision-and-invite form, starting with the intended person/entity and account purpose. Show canonical email/contact, country/locale, employee or customer relationship, selected account mode and role mapping. Explain that “business manager” or “restaurant staff” requires a separate verified membership assignment. A live duplicate lookup returns a masked authorized match and opens the existing-account workflow; it should never leak the complete user directory merely because an operator types a short query.

The second stage lists default tools and pending requirements: identity verification, profile completion, business membership, agreement acceptance, subscription evidence and payout onboarding. Distinguish informational mode selection from paid or regulated capability. Verification and entitlement assertions sit in a privileged evidence panel, not ordinary checkboxes beside a name. Staff provisioning requires sponsor, department and effective date; customer assistance records the user's consent or support case.

Before submission, display the exact role, identifiers to be created, pending state and invitation effect. The primary action is “create and invite,” not “create verified user.” A progress receipt shows account ID, successful domain initialization and remaining setup steps. A timeout provides command lookup; retry cannot create a second account or duplicate helper profile. Errors identify a recoverable field or initialization step without echoing credentials or raw database details. Mobile keeps the summary and pending obligations above confirmation. Preserve non-sensitive drafts only where appropriate; do not store initial credentials, identity documents or verification evidence in browser local storage.

Acceptance and ownership

Identity and Customer Operations own provisioning. Test every supported account type, employee roles, invalid country, duplicate email, timeout/retry, denied privilege and business membership isolation. Verify per-field audit rollback on failure. Test entity-ID/ownership initialization specifically through this endpoint. Demonstrate invitation expiry/resend, unverified onboarding and valid subscription-provider handoff without manufacturing a charge or paid entitlement.

52. Search Logs

Business purpose and observed implementation

Search Logs records how staff use the cross-entity search system. The list supports operator and query filters plus limit/offset and total (server/controllers/adminSearchController.js:376). Statistics expose search volume, common queries and top operators (server/controllers/adminSearchController.js:433). This is investigation evidence with sensitive search terms/IPs, not an employee productivity leaderboard or a customer analytics dataset. Search authority and evidence access must remain independently controlled.

Findings, expansion scenario and required changes

P2 confirmed validation flaw: list limit clamps only the upper bound; a negative parsed value reaches PostgreSQL LIMIT and fails instead of being rejected (server/controllers/adminSearchController.js:394). Validate positive bounded integers and move large-history browsing to keyset pagination. **P1 privacy requirement:** returned fields and popular-query statistics include literal queries, which can contain customer identifiers or private fragments. Define minimization, reveal authorization, retention and legal-hold rules. Do not broadly expose popular sensitive queries because they are aggregated. Verify write-time storage/encryption and decryption behavior before making encryption-at-rest claims; this handler alone cannot establish the entire storage policy.

Result counts currently do not establish why each entity source returned empty. The main search transforms failed subqueries into zero-like results elsewhere; logs need completeness/error fields to distinguish “no match” from dependency failure. At enterprise size, use bounded reporting windows and background aggregations instead of repeatedly scanning all history. Include attempted denied searches and export use under appropriate privacy policy, without logging full returned customer payloads.

Operator design recommendations

Build an access-investigation log with timestamp, staff identity, scope, requested entity types, purpose/case, query category, source completeness, result count and correlation ID. Default the literal query to masked form. Filters include date range, staff/team, sensitive entity class, denied access, incomplete search, unusual volume and linked case. Show the search interval and retention boundary so reviewers know whether missing old events are expired, absent or unavailable. Do not rank staff by raw search count as a performance metric.

The detail drawer reveals literal query only after a separately authorized purposeful action and records that reveal. It shows which sources were attempted, succeeded, denied or unavailable, alongside the staff's effective scope at the time. A source error cannot masquerade as a successful zero-result search. Links to returned record IDs perform fresh authorization; historic search access does not grant continuing access to current private records.

An anomaly panel groups repeated broad queries and unusual exports for investigation, with an assigned reviewer and disposition rather than automatically accusing an employee. Scoped export requires an approved date/predicate, privacy redaction, retention expiry and asynchronous job status. Legal hold is explicit and cannot be undone by routine purge. Paging remains stable as new searches arrive, and selected evidence is preserved by ID rather than row position. On failure retain the last verified sample and error timestamp. Keyboard users can reach reveal, case assignment and export status without depending on hover-only controls; mobile prioritizes operator, outcome and time over dense query text.

Acceptance and ownership

Privacy Engineering owns minimization; Security Operations owns investigation. Test negative/oversized limits, source outage, sensitive-query masking, denied reveal, expired retention, legal hold and scoped export. Verify pagination with identical timestamps and concurrent writes. Seed multi-year history and measure query/report budgets. Complete one anomaly review with preserved provenance, while proving an ordinary support administrator cannot browse unrestricted staff search history.

53. Settings

Route and business purpose

/admin/super/settings controls platform configuration, feature flags, and proposed configuration changes. At worldwide scale, it is an operational change-control surface, not merely a preferences form: a valid change can alter fees, availability, or customer behavior across modules.

Verified current implementation

Direct settings writes require manage_settings; change approval has the separate approve_settings permission in server/routes/settingsChanges.js:19. The direct controller locks the setting, validates the value, increments its version, and commits audit evidence inside the transaction: server/controllers/superAdminController.js:2228. Rollback creates a new validated version rather than deleting history. The page already offers previews, schedules, change requests, confirmations, flag controls, and dirty-state handling. Migration 023 defines settings/flags; migration 606 adds change requests. Focused request/applier tests exist.

Concrete flaws and gaps

- **CONFIRMED DEFECT - no stale-write rejection.** The update reads under a lock but never compares the operator's loaded version. A second editor can overwrite the first editor's committed value; serialization alone does not protect user intent (server/controllers/superAdminController.js:2236).
- **DESIGN GAP - partial configuration bundles.** Save All sends individual requests and reports partial success, which is useful but cannot enforce an invariant across multiple settings (client/src/admin/pages/super/PlatformSettings.jsx:1172). A fee pair or dependency pair can temporarily remain inconsistent.
- **DESIGN GAP - approval preview is not an execution contract.** Application rereads the then-current value without checking that it matches the reviewed baseline (server/controllers/settingsChangeController.js:157). Direct writes also remain available alongside proposals; which keys mandate review is not encoded as a per-key business policy.

Expansion failure scenario

Two country-launch teams edit shared settings. One approves a scheduled proposal; another directly changes the same key. The schedule later applies successfully to a different baseline. Audit rows exist, yet the approved impact no longer describes the actual change. A partially saved multi-key bundle makes incident reconstruction harder.

Required changes and operator design

Add expected_version to direct writes and reviewed proposals; reject mismatches with 409 and the current value/version. Introduce transactional configuration bundles with cross-key validation. Classify review-required keys and explicit emergency overrides server-side. Connect changes to Country Packs rollout requirements and Reliability release evidence without making a settings approval equivalent to deployment approval.

Operator design recommendations

Retain the existing sectioned editor, but add a pinned header identifying environment, global/country applicability, last successful server read, and loaded configuration revision. KPI chips should count dirty keys, pending review requests, scheduled requests due within 24 hours, and failed applications; each opens its corresponding filtered queue rather than an unexplained total.

Use a settings grid with key, description, effective value, proposed value, version, dependency group, and review requirement. Search key/description; filter by group, modified state, and change stage. Save views such as “fee bundle changes” and “scheduled launches,” extending the current filters rather than replacing them. A detail drawer should show validation bounds, consumers, before/after diff, revision history, proposal reviewers, and application results.

Row actions are Edit, Propose, and View history; bundle actions are Preview, Submit for review, and Discard draft. Show the exact keys and predicted dependent effects before confirmation. Save All must not quietly imply atomicity while it remains a loop. On conflict preserve local drafts and offer a three-way comparison; after timeout query the change ID before retrying. Failed schedules need an assignee, error reason, and explicit re-preview action. Empty means no matching keys; no-access names the missing permission; unavailable preserves the last snapshot with writes disabled. On mobile use stacked diffs, accessible field errors, keyboard-operable drawers, and RTL-safe logical alignment so Arabic values and machine keys remain legible.

Acceptance tests and owner

Platform Configuration owns implementation; Finance approves monetary keys. Two writes using the same version must produce one success and one 409 without a second audit event. A scheduled request whose baseline changed must require renewed review. A bundle containing one invalid key must commit zero changes. Reuse request/applier tests, then add rollback and timeout-recovery cases that assert database state, not only HTTP codes.

54. Audit Log

Route and business purpose

/admin/super/audit-log is the cross-platform administrative evidence viewer. Security, Finance, and Legal need to reconstruct who changed a resource and correlate that change with an operational incident. Event storage and the ability to investigate those events are separate capabilities.

Verified current implementation

The API requires view_audit_log and bounds page size at 100 (server/controllers/superAdminController.js:2430). The screen renders timestamp, administrator, action, target, description, and IP (client/src/admin/pages/super/AuditLog.jsx:147). Hash-chain serialization and update protection already exist in migration 406; migration 407 provides verification. Importantly, server/migrations/624_ws5_audit_evidence_guards.sql:138 replaces silently swallowed deletes with raising triggers and revokes PUBLIC truncate privileges. These are real safeguards, not missing append-only storage. Actual non-owner production database privileges remain unverified.

Concrete flaws and gaps

- **CONFIRMED DEFECT - response ordering race.** The page effect has no cancellation or request-generation check; a slow earlier filter request can replace newer results (client/src/admin/pages/super/AuditLog.jsx:19). Investigators can see records for filters no longer displayed.
- **DESIGN GAP - unstable pagination contract.** The controller uses offset pagination ordered solely by creation time, without an ID tie-breaker or snapshot boundary (server/controllers/superAdminController.js:2442). Identical timestamps and ongoing writes can cause repeats or omissions across pages.
- **DESIGN GAP - evidence hidden from the operator.** The table does not expose structured before/after changes, target-key search, or chain-verification results. Existing evidence is materially richer than the rendered description.
- **DESIGN GAP - storage integrity does not guarantee writer completeness.** server/services/auditService.js:19 supports transactional fail-closed logging, but standalone callers can remain best-effort. Governance's mutation paths and Workforce's post-write audit sequence illustrate different coverage semantics; the viewer supplies no distinction.

Expansion failure scenario

A reviewer investigates a country-policy change during a high-volume incident. New entries shift offset pages while an earlier action-filter request arrives late. The screenshot appears authoritative but combines a different filter with incomplete traversal. A valid chain proves retained rows were not altered; it cannot prove every sensitive mutation emitted a row.

Required changes and operator design

Add cursor pagination on (created_at,id) with a fixed upper boundary, structured event lookup by target key and correlation ID, and an explicitly privileged integrity-verification endpoint. Define mandatory audit producers with atomic mutation/evidence guarantees. Do not recommend rebuilding the existing hash chain or weakening its delete guards.

Operator design recommendations

Replace the bare log header with “Administrative evidence,” selected time interval, timezone, snapshot boundary, and last successful load. KPIs should report matching events, distinct actors in the selected interval, and last verified chain checkpoint; “not checked” is different from “valid.” Never infer completeness from zero broken links.

Retain existing table fields and add event ID, target key, correlation ID, and evidence availability. Search actor/resource/correlation independently; provide action-family, module, UTC interval, and integrity filters. Saved views should encode incident references and filter parameters, not cache personal data. A drawer should contain an immutable event envelope, structured before/after diff, linked resource, producer identity, and hash/checkpoint evidence. Sensitive IP details can require deliberate reveal with read logging.

Allowed row actions are Open resource, Copy evidence reference, and Add to investigation collection. Bulk action exports selected immutable references plus a manifest and filter snapshot; never offer edit/delete. Export preview states row count, restricted fields, and destination purpose before confirmation. Abort obsolete searches and pin results to the requested snapshot. On verification failure show the first broken reference and escalation path; on service failure preserve prior results under a stale banner. Assignment

belongs to an investigation case with a named Security reviewer and next evidence request. Empty explicitly says no matching events, not “system compliant”; no-access offers permission-request guidance without revealing rows. Mobile shows compact event cards with a full-screen drawer. Keyboard focus, semantic diff labels, and bidirectional handling for actor names and opaque IDs are essential.

Acceptance tests and owner

Security Engineering owns evidence contracts; Internal Audit owns investigation usability. Delay request A until request B finishes: only B may render. Insert concurrent same-timestamp events between cursor pages: every snapshot event must appear exactly once. Tampering must fail under migrations 406/624. Simulated audit failure on mandatory producers must leave neither mutation nor partial evidence. Confirm viewer “unchecked/unavailable” states never render as a successful integrity check.

55. User Action Log

Route and business purpose

/admin/super/audit-log/user-actions specializes in administrative changes to users, including old/new values and notification evidence. It must support complaints, reversals, and access reviews without confusing a recorded change with successful communication to the affected person.

Verified current implementation

The current page supports actor, target, action, region, and date filters, total counts, and abortable requests (client/src/admin/pages/super/AuditLogUserActions.jsx:47). API scope resolution fails closed when scope is unavailable (server/controllers/auditController.js:19). CSV export already uses the exports database budget, no-store headers, and spreadsheet-formula neutralization (server/controllers/auditController.js:75). Migrations 116/124 establish action records and nullable principals; migration 624 raises on deletion. These existing protections should remain.

Concrete flaws and gaps

- **CONFIRMED DEFECT - incomplete export without disclosure.** Export fetches only 500 records at offset zero and ignores the matching total; the page downloads it without a truncation warning (server/controllers/auditController.js:84, client/src/admin/pages/super/AuditLogUserActions.jsx:76). An investigator can mistake a capped file for the complete selected history.
- **CONFIRMED DEFECT - retained events disappear from the list.** The data query inner-joins both user principals while the count query counts the base log. Nullable/missing principals therefore yield inconsistent totals and invisible retained evidence (server/services/auditService.js:221). This is a query-contract defect even without proving live deletion incidence.
- **CONFIRMED DEFECT - imprecise notification attribution.** Notification marking updates rows for the same administrator/target within 30 seconds rather than exact event IDs (server/services/auditService.js:142). Separate edits can be marked notified by one notification attempt.
- **DESIGN GAP - truncated field evidence.** The table shortens old/new values and provides no dedicated evidence drawer; a boolean “notified” omits channel, delivery attempt, and failure reason.

Expansion failure scenario

A support team changes a user's contact and access settings in quick succession. One email succeeds and a second fails, yet nearby events acquire the same notification flag. A later privacy deletion removes a principal join. A capped export then obscures both the missing event and the inaccurate delivery conclusion.

Required changes and operator design

Return export counts/truncation metadata immediately; use a snapshot-bound asynchronous export for complete histories. Left-join principals and retain safe actor/subject snapshots. Associate notification jobs and delivery receipts with exact action IDs. Cross-link Workforce grants, Profile Visibility changes, and Legal revocations while preserving their distinct event schemas.

Operator design recommendations

Keep the current filters and add a header showing subject scope, selected interval, timezone, and last successful query. Define KPIs as matching action events, events with delivery failure, and events lacking a retained principal; do not count field-level rows as unique customers. Place export scope and completeness beside the download button.

Table columns should be event ID, time, actor snapshot, affected user reference, field/action, summarized before/after, region, and notification state. Add searches for exact user/event ID and a notification filter with queued, delivered, failed, and unknown states. Saved views can include “contact changes with failed delivery” and “offboarding changes,” using existing date/region filters. A drawer displays full redacted values, initiating request, related events, notification attempts, and linked reversal evidence.

Row actions are Open affected user, Inspect delivery, and Create follow-up task. Permit bulk export or assignment of follow-ups, not bulk reversal of arbitrary user fields. A notification retry must preview recipient/channel, redact sensitive content, and validate that the contact address is still appropriate. Confirm retries by exact event ID; recover timeouts by querying delivery-job status rather than sending again. Missing principals remain visible with a retained-reference label. A failed list request must not present the previous total as current. Empty, no-access, and export-unavailable states explain their different causes. Assign delivery failures to Customer Operations with a due next contact attempt. On narrow screens place old/new content in labeled stacked blocks; support keyboard disclosure and RTL names without reversing IDs or email strings.

Acceptance tests and owner

Customer Operations owns delivery follow-up; Security Engineering owns retention and export integrity. Seed 501 matching events: synchronous export must declare truncation or asynchronous export must include all 501. Null either principal: list and count must still agree. Generate two edits in 30 seconds and deliver only one receipt: exactly one event becomes delivered. Existing audit-export safety and fail-closed tests should continue passing.

56. Legal: acceptances, legal audit, compliance metrics, privacy requests

Route and business purpose

The four actual Legal routes in the coverage table share client/src/admin/components/LegalSubNav.jsx:5. They respectively track consent evidence, legal events, aggregate acceptance/request metrics, and privacy cases. Older document-editor/mock pages elsewhere in the tree are not these routed screens. Document draft/publish APIs remain relevant dependencies, not proof of a routed editor.

Verified current implementation

Acceptances have filters, pagination, revocation, and CSV; Legal audit has filters and details. Each API has its own permission gate: operations for acceptances/metrics/privacy, audit permission for events, export permission for CSV (server/routes/legalDocuments.js:291). Drafts persist through migration 644; acceptance and audit schemas are established by migrations 162, 164, 587, and 588. The latest ac2abcf7 date correction and its parameter tests are present and passed locally; the old thisMonth defect is not a finding.

Concrete flaws and gaps

- **CONFIRMED DEFECT - revoked consent still looks previously accepted.** Acceptance lookup ignores deleted_at; revocation sets that field. Reaccepting the same version returns alreadyAccepted and a newly fabricated timestamp rather than a new active record (server/controllers/legalDocumentsController.js:53, server/controllers/legalDocumentsController.js:515).
- **CONFIRMED DEFECT - metrics do not measure current valid consent.** The denominator counts active undeleted users while the numerator counts every historical acceptance, including revoked or inactive subjects. Document workflow totals are hard-coded zero (server/controllers/legalDocumentsController.js:309). Rates can mislead even when the page renders correctly.
- **CONFIRMED DEFECT - filtered Legal exports are unfiltered.** Both CSV handlers ignore query filters and cap at 5,000 rows (server/routes/legalDocuments.js:297, server/routes/legalDocuments.js:337). Their quoting helper is not the User Action export's formula-neutralization control.
- **DESIGN GAP - completion is a status, not verified fulfillment.** Privacy updates allow completed without a per-system manifest or transition precondition; all transitions log ccpa_completed (server/controllers/ccpaController.js:350). Data compilation uses unrestricted SELECT * and returns an empty object on failure (server/controllers/ccpaController.js:525). Exact live field exposure and fulfillment coverage are **UNVERIFIED**, not assumed safe.

Expansion failure scenario

A new document version rolls out globally. Historical/revoked consent inflates adoption while privacy staff mark cases completed despite failed data collection. A filtered audit download unexpectedly includes unrelated subjects. Governance dashboards inherit those misleading metrics; none establish legal compliance.

Required changes and operator design

Bind acceptances to immutable published document revisions and append revocation/reacceptance events. Compute current-version active acceptance populations consistently. Share scoped, formula-safe exports. Replace privacy completion with verified identity, jurisdiction-specific counsel-configured deadlines, system tasks, delivery receipts, and held-data exceptions.

Operator design recommendations

Use a shared Legal header with country/jurisdiction, document revision, timezone, and data freshness. Keep four navigation panels; do not merge evidence with fulfillment. Acceptance KPIs count eligible subjects, current valid acceptances, and revocations. Its list needs subject reference, document/version/locale, accepted time, revocation state, and evidence ID; filters include revision and validity. A drawer shows the exact published artifact and consent timeline. Revoke requires reason and an impact preview; bulk action is evidence export, not indiscriminate consent removal.

Legal audit needs actor, action, resource, case ID, time, and event status, with saved case views and immutable details. Export confirmation must name applied filters and capped/full scope. Compliance metrics should show numerator, denominator, exclusion rules, calculation time, and revision coverage; unavailable figures are unknown, never zero.

Privacy requests need type, jurisdiction, verified identity, deadline, stage, assigned specialist, outstanding system tasks, and delivery result. Search case/subject; save “identity pending” and “fulfillment blocked” views. The drawer contains identity evidence, allowlisted data manifest, deletion/hold exceptions, correspondence, and task receipts. Allow Assign, Request information, Generate preview, Approve fulfillment, and Retry delivery only when prerequisites pass. Timeouts recover via case/job status; conflicts preserve notes and require reload before transition. Empty panels distinguish no cases from no access and backend failure. On mobile, present deadlines and blocked tasks above attachments; keyboard announcements, RTL legal text, and accessible confirmation dialogs must survive translated long content.

Acceptance tests and owner

Privacy Engineering and Legal Operations are jointly accountable; counsel owns jurisdiction policy. Revoke then reaccept must create valid new evidence without losing history. Metrics must exclude revoked/inactive/old-version records and disclose denominators. Filtered CSV must contain only matching rows and neutralize formulas. Failed collectors must prevent completion. Every privacy transition must emit its actual event type. Existing Legal parameter tests must remain green.

57. Visitors

Route and business purpose

/admin/super/visitors provides pageview history and geography/device summaries. Growth and incident teams can use it to understand recorded traffic; it is not a customer census or an employee surveillance tool. Six million users require honest measurement definitions as well as bounded queries.

Verified current implementation

Both visitor APIs require the dedicated view_visitor_analytics permission (server/routes/siteVisits.js:14). The recording service respects DNT, strips query/hash components from paths, sanitizes referrers, and stores hashed rather than raw IPs (server/services/siteVisitService.js:60). Migration 613 adds history indexes and a nullable user relationship. The page fetches list and summary together and ignores obsolete effects (client/src/admin/pages/VisitorHistoryPage.jsx:70). Existing tests verify sanitization, DNT, summary/filter behavior, and safe beacon failure.

Concrete flaws and gaps

- **CONFIRMED DEFECT - selected end day is largely excluded.** The UI sends date-only values; the controller parses the upper bound as midnight and uses visited_at <= to (server/controllers/siteVisitController.js:31). A range ending today excludes almost all today's events.
- **CONFIRMED DEFECT - country KPI is capped by a chart limit.** The summary returns only the top 15 countries, while the page labels array length as countries (server/controllers/siteVisitController.js:128, client/src/admin/pages/VisitorHistoryPage.jsx:120). A worldwide footprint can appear to stop at 15.
- **DESIGN GAP - IP hashes are labeled unique visitors.** Counting distinct hashed IPs measures observed network identifiers, not distinct users or devices (server/controllers/siteVisitController.js:123). Shared networks merge people; network changes split them. DNT exclusions further limit coverage.
- **DESIGN GAP - stale results remain ambiguous after failure.** The page retains prior rows/summary with an error but lacks a snapshot-age label and dedicated retry workflow. The service has retention pruning code; actual job operation and production volumes are **UNVERIFIED** rather than declared absent.

Expansion failure scenario

An executive reviews a launch-day report across 40 countries. The selected end date removes that day's traffic and the countries tile remains 15. Shared workplace IPs suppress apparent uniqueness. The team makes launch decisions from correctly stored but incorrectly described aggregates.

Required changes and operator design

Specify date boundaries as [start,endExclusive) in an explicit timezone. Return a separate distinct-country count and top-country coverage. Rename the hash metric to an observed-network estimate; disclose collection exclusions. Keep identity-level inspection separate from aggregated analysis and connect retention policy to Legal without widening collection.

Operator design recommendations

The header should read “Recorded pageviews,” with timezone, inclusive dates, exact effective UTC boundaries, and last successful aggregation time. KPI definitions must distinguish pageviews, observed network hashes, signed-in subjects, and distinct countries. Add a collection-coverage note for DNT and missing attribution; do not imply coverage of all six million users.

Use a list with visit time, normalized path, referrer host, country/city confidence, device, and optional subject reference. Preserve country/path filters but debounce text searches and offer Apply for range changes. Saved views should include “launch landing pages” and “unexpected referrer traffic.” Top-country charts must show “top 15 of N” and the omitted share. A row drawer should expose sanitized collection fields and retention expiry, not reconstruct raw IP or stripped query parameters.

Allowed actions are Open normalized page, Inspect collection metadata, and Create a traffic investigation. Bulk exports should be aggregated by default with a preview of date scope and privacy exclusions. Identity reveal needs a separate recorded purpose if introduced. Collection outages display unknown coverage; query failure preserves a labeled stale snapshot and Retry. A conflict in saved-view dates should prompt timezone clarification rather than silently alter bounds. Route anomalies can be assigned to Growth Analytics or Security with a next collection/check task. Empty means no recorded visits in the range; no-access describes the dedicated grant; unavailable means no trustworthy summary. Mobile should prioritize range controls and totals, collapse geography into cards, label form controls programmatically, and isolate machine paths from RTL prose.

Acceptance tests and owner

Growth Analytics owns metric definitions; Privacy Engineering owns collection boundaries. A visit at 23:59 on the selected end date must be included, the next day excluded, including DST transitions. With 16 countries the KPI must show 16 and chart 15. Shared-IP fixtures must not be called distinct people. Failure must display stale/unknown status rather than a fresh zero, while existing DNT/path tests remain passing.

58. Profile Visibility

Route and business purpose

/admin/super/profile-visibility controls administrative hiding/restoration of public profiles. It affects discovery and livelihoods, so operators need precise change evidence, approval execution status, and protection for in-flight work, not a second generic moderation dashboard.

Verified current implementation

The component already implements search, saved views, previews, active-work checks, reasons, bulk dry runs, scheduling, approval queues, history, notification state, and recovery. API routes require manage_profile_visibility. Migrations 614/617/618 add overrides, schedules, and approvals; 652-654 add durable claims, per-item outcomes, and recovery. Approval decisions reject self-review and atomically reassert pending state (server/services/profileVisibilityApprovals.js:263). This is substantially implemented; the old missing-concurrency-guard finding is not current.

Concrete flaws and gaps

- **CONFIRMED DEFECT - executable approvals lose their payload.** The initial SELECT omits payload, but bulk/scheduled execution uses approval.payload, not the full reviewed RETURNING row (server/services/profileVisibilityApprovals.js:258, server/services/profileVisibilityApprovals.js:312). Bulk execution can receive no items; scheduled execution receives no schedule payload despite a recorded approval.
- **CONFIRMED DEFECT - fabricated before-state.** Shared application records the logical opposite of the requested flag, not the actual previous flag/reason (server/controllers/profileVisibilityController.js:340). Repeating Hide on an already-hidden profile creates incorrect evidence and undermines Undo.
- **CONFIRMED DEFECT - Undo has a check/write race.** Undo reads the latest event and current state, then updates without a version predicate or enclosing lock (server/controllers/profileVisibilityController.js:873, server/controllers/profileVisibilityController.js:885). Another operator can change the profile between those steps.
- **DESIGN GAP - approval versus completion remains ambiguous.** The review is recorded before execution, with best-effort review audit. Existing durable bulk recovery must be visibly connected to review execution failure, not replaced with blind resubmission.

Expansion failure scenario

Country teams approve a large restoration after a false-positive incident. The decision says approved while no intended items execute. A manual retry generates misleading before-state; concurrent Undo then overwrites a fresh hide decision. Active-work checks exist, but the operator cannot reliably reconcile intent, decision, and effects.

Required changes and operator design

Execute the reviewed immutable payload through the existing durable workflow. Capture actual prior values under lock, include a visibility revision, and compare that revision in Undo. Model approved, queued, executing, partially completed, and failed separately. Connect cases to User Action Log and moderation evidence without treating notification success as visibility success.

Operator design recommendations

Extend the present screen rather than rebuilding its already-rich controls. The header needs selected profile types/region, snapshot freshness, and current moderation case. KPI definitions should distinguish currently hidden profiles, pending decisions, approved-but-unexecuted work, partial batches, and failed notifications. Approval totals must never stand in for applied changes.

Keep saved views and add execution-stage filters. List columns should include profile reference/type, current hidden reason, active-work blockers, last visibility revision, proposed action, approval state, and execution outcome. The detail drawer needs current public-preview behavior, exact previous values, reason timeline, protected bookings, approval fingerprint, per-item results, and delivery attempts. Explain what Hide changes and what it deliberately does not cancel.

Allow row Preview, Request change, View history, and version-bound Undo. Bulk actions remain Dry run and Submit for review; recovery acts only on unresolved items from an existing operation ID. Before confirmation show eligible, blocked, already-in-state, and affected counts, with active-work exceptions requiring explicit authority. Approval requires a different reviewer and the unchanged payload fingerprint. On execution failure retain the approved decision and expose Recover execution, not another Approve button. After timeout query the durable operation before retrying. Assign partial batches to Trust

Operations with a next-item remediation queue. Empty distinguishes no eligible profiles from no pending work; no-access and backend unavailable disable writes without erasing history. Mobile needs a sticky batch summary and full-screen review diff; accessible checkbox counts, status announcements, and RTL-safe profile previews support large multilingual reviews.

Acceptance tests and owner

Trust Operations owns decisions; Marketplace Engineering owns effects. A bulk approval containing three items must execute exactly those three, and a scheduled approval must preserve time/payload. Repeating Hide must record the actual unchanged before-state. Concurrent Undo and fresh Hide must yield one conflict, not overwrite. Extend existing T58 race tests with executable-payload and durable-recovery assertions; verify audit/state rollback together.

59. One Truth

Route and business purpose

/admin/super/one-truth combines platform statistics, user dossiers, and suspected duplicate-account signals. At six million users its useful purpose is a trustworthy investigation index and reconciled business overview, not an assertion that every displayed statistic is exhaustive truth.

Verified current implementation

The router requires super-admin access (server/routes/oneTruth.js:6). The controller probes available schema columns, paginates ordinary user results, enriches dossiers, and reports regional/filter limitations. Suspect detection has explicit weights and a bounded candidate load (server/controllers/oneTruthController.js:674). The component has metric groups, suspect/user tabs, filters, warnings, and user-detail display (client/src/admin/pages/super/OneTruthPage.jsx:469). The dynamic schema behavior is a compatibility feature; it is not proof that every optional metric exists in production.

Concrete flaws and gaps

- **DESIGN GAP - suspect coverage selects the oldest 5,000.** The query orders ascending creation time with a 5,000 limit and computes pairs in memory (server/controllers/oneTruthController.js:698). The existing warning is valuable, but recent abuse beyond this candidate window remains outside detection. Increasing the cap alone worsens pairwise work.
- **CONFIRMED DEFECT - monetary presentation loses currency semantics.** Payment/commerce aggregates sum amounts without grouping currency; the renderer formats monetary values as dollars (server/controllers/oneTruthController.js:966, client/src/admin/pages/super/OneTruthPage.jsx:129). Multi-currency production incidence is **UNVERIFIED**; the source contract cannot safely represent it.
- **DESIGN GAP - scope warning can accompany global data.** If a requested region cannot be resolved, the controller warns and returns unscoped results (server/controllers/oneTruthController.js:98). Super-admin authority is present, so this is reporting-scope confusion, not a demonstrated unauthorized disclosure.
- **DESIGN GAP - dossier reads lack the purpose contract used elsewhere.** Detail includes identity/business/login history (server/controllers/oneTruthController.js:1022); the router does not attach Governance's purpose-limited investigation read controls. Super-only is not an investigation-purpose trail.

Expansion failure scenario

A reviewer sees “no suspects” for a new market because its accounts were created after the oldest candidate window. Meanwhile a currency-mixed revenue card appears as USD and an unresolved region silently broadens the report. The same reviewer opens detailed dossiers without creating a case or recorded purpose.

Required changes and operator design

Create indexed candidate signals and a scheduled investigation pipeline with incremental coverage watermarks; retain explainable pair evidence. Return currency-separated totals or explicit conversion basis. Fail scope resolution visibly before displaying differently scoped results. Reuse controlled investigation access for sensitive dossier sections and link financial cards to Treasury/reconciliation rather than presenting incomparable sums.

Operator design recommendations

Place a coverage header above current metric groups: selected geography, actual resolved scope, source timestamps, and suspect-pipeline watermark. Define KPIs as evaluated accounts, unevaluated new accounts, open suspect pairs, and confirmed cases. Display monetary cards per currency with source ledger and reconciliation status; hide combined dollar totals until a conversion contract exists.

The suspect queue should show pair references, signal types, weighted score, latest shared evidence time, candidate coverage period, and case disposition. Filters need signal, country, creation cohort, score, and reviewed/unreviewed state. Saved views such as “new-market shared contact” must disclose coverage gaps. A drawer compares two identities side by side, explaining each matched signal, counterevidence, and permitted redacted fields. Dossier tabs require a case/purpose before sensitive history opens.

Allowed actions are Assign investigation, Mark explained, Escalate, and Open linked moderation case; a score must not automatically merge or suspend accounts. Bulk assignment is allowed after a duplicate-case preview, but sanctions require separate evidence and confirmation in the responsible module. On stale evidence show recalculation status and block final disposition until refreshed; unresolved geographic scope

needs an explicit switch to Global. A failed enrichment preserves the base identity with missing-section labels. Empty means no candidates within the disclosed evaluated population. No-access distinguishes aggregate access from sensitive dossier access. Mobile uses alternating comparison cards rather than compressed side-by-side PII; keyboard-accessible evidence expansion and RTL-safe names/IDs prevent mismatches during review.

Acceptance tests and owner

Trust Analytics owns candidate coverage; Finance Data owns monetary semantics. A new account beyond the first 5,000 must enter incremental detection. USD and another currency must remain separate without declared conversion. Unknown region must not silently render a scoped-looking global report. Every elevated dossier read must record case/purpose or reject. Validate scoring against false-positive fixtures and measure candidate-query plans in an isolated scale environment.

60. Franchise HQ

Route and business purpose

/admin/super/franchises manages franchise applications, agreements, training/checklists, and store activation. Worldwide expansion requires a distinction between a pipeline label, an executed agreement, and an actually provisioned operating business.

Verified current implementation

The API permits authorized admins with manage_operations, whereas navigation is super-only (server/routes/franchise.js:28). This is an explicit UI/API audience difference, not a public endpoint. Migration 247 supplies agreements and checklists. The page has application detail, notes, checklist updates, agreement editing, and activation. Activation locks the application and checks ownership, approval, signed/waived agreement, completed training, ready launch state, and checklist completion (server/controllers/franchiseController.js:875, server/controllers/franchiseController.js:910). Those guards already exist.

Concrete flaws and gaps

- **CONFIRMED DEFECT - first-page pipeline masquerades as totals.** The server defaults to 20 applications; the page loads without pagination controls and computes metrics from applications.length (server/controllers/franchiseController.js:362, client/src/admin/pages/super/FranchiseHQPage.jsx:197). Older applications and their next steps become practically invisible.
- **CONFIRMED DEFECT - editable launched state bypasses launch meaning.** General PATCH accepts status=launched, sets launch time/readiness, and does not run activation prerequisites (server/controllers/franchiseController.js:503). This does not create a business, but falsely permits the state that should describe guarded activation.
- **DESIGN GAP - agreement replacement is not executed-version history.** Agreement upsert replaces terms for an application instead of retaining immutable signed versions (server/controllers/franchiseController.js:770). Manually entered signature metadata is not independent proof of execution.
- **CONFIRMED DEFECT - territory aggregates omit country.** Territory ownership is checked, but helper/job geographic predicates use city/state without the loaded country (server/controllers/franchiseTerritoryController.js:72). Repeated place names can contaminate franchise-owner reports across countries; exact live collisions are unverified.

Expansion failure scenario

A territory in a repeated city name receives another country's helper/job totals. HQ sees only 20 applications and marks one launched through editing despite unresolved training. Country Packs, revenue planning, and Workforce staffing then depend on a launch that has no activated business.

Required changes and operator design

Paginate the queue and supply independent stage aggregates. Derive launched exclusively from successful activation; require controlled transitions for pipeline states. Version agreements and link counsel-approved country/entity evidence. Use canonical country/territory identifiers in every aggregate and reconcile franchise royalties with Treasury rather than generic platform totals.

Operator design recommendations

Create an HQ header with country/program filters, reporting date, and pipeline snapshot time. KPIs must count all matching applications by stage, ready-but-not-activated stores, and overdue checklist tasks, not the loaded page. Display provisioned store count separately from applications labeled approved.

The queue needs application ID, applicant, country/territory, stage, executed agreement revision, training state, readiness blockers, linked business, and next launch milestone. Search contact/application/territory; saved views should include “training blocked” and “ready for activation.” A detail drawer should group applicant identity, territory boundaries, agreement versions, financial terms, checklist evidence, training receipts, and activation history. Existing notes remain a case timeline, not substitutes for evidence fields.

Allowed row actions are Assign reviewer, Request information, Update permitted stage, Record checklist evidence, and Preview activation. Bulk actions may assign owners or export the pipeline, never mark stores launched. Activation preview must name the business owner, proposed entity/territory, all waived requirements and approvers, and expected business record. Confirm only against an unchanged application revision. After timeout retrieve linked-business/activation status before retrying; conflicts reload blockers while preserving notes. Agreement failure leaves the prior signed artifact intact. Empty distinguishes no

applications from no applications meeting readiness; no-access explains operations permission; unavailable disables activation while retaining a stale pipeline label. Field staff need mobile checklist uploads and readable evidence status, keyboard-accessible stages, and RTL territory names with isolated agreement IDs. Assign blockers to the relevant Legal, Training, or Franchise Operations function with one explicit next milestone.

Acceptance tests and owner

Franchise Operations owns pipeline truth; Legal owns executed agreements. With 21 applications, all must be traversable and totals remain 21. Direct PATCH to launched must reject; activation with any blocker must create no business. Duplicate activation must not create another business. Same city/state in two countries must produce isolated territory reports. Signed terms must remain retrievable after amendment.

61. Careers

Route and business purpose

/admin/super/careers mounts the shared hiring board with platform employer type. It should turn approved requisitions into controlled offers and onboarding handoffs. At 3,500 worldwide employees, candidate progression must remain distinct from creating corporate permissions or employment/payroll records.

Verified current implementation

client/src/admin/pages/super/CareersAdmin.jsx:6 reuses HiringPipelineBoard. Opening/application listings are already paginated; stage transitions use transactions and a transition map (server/controllers/careerController.js:584). Platform management checks role/global operations permission; business hiring has separate ownership checks. Offer decisions atomically require pending status (server/controllers/careerController.js:797). Migration 531 supplies application events, interviews, and offers. Existing hiring tests exercise these workflows; do not repeat an obsolete claim that all hiring lists are unbounded.

Concrete flaws and gaps

- **CONFIRMED DEFECT - expired offers remain acceptable.** Offer acceptance checks pending status but never expires_at, then writes application stage hired (server/controllers/careerController.js:767).
- **CONFIRMED DEFECT - terminal candidates can be resurrected through offers.** CreateOffer inserts regardless of current terminal stage; DecideOffer accepts and sets hired regardless of current rejected/withdrawn stage (server/controllers/careerController.js:729, server/controllers/careerController.js:801). The normal stage transition map therefore does not cover every hiring path.
- **CONFIRMED DEFECT - interview timezone comes from unrelated edit state.** Booking uses a prompt and form.timezone, not the selected opening/candidate's explicit scheduling context (client/src/components/careers/HiringPipelineBoard.jsx:151). Browser date parsing plus an unrelated timezone can describe inconsistent appointments.
- **DESIGN GAP - offer composition and staff handoff are incomplete.** The board sends title/currency/team role without compensation/start/expiry fields (client/src/components/careers/HiringPipelineBoard.jsx:167); platform hired status does not itself establish a Workforce/HR onboarding record. Employer API routes use ordinary authentication rather than the admin middleware's TOTP contract, despite platform authority checks; browser/MFA behavior remains unverified.

Expansion failure scenario

A candidate rejects an opening, later accepts an old pending offer, and becomes hired after its intended expiry. Recruiters in different timezones schedule conflicting interviews. The platform board shows a successful hire while Corporate Setup and Workforce still lack a controlled start-date/access handoff.

Required changes and operator design

Apply terminal-stage/expiry predicates inside the offer transaction. Require explicit reopening approval before new offers to terminal candidates. Add a structured timezone-aware interview editor and a versioned offer draft/review/send flow. Create an idempotent onboarding handoff to HR, never automatic broad admin grants on offer acceptance.

Operator design recommendations

Retain the existing pipeline board and pagination. Its header should identify platform employer, requisition, approved headcount, location/timezone, and refreshed time. KPIs need open requisitions, active applications, interviews awaiting confirmation, pending unexpired offers, and accepted offers awaiting onboarding, with explicit definitions.

Each candidate card/list row needs application ID, stage, stage age, recruiter, next interview in candidate and recruiter time, offer revision/expiry, and onboarding handoff state. Search candidate/requisition; filter location, stage, recruiter, and blocked next step. Save views like “offers expiring this week” and “accepted without onboarding.” A drawer contains candidate consent/contact, stage history, interview participants/calendar status, offer terms/revisions, approval evidence, and handoff receipts.

Replace the browser prompt with a labeled interview form that chooses timezone, resolves ambiguous DST times, and previews both participants' clocks. Offer actions are Draft, Request approval, Send, Withdraw, and Inspect decision; sending requires compensation currency/period, entity, start date, expiry, and an approved requisition. Bulk assignment is allowed; bulk hiring or offer sending is not. Transition

confirmations explain terminal effects and preserve historical offers. On timeout resolve interview/offer ID before retrying; on concurrent stage change show a refreshed transition comparison. Failed onboarding remains a separate assigned task, not a reversed acceptance. Empty states distinguish no requisitions, no candidates, and no matching filter; no-access and unavailable states suppress employer writes. Mobile cards must preserve next appointment and expiry, with accessible stage selectors and RTL-safe candidate names alongside isolated date/currency values.

Acceptance tests and owner

Talent Acquisition owns candidate workflow; People Operations owns onboarding. Expired pending offers must reject without stage change. Terminal applications must require explicit reopening. Simultaneous accept/withdraw must yield one decision. DST scheduling must preserve intended instant and displayed timezone. Accepted platform offers must produce one durable onboarding handoff, and no admin access before its separate approval.

62. Country Packs

Business purpose and observed implementation

Country Packs controls the database-owned list of launch countries, locale/currency metadata and country configuration. It is not legal permission to operate in a jurisdiction. The current service explicitly fails closed on registry failure: no fallback US-active response is returned (server/services/countryPackService.js:76). Reserved row metadata cannot be shadowed by JSON configuration (server/services/countryPackService.js:20, :112). Those earlier audit findings are repaired and must not be repeated as open. The registry uses a 60-second cache and an update operation that clears the local process cache.

Findings, expansion scenario and required changes

P1 launch-governance gap: activation is still a status update, without an embedded launch-evidence package proving supported payments, refunds, tax treatment, identity requirements, customer support, language and service coverage. An active picker entry is not a working full-country customer journey. Create versioned country/product launch decisions linked to accountable owners and evidence; country-level activation must not automatically advertise products not supported there. **P2 consistency gap:** update uses no expected revision, and invalidation is process-local (server/services/countryPackService.js:198). A second worker can serve old config until TTL. Define acceptable propagation, shared invalidation/version reads and behavior during country suspension. Existing committed orders must remain supportable even when new intake closes.

Currency validation accepts a three-letter pattern, not proof that a payment provider supports that currency. Locale and configuration fields require schemas and provider/product-specific validation. Separate an administrative country pack from record-level region authorization and data residency. Worldwide growth requires an explicit staged launch/suspend/drain model rather than deleting records or reinterpreting historical amounts in a new default currency.

Operator design recommendations

Create a country-product readiness grid. Rows identify country and product; columns show customer intake, fulfillment availability, payment/refund capability, language coverage, required legal evidence, support coverage and current approved configuration revision. The screen distinguishes disabled, preparing, approved for launch, active, intake suspended and draining existing commitments. Filters include missing evidence, provider restriction, unsupported language and expiring launch approval. A country's flag and currency remain orientation aids, not its readiness indicators.

The pack drawer combines canonical metadata with a schema-driven configuration editor. Show the exact products affected, old/new locale and currency defaults, and the evidence owner for each prerequisite. A preview simulates a new customer journey in that country using provider test mode and authorized fixture records. Historical invoices and balances retain their stored currency; the preview must explicitly state that a default-currency change does not convert them.

Activation requires approved evidence and a bounded rollout scope. Suspension asks whether to stop new requests, new financial authorizations or all unsafe activity, and lists open commitments requiring continued support. Never offer “disable country” as a destructive shortcut to cancel everything. Version conflicts show the current pack and preserve the proposed diff. Shared-cache propagation is displayed separately from “change saved.” The offline state explains that availability cannot be verified rather than substituting an active country. Arabic and international operators see local dates and canonical ISO identifiers side by side; keyboard access reaches the full readiness grid without depending on flags or map interaction.

Acceptance and ownership

Market Launch owns approvals; Payments, Legal, SRE and Support each sign their prerequisites. Test unavailable registry, reserved-field patches, unknown locale/currency, stale parallel edits and two-worker propagation. Suspend new intake while resolving an existing refund and support case. Run a full authorized country/product journey and preserve its evidence package. Certification is specific to country, product and version, never “worldwide ready” from a single active status.

63. Corporate Setup

Business purpose and observed implementation

Corporate Setup manages department configuration and defaults; title assignment and effective authority are related but separate. The resolver combines active assignments, department defaults and allow/deny overrides, with a bounded cache and country context (server/services/corporateRolesService.js:30, :117). Department-default updates now validate unknown permission codes and use a transaction before replacing the set (server/controllers/corporateRolesController.js:160). The old “unknown defaults are silently accepted” finding is closed. Corporate permissions and legacy admin grants remain distinct systems; a department label must not be presented as equivalent to every admin entitlement.

Findings, expansion scenario and required changes

P1 authority-model gap: at 3,500 employees the company needs a single explanation of effective authority across corporate assignments, legacy grants, delegations, internal gates and product-team membership. Consolidate evaluation/provenance, not data by deleting one system. Verify actual route adoption before suggesting that creating a title secures every business endpoint. **P2 change-control gap:** department defaults replace an entire set without an expected-version predicate and are followed by local cache invalidation (server/controllers/corporateRolesController.js:193). Two editors can overwrite changes. Introduce an impact-reviewed revision, explicit approver for high-risk expansion and shared revocation propagation.

Time-bounded acting assignments, sponsorship, reporting-line cycles, inactive departments and separation-of-duties rules need complete operational coverage. A country filter is not proof of record-level territorial enforcement. Verify future/end-dated assignments and denials across multiple simultaneous titles. Keep HR organization, compensation authority and application capability distinct; naming someone CFO must not automatically authorize all customer refunds or credit approvals.

Operator design recommendations

Use three coordinated panes: organizational structure, department permission templates and affected staff. The structure shows department code, owner, parent, country scope and lifecycle status. The template pane groups capabilities by business risk and distinguishes read, approve, execute and export. The affected-staff pane shows which current and future assignments inherit a change. Filters expose unowned departments, inactive titles with active staff and high-risk combinations requiring review.

Editing defaults opens a before/after authority preview with explicit new capabilities and removed capabilities. Show the actual routes that enforce each capability and label unimplemented mappings as unavailable, rather than advertising a permission that never gates a server action. Reviewers can inspect one employee's effective authority including denies and legacy grants; this diagnostic must not modify the employee or elevate the viewing operator.

Publish template revisions through an approval workflow proportional to risk. An organizational rename can be low risk; expanding financial execution or customer-data export requires independent approval and propagation evidence. Retirement previews assignments and owned work, offers a controlled successor and retains old organizational history. Conflicts preserve the draft and list the approved revision; no silent full-set overwrite is acceptable. The audit drawer records intent, impact set, decision, effective date and revocation reconciliation. Mobile starts with ownership and unresolved staff impact, then offers the structural tree as an accessible text hierarchy. Keyboard and screen-reader users can distinguish a collapsed department from one with no employees.

Acceptance and ownership

Corporate IAM and People Operations own the model. Test unknown capabilities, simultaneous title holds, explicit denies, future/end dates, country context and cache loss. Verify conflicting template changes and route-level enforcement for each risk class. Retire a department with open staff responsibilities and prove no implicit privilege widening. Produce one effective-authority explanation matching direct API denials, export behavior and realtime access for a regional employee.

64. Mode Policies

Business purpose and observed implementation

Mode Policies configures platform approval for selected role actions; it does not replace account mode switching, business membership or financial authorization. Current service validation, expected policy version, row locks, revision records and change reason are substantial repairs (server/services/modePlatformApprovalService.js:95, :207, :247). Approval validity is tied to the policy revision (server/services/modePlatformApprovalService.js:372). The gate middleware returns a retryable 503 on verification errors (server/middleware/requirePlatformApproval.js:61); the earlier fail-open recommendation is no longer current.

Findings, expansion scenario and required changes

P1 adoption verification gap: source search of current routers found no direct requirePlatformApproval(...) installations. The gate exists, but editing a policy must not be represented as protecting every relevant action without an explicit caller/route manifest and tests. Verify indirect invocation too before declaring all business actions ungated. Register each supported mode/action pair against an authoritative server operation and its necessary permission, ownership and financial checks. A descriptive action string that no route uses creates false confidence.

P2 operating-model gap: policy changes invalidate old approval versions, but the company needs a clear migration/review path for already approved people, active bookings and pending applications. Provide impact preview and a re-review queue rather than silently halting existing commitments. Retain schema revisions with application evidence; field labels, document requirements and notifications require localization. Auto-approval must have narrow objective eligibility and measurable risk, never substitute for a regulated provider decision. Missing policy behavior is a product decision requiring an explicit default, not an accidental unrestricted action.

Operator design recommendations

Make a mode/action policy matrix showing mode, canonical action, enforcing route/service, current policy revision, approval requirement, auto-approval rule, required evidence and affected approved/pending population. A mapping column explicitly says enforced, not installed, or verification incomplete. Filters include high-risk actions, inactive modes, unmapped actions and policy changes awaiting application review. This prevents a clean-looking toggle from implying protection the backend does not apply.

The policy editor is schema-driven, showing field type, requiredness, acceptable evidence, privacy retention and localized label. Next to each field, preview the applicant form and reviewer evidence panel. Model the exact action restriction separately from role selection. Changing a financial action includes its existing payment/ownership checks in the impact preview; approval can unlock eligibility only, not execute a payment or grant a different user's resource.

Before publish, display old/new revision, changed actions, applicants needing re-review and the treatment of already committed work. Require a reason and a separate approver for high-risk expansions or auto-approval. A conflict shows the newly committed policy and offers a reviewed rebase, not automatic overwrite. After save, expose propagation and enforcement-test status separately from configuration success. A support link directs affected users to reapplication or appeal with their policy revision. Empty or unavailable policy data is labelled, and unsafe gate-dependent actions pause. Keyboard, RTL and mobile layouts preserve the action mapping and risk summary above the publication button.

Acceptance and ownership

Product Policy owns mode eligibility; IAM owns gate adoption. Build direct endpoint tests for every registered action, including no policy, policy outage, expired/stale approval and admin bypass semantics. Prove a changed schema does not reinterpret old evidence. Run a policy revision with pending/approved users and active work, showing re-review plus unchanged committed financial obligations. Test concurrent policy edits and two-worker invalidation. A configuration-only screenshot is not closure evidence.

65. Mode Approvals

Business purpose and observed implementation

This queue handles individual applications against platform mode policy. Current decisions use expected status: approve/reject requires requested, and revoke requires approved (server/services/modePlatformApprovalService.js:486). Rejection needs a reason, and a lost competing decision reports conflict. These repairs close the older unconditional-decision race. Policy-version matching also prevents an old approval satisfying a newer policy. The UI confirms approval and offers reasoned rejection (client/src/admin/pages/super/ModeApprovals.jsx:26).

Findings, expansion scenario and required changes

P2 evidence-history gap: resubmission upserts the user/mode record and replaces application data and review fields (server/services/modePlatformApprovalService.js:458). Policy revisions are stored, but a separate immutable application-attempt/decision history is needed to preserve what each reviewer saw across reapplication. Do not infer that the current row contains every prior decision. Decisions should bind the application and policy revision, evidence version, reviewer authority and case reason, with no-self-review/separation rules where appropriate.

At large population, bounded pending API retrieval needs exposed navigation, truthful totals, aging and routing to qualified reviewers. The current screen uses fixed dark styling and raw mode values; light theme, localized mode/date labels and typed boolean/zero evidence need explicit checks. Design an appeal/re-review path, expiry and suspension that remains separate from account ban, identity verification and payout readiness. Application approval cannot make an uninstalled gate enforce policy or make a Stripe account ready to receive funds.

Operator design recommendations

Use an oldest-first application queue with applicant identity, mode/action eligibility requested, policy revision, submission attempt, evidence completeness, age/SLA, reviewer owner and current state. Filters include missing evidence, policy changed since submission, duplicate/related applications, appeals and review country. Work assignment depends on authority and locale expertise; queue claims must not silently become the eligibility decision itself. Display the entire predicate backlog and oldest age, with stable cursor navigation.

The review drawer keeps applicant-provided evidence separate from independently verified facts. Show typed values correctly: false and zero are not missing answers. Attach the immutable policy/form revision and compare any prior rejected attempt without overwriting it. Clearly state which exact actions approval enables and which further memberships, provider checks or financial approvals remain required. Sensitive attachments use scoped access and retention controls rather than inline public links.

Approve requires a decision preview and confirms the applicant is still eligible under the displayed revision. Reject records a structured reason, applicant-safe explanation and internal notes separately. Request-more-information and appeal route back to a tracked attempt instead of manufacturing a new approved record. Revocation previews the future action restriction and open commitments requiring support; it does not retroactively refund or cancel them. A competing reviewer outcome replaces stale buttons with the committed event. Unknown responses recover by decision reference. Responsive light/dark and RTL layouts preserve evidence labels, actionable errors and keyboard focus; confirmation should name the applicant and policy version, not merely ask “Are you sure?”

Acceptance and ownership

Eligibility Operations owns review; Privacy owns evidence handling. Test two competing reviewers, policy change mid-review, resubmission history, typed fields, denied reviewer, appeal, revocation and decision timeout. Verify actual downstream gate denial/allowance and continued account/payment checks. Seed thousands of applications and reach the oldest unresolved case. Produce an immutable decision receipt and applicant notification in all five languages without disclosing internal investigation notes.

Additional review: Restaurant Top-up Approvals

Route and business purpose

/admin/super/restaurant-topups is the newly added restaurant eligibility queue for customer wallet funding. Despite the super-path name, it is delegatable with manage_financials. It must connect eligibility decisions to money already received, uncredited payments, and chargeback recovery, while preserving restaurant-specific wallet isolation.

Verified current implementation

The route is manifest-defined and grant-guarded (client/src/admin/adminRouteManifest.js:112; client/src/admin/AdminApp.jsx:325-326). All eligibility/clawback APIs require current admin authentication and financial management authority (server/routes/adminRestaurantTopups.js:16-24). Approval rechecks active restaurant and connected payouts; decision and audit are transactional (server/services/restaurantTopupService.js:268-304). Webhook completion locks the top-up, rechecks eligibility, and posts idempotent wallet credit (:527-571, :585-608). Clawback locks the top-up and uses deterministic ledger replay, explicitly allowing/flagging negative balance with two transactional audits (:905-911, :939, :953-1005). Migration 663 supplies tables and the exceptional negative-balance mechanism (server/migrations/663_restaurant_topup_eligibility.sql:14-42, :73-92). Tests cover duplicate credit, in-flight suspension, negative clawback, and replay (server/__tests__/restaurantTopups.test.js:787-817, :959-1003). These controls exist; deployment of migration 663 is **UNVERIFIED** against reported readiness 661/661.

Concrete flaws and gaps

1. **CONFIRMED DEFECT, P1: stale eligibility decisions overwrite newer ones.** Service reads eligibility without locking/version and upserts any target state (server/services/restaurantTopupService.js:137-142, :277-289). The UI constrains actions by loaded state but sends no expected version (client/src/admin/pages/super/RestaurantTopupApprovals.jsx:183-206). A stale approval can reverse a newer suspension.
2. **DESIGN GAP, P1: captured-but-uncredited money lacks an operator recovery queue here.** Ineligibility at webhook credit parks the row as failed, raises an incident, and future nonpending completion is ignored (server/services/restaurantTopupService.js:536-571, :637-643). Admin routes offer eligibility and clawback, not preview/reconcile/refund/recredit for this exception (server/routes/adminRestaurantTopups.js:20-24). Reapproval alone does not settle received money.
3. **DESIGN GAP, P1: clawback evidence/approval is under-specified.** The endpoint accepts a top-up ID and reason; the service debits the full credited amount, with no required provider dispute reference or independent approval (server/controllers/adminRestaurantTopupController.js:107-113; server/services/restaurantTopupService.js:927-949). Existing replay and audits prevent duplicate debits, but do not establish why a debit is justified. No missing wallet isolation is alleged.

Expansion failure scenario

One reviewer suspends a restaurant while another approves stale data. In-flight captured funds become failed/uncredited records, then are overlooked because the screen only lists eligibility applications. A chargeback clawback creates negative balance with no visible debt-resolution owner. Support, provider incidents, restaurant Credit, reconciliation exceptions, and finance must share transaction evidence.

Required changes and operator design

Serialize/version eligibility decisions and require approval rationale. Add a permitted top-up exception list/detail and recovery preview with captured amount/currency, wallet ledger, provider event, and receipt status. Recovery must be an explicit idempotent finance operation, never blind webhook replay. Link clawback to verified dispute evidence and threshold-based independent approval; preserve the exceptional negative flag, restaurant isolation, and append-only ledger replay. Compare webhook amount/currency to the stored intent as a proposed integrity check; mismatch behavior is currently **UNVERIFIED** rather than an asserted exploited defect.

Operator screen design recommendations

Keep the existing status queue and decision dialog, adding a header with financial scope, source revision/readiness status, and separate eligibility/payment-data refresh times. KPI tiles are pending eligible applications, approved merchants, captured-but-uncredited payments, and negative wallets awaiting recovery. Do not infer the latter two from application count. Filters should include restaurant ID/name, country/currency, payout readiness, application age, reviewer, and exception type; save “Pending ready for review,” “Captured uncredited,” and “Chargeback debt followup.”

Application rows show restaurant identity, operating state, currency, application age, eligibility version, payout-readiness timestamp, and last decision. Drawer sections contain restaurant/business links, payout prerequisites with provenance, prior decisions/reasons, pending and succeeded top-ups, uncredited incidents, wallet/reconciliation evidence, and accountable finance case. Keep approval separate from customer funding initiation.

Actions are approve with rationale, reject/suspend, inspect a top-up exception, request recovery, and preview evidence-backed clawback. Before eligibility suspension show in-flight intent count and the resulting exception workflow. Clawback preview shows provider dispute, amount/currency, exact wallet, current/projected balance, negative outcome, and approval requirement. Bulk actions may assign application reviews; monetary recovery/clawback remains individual with receipt and independent approval where required.

On conflict reload current decision and preserve rationale. A timeout recovers the decision/ledger receipt before retry. Empty pending queue must still show unresolved money exceptions. Missing migration/source is unavailable, not zero applications; no access differentiates read from financial decision authority. Mobile prioritizes projected negative balance and evidence before confirmation, with accessible dialog focus, keyboard paging, localized currency units, and bidi-isolated financial references.

Acceptance tests and owner

Restaurant payments engineering and financial risk own this launch gate. Race approval/suspension: stale approval returns 409. Suspend after capture: exactly one visible reconciliation case retains provider/wallet evidence and a documented recovery outcome. Repeat recovery/clawback: one ledger effect, consistent audit/receipt. Unjustified or mismatched dispute evidence blocks clawback; approved negative balance creates a named followup case. Isolated migration 663 and transactional replay tests must pass before deployment; current readiness/CI evidence must identify the actual feature revision.

Workforce Identity and Access Lifecycle workspace

Existing capability and business purpose

This is already a primary workspace, not a missing tab. It provides a staff roster, delegation registry and lifecycle events. The roster covers admin/super-admin identities and is limited to 200 (server/controllers/workforceController.js:52). Delegation validates active recipients, recognized scopes and the grantor's global authority (server/controllers/workforceController.js:197). Self-delegation is rejected; revoke is conditional. Preserve these existing improvements. The workspace is not yet evidence that every corporate employee, restaurant employee or business-team member participates in a company-wide joiner/mover/leaver model.

Findings and long-term requirements

P1 confirmed atomicity gap: delegation INSERT commits before a fail-closed audit call (server/controllers/workforceController.js:237). Throwing after the insert can return failure while the authority record remains active. Revoke has the same write-then-audit separation. Put authority and durable decision evidence in one transaction and provide command-status recovery. Verify how delegated authority is consumed by every relevant permission check; registry presence alone does not prove effective entitlement. Delegations must not outlive parent authority and must be revoked when sponsor/grantor authority disappears.

P2 workforce coverage gap: a capped privileged roster is not a 3,500-employee HR directory or recertification program. Add identity links and workflow states for invitation, start, transfer, leave, termination and emergency coverage. Reuse Corporate Setup and Admin Accounts, preserving non-admin staff roles. Assign sponsored authority, periodic review and handoff tasks. Integrate a chosen workforce identity provider only through approved provisioning and revocation, not by importing broad admin flags. Access review completion must reference actual effective permissions and server denials.

Operator design recommendations

Use a lifecycle command center with named queues for onboarding blockers, transfers pending old-access removal, leavers awaiting revocation, delegations expiring and recertification overdue. The roster includes employee identifier, employment source, sponsor, organization/country, identity health, effective capability bundles, earliest expiry and open responsibilities. Separate corporate employees from customer-business staff. Counts report the whole authorized workforce predicate, with cursor search and per-country visibility rather than truncating to the newest privileged identities.

The employee drawer presents a chronological authority graph: employment record, corporate assignments, direct grants, delegations, route gates and provider connections. Each edge has purpose, source, effective time and expiry. A transfer preview shows removed old authority before added new authority and names unresolved cases, approvals and resource ownership. Handoff delegates work objects explicitly; it never transfers an entire person's access tokens or implicitly makes a replacement super-admin.

Leaver action starts with an immediate deny/revoke command and independent follow-up tasks for cases, devices, partner access and managerial acknowledgement. Display complete, failed and unverified revocation targets separately, plus an incident escalation when access cannot be verified. Recertification compares business need, effective authority and actual use, requiring a named reviewer; an unanswered review expires according to approved policy. Temporary emergency access includes short expiry, sponsor and post-use review. On audit/storage failure stop authority expansion and show a recoverable command state. Mobile prioritizes urgent leavers and expiring coverage, while a fully keyboard-operable graph alternative lists every authority edge and decision.

Acceptance and implementation ownership

People Operations, Security IAM and Corporate Engineering jointly own this workspace. Test hire, transfer, leave, termination, sponsor departure, delegation parent expiry, audit failure and competing revocation. Prove an employee does not require admin role for ordinary work. Verify API/export/socket denial after departure across two workers and one unavailable provider, with unresolved tasks still visible. Closure requires one complete multi-system offboarding drill and a retained quarterly review, not merely populated roster cards.

Treasury and Entity Accounting workspace

Existing capability and business purpose

Treasury is already implemented as an entity/account/period registry and double-entry journal foundation. Posting locks the accounting period, checks entity ownership, account currency and open period, and commits inside a transaction (server/controllers/treasuryController.js:181). Closing uses a conditional open-state update (server/controllers/treasuryController.js:333). This is substantially safer than an unaudited balance-edit tool. It must remain distinct from authoritative customer payment/credit ledgers and must not become a second mutable source of customer balance.

Findings and long-term requirements

P1 retry risk: posting inserts a journal entry without a command idempotency key or unique externally meaningful source-event reference (server/controllers/treasuryController.js:205). A timed-out manual or imported posting can be repeated. Bind every posting to immutable source evidence and ensure one accepted command/event yields one journal effect. Verify audit and independent approval requirements for manual adjustments. **P2 accounting scope gap:** an entity journal alone does not establish bank/provider settlement reconciliation, intercompany balancing, FX evidence, jurisdiction-specific tax reporting or a controlled close. Build these as workflows over existing ledgers with an approved accounting model, not generated rules presented as legal/tax advice.

Periods currently validate date shape/order, but enterprise close needs checks for overlapping periods, late-arriving source events, unresolved exceptions and reopening authority. Bound journal retrieval with stable paging and asynchronous scoped exports. Currency minor-unit handling and FX valuation must remain explicit. At six million users, reconciliation should be incremental and partitionable with freshness/completeness evidence, not a complete cross-ledger scan on every dashboard request.

Operator design recommendations

Give finance a close-and-reconcile workspace with legal entity, currency and accounting period pinned in the header. Show source coverage, settlement lag, unmatched count/amount, control total, journal balance and close blockers separately. Never aggregate unlike currencies into one unexplained total. Queue views include unmatched settlements, duplicate source references, aged receivables/payables, manual journals awaiting approval and period-close exceptions. Rows carry source transaction/reference, entity, currency/minor amount, expected accounting treatment, discrepancy, age and accountable owner.

The journal drawer shows immutable source evidence, debit/credit mapping, period, currency and approval chain. Manual entries require balanced preview, reason, documentation and reviewer authority. Imported events cannot be approved by changing a customer balance; correction uses a reversal/adjustment under the existing ledger rules and a new accounting entry. A command timeout offers lookup by source reference before retrying. Duplicate source posting returns the existing outcome without another entry.

The close checklist identifies incomplete provider windows, unapproved adjustments, unresolved material exceptions and missing entity signoff. Closing records the exact evidence snapshot; reopening is a new authorized event, not deletion of the old close. Entity/country filters restrict detail, exports and actions consistently. Export is a background job with predicate, as-of timestamp and retention expiry. Mobile prioritizes due exceptions and approval previews; keyboard navigation traverses debit/credit fields in a predictable order. Accounting owners must explicitly approve currency, tax and materiality policy before the interface labels a period compliant.

Acceptance and implementation ownership

Finance Engineering and the accountable controller own this workspace. Test duplicate journal commands, timeout/retry, cross-entity/currency rejection, post-versus-close races and source reversal. Run one complete provider settlement reconciliation and period close with unresolved exceptions blocked, then controlled reopen. Validate exports and totals against source ledgers and accounting review. Demonstrate recovery from partial provider imports without duplicating a charge, customer balance or journal effect.

Global Service Operations workspace

Existing capability and business purpose

Service Operations already exposes capacity-related counts and a versioned runbook registry. Its controller explicitly acknowledges that booking records lack authoritative admin-region linkage and labels attribution unavailable/proxy where appropriate (server/controllers/serviceOpsController.js:9). Dynamic relation discovery is constrained by safe identifier checks. Do not repeat the old recommendation to fabricate a regional utilization graph from unlinked bookings. This workspace should support actual delivery and service capacity, safety and accountable operational decisions across business units.

Findings and long-term requirements

P1 decision-quality gap: booking/job demand counts are not supply, utilization or promised fulfillment capacity. There is no capacity denominator in these aggregates: people, shifts, equipment, availability, capability and actual reservations must come from their authoritative services. Add explicit service-area attribution, not merely reuse staff grant territories. A high booking count cannot justify accepting work without verifying constraints. **P2 coordination gap:** runbooks are useful registry records, but execution needs incident/task ownership, revision-aware steps, approvals and evidence. Update increments version without a supplied expected version (server/controllers/serviceOpsController.js:283), so conflicting editor changes can overwrite content.

For global operations, define service-specific demand queues and capable supply with freshness. Internal delivery and marketplace fallback must remain mutually exclusive and price-neutral according to domain rules. Regional emergencies need intake throttles, staff safety escalation, provider outages and customer communication without unsafe bulk cancellation. Record operator intent, approval and reconciliation across orders, deliveries, jobs and bookings rather than building another copy of them.

Operator design recommendations

Present a service-area command view with confirmed demand, pending requests, capable available resources, reserved resources, projected risk and source freshness. Distinguish actual measurements from forecasts and unavailable attribution. Filters select operational area, service type, promised window, priority, capability requirement and exception. A geographic display is supplementary; a textual queue must expose unmapped work and timezone. Alert cards lead to specific blocked work, not a count that lacks a recovery action.

Selecting work opens the authoritative request plus constraints: handling/capacity, assignment, time windows, current milestone, owner and safe next transition. Staff can drill into a driver's/resource's actual busy intervals without declaring every displayed calendar item busy. An assignment preview lists conflicts, source freshness and required review; it must not overwrite dispatch ownership or broaden employee permissions. Fallback release shows the operator reason and customer commitment preserved by the delivery service.

Runbook launch binds the run to a fixed revision, affected scope, commander and approvals. Steps have owner, due time, prerequisite and completion evidence; changing the registry does not change an ongoing execution silently. A regional incident banner separates new-intake restrictions from support for existing commitments and gives a planned update time. Competing actions show committed ownership and require re-review. Dependency outage retains evidence timestamps and pauses unverifiable commitments while allowing pending requests. Mobile orders urgent assigned work before capacity charts, with large milestone controls and required proof. Accessibility includes readable status labels, logical RTL ordering and local operational time alongside UTC references.

Acceptance and implementation ownership

Service Operations and Dispatch Engineering own this workspace. Test missing attribution, stale resource data, conflicting intervals, timezones/DST and one shared resource across product types. Demonstrate incident intake suspension with existing fulfillment/support continuing. Run an approved runbook revision with one blocked step and a shift handoff. Verify every command uses the existing authoritative service and keeps internal/marketplace ownership and payment invariants intact.

Reliability and Release Operations workspace

Existing capability and business purpose

Reliability already has release history, migration inventory and database budget snapshots. Its controller explicitly calls release history manually recorded, not a CI/deployment feed (server/controllers/reliabilityController.js:7). Read failures return unavailable errors rather than fake empty success. Budget snapshots expose web/jobs/exports/realtime categories (server/controllers/reliabilityController.js:92). These are foundations for a release command center, not proof of independent recovery or production promotion controls.

Findings and long-term requirements

P1 evidence gap: a manual “deployed” row accepts a nonempty commit string and operator notes, without binding an actual artifact digest, CI execution, deployment ID or readiness result (server/controllers/reliabilityController.js:39). It can record intent, not certify deployment. In the final refresh CI at 2f93a426 is green, but /ready still has null git SHA/digest/deployment ID. Connect signed/verified CI and Coolify evidence to a release state machine with separate built, approved, migrating, deployed, verified and rollback-requested events.

P1 recovery gap: backups have successful R2-upload metadata, but no current restoration-test row was found in the earlier live evidence query. This is unverified recovery, not proof backups do not exist. Add immutable restore-run evidence and a second-operator drill against an isolated target. **P2 visibility gap:** migration inventory is not checksum compatibility or a production-sized upgrade rehearsal; local workload budget state is not necessarily fleet aggregate. Tag scope/source/process and aggregate appropriately. Keep failure drills off the production Mac and provider live money operations.

Operator design recommendations

Use a release-and-recovery timeline linking commit, CI run, artifact digest, migration set, environment, deployment operation and verified readiness. The header states current observed release identity and any missing metadata. Queues show red/insufficient checks, pending promotion, migration divergence, verification incomplete, stale backup/restore evidence and exhausted workload budgets. Distinguish a manual note from an automatically verified event using explicit provenance labels.

The release drawer shows changed domain surfaces, required tests, migration expand/contract compatibility, rollback constraints and accountable approver. Promotion must be impossible through this interface while critical checks fail or required coverage is skipped. Emergency override has a reason, named authority, expiry and mandatory retrospective; it is not a permanent green checkbox. Deployment progress reads actual infrastructure events and finishes only after release identity, readiness, worker health and queue behavior are verified.

The recovery panel pins one backup object/evidence reference, integrity result, recovery profile, isolated restore target and measured RPO/RTO. Show database recovery, uploads, configuration dependencies, financial invariants and representative journey checks as separate stages. A dry run cannot be labelled recovered. Failed stages lead to a retry/reconciliation task without deleting the previous attempt. Fleet budget panels state process/fleet scope and sample age; investigation remains possible during partial monitoring loss. Mobile prioritizes active rollout and recovery blockers rather than graphs. Keyboard-accessible timelines and textual stage summaries work without animation or status colors.

Acceptance and implementation ownership

SRE and Release Engineering own this workspace. Prove rejected red/insufficient promotion, digest mismatch, repeated deployment callback, migration failure, worker startup failure and controlled rollback. Restore a current offsite snapshot into an independent environment, validate domain invariants and measure recovery time. Complete a shift handoff using only retained runbooks/evidence. Latest green unit CI is credited, while real recovery, scoped authenticated coverage and capacity certification remain explicitly unverified until executed.

Governance, Policy and Contract Obligations workspace

Existing capability and business purpose

Governance already offers a versioned policy registry and contract records with create/update/retire/terminate operations (server/controllers/governanceController.js:64, :184). The comments correctly distinguish registry text from policy automatically enforcing platform behavior. Every mutation attempts an admin audit event. It is a useful governance foundation; it is not yet a complete worldwide obligations calendar, evidence-retention engine or vendor compliance program.

Findings and long-term requirements

P1 confirmed update-validation gap: contract update checks date order using submitted fields, then applies a partial patch without validating the merged existing dates (server/controllers/governanceController.js:236). Changing just signed or expiry date can violate the relation to the retained other date. Read/lock the current record, validate the merged revision and persist it with expected-version protection. **P2 evidence gap:** policy version increments and contract audit calls follow standalone writes. Preserve actual previous revisions, decisions and atomic evidence; a higher version number alone does not retain all old wording. Distinguish operational retirement from erasing a policy needed to explain historic approvals.

For six million users and worldwide operations, track jurisdiction, legal entity, product, accountable owner, required reviews, acceptance version, contractual obligations, renewal/notice dates and evidence expiry. Connect work to an approved policy revision rather than matching text labels. Liability/insurance, data-processing obligations and partner response SLAs require specialist business/legal review. This audit identifies engineering controls; it does not certify legal compliance in every country or generate binding legal interpretations.

Operator design recommendations

Use a policy-and-obligation register with separate views for active rules, approval pending, obligations due, contracts nearing notice date and missing evidence. Rows show stable identifier, entity/country/product scope, owner, approved revision, effective dates, evidence completeness and next obligation. Filters distinguish published policy from draft, external contract from internal rule, legal hold from routine retention and customer acceptance requirement from staff procedure.

The detail drawer renders the signed contract/reference or approved policy revision beside structured obligations and linked workflows. Owners see notice dates, renewal terms, insurance/evidence expiry, data-processing commitments and the actual responsible task. A storage reference is not marked “signed evidence verified” until authorized verification succeeds. Sensitive documents require record-level access and safe previews; queue summaries avoid distributing private contract terms to every administrator.

Edits open a redlined before/after preview with merged dates, affected products, required reacceptance or training and proposed effective date. High-risk changes require a qualified independent approver. Retirement/termination lists open obligations and surviving commitments, then records a new lifecycle event; it cannot clear outstanding tasks automatically. Conflicts show the current approved version, while failed audit/evidence writes prevent ambiguous publication. The obligation calendar groups tasks by owner and due date with escalation and vacation coverage. Mobile leads with upcoming notice deadlines, not full contract text. Accessible revision comparisons, locale-aware dates and explicit timezone/working-day definitions keep worldwide operators from misreading a deadline.

Acceptance and implementation ownership

Legal Operations and Governance Engineering own this workspace. Test one-sided date patches, parallel edits, missing documents, expired evidence, renewal notice, terminated contracts with surviving duties and legal holds. Demonstrate a policy revision through approval, publication, required acceptance, operational enforcement and retained historical explanation. Run a cross-country obligation handoff with an unavailable owner and proof of escalation. Compliance readiness requires qualified legal signoff plus executed controls, not a successful registry CRUD test.

Proposed workspace 1: Workforce Recertification and Continuity

Why this management capability needs a dedicated view

Admin Accounts, Permissions, Corporate Setup and Workforce already contain pieces of identity management. What is not established is a complete enterprise review campaign linking all those authority sources to employment, sponsor accountability and continuity of owned work. This proposed view extends those services; it is not a new employee identity database. At 3,500 employees, spreadsheet reviews and personal knowledge cannot reliably handle transfers, long leave, departing approvers and overlapping acting roles. The business needs an answer to “who is still authorized, why, and who covers the work if they leave?”

Workflow and data contract

A campaign freezes an authorized evidence snapshot: identity, employment status, sponsor, effective authority by source, last use, owned approvals/cases and expiry. Review tasks are assigned to qualified independent reviewers with deadlines, reminder/escalation and explicit keep/reduce/revoke decisions. Changes execute through existing grant/assignment services; a review conclusion is not itself an access mutation. A reconciliation step verifies that actual authority now matches the decision across API, exports and realtime. Departure triggers immediate revocation separately from continuity handoffs, which remain visible until resolved.

Persist campaign ID, snapshot version, authority edge IDs, reviewer authority, decision reason, implementation command and verification outcome. Never rebuild permissions from display labels. Limit reviewer visibility to their scope and mask sensitive use details unless justified. Support evidence retention/legal hold and time-bounded emergency coverage. Integration with an HR/identity provider needs explicit source-of-truth ownership and idempotent provisioning; failure to synchronize must generate a task, not silently preserve access indefinitely.

Operator design recommendations

The command center has campaign overview, reviewer inbox and continuity exceptions. Overview shows review coverage, high-risk unreviewed authority, overdue decisions, execution failed and verification incomplete as separate figures. It must not mark a campaign complete because every checkbox was clicked. Filters select country, department, sponsor, authority type, risk and due interval. Each task row identifies employee, effective role/scope, source grant count, last use, owner and unresolved obligations.

The reviewer drawer explains authority provenance and the business responsibilities it supports. It shows direct grants, delegated grants, corporate assignments and internal gates separately, including conflicts and explicit denials. “Reduce” offers a reviewed diff to existing role bundles; it does not invent a free-text permission. High-risk self-review is disabled with an alternate reviewer path. The continuity section lists cases, financial approvals, customer obligations and scheduled work needing named successors, with acceptance by the receiving employee.

Submitting a decision opens a preview of exact authority edges and handoff objects. A stale snapshot requires refreshed comparison. Revoke can proceed immediately for a leaver even if a task handoff fails; the remaining handoff is escalated with a deadline. Verification panels name each downstream target and evidence time, never aggregate partial revocation into a green success. Campaign exports are scoped asynchronous jobs with retention and reveal logging. On mobile, the employee's risk, decision and unresolved continuity tasks stay visible above long permission lists. Keyboard and screen-reader workflows identify the person, edge and final outcome unambiguously.

Acceptance and business ownership

People Operations and Security jointly own campaigns; department heads own business need. Run a seeded 3,500-person campaign including cross-country reviewer scope, self-review denial, sponsor termination, mid-campaign transfer and an unavailable downstream system. Prove authority changes are executed once, old decisions cannot overwrite newer roles and completion waits for verification. Demonstrate a departing finance approver's work handoff without transferring credentials or widening their successor's money authority. Closure evidence is a completed campaign and offboarding drill, not a new navigation link.

Proposed workspace 2: Financial Reconciliation Exceptions

Why this management capability needs a dedicated view

Money, Credit, Restaurant Credit, Payouts and Treasury already own financial data and parts of reconciliation. The missing enterprise operating contract is one scoped exception queue linking unresolved source outcomes, customer exposure, ownership and approved recovery across those products. It must not merge OxSteed Credit with Restaurant Credit or introduce a new balance ledger. At six million users, occasional partial webhooks, duplicate callbacks, settlement delays and unknown provider outcomes become daily operational work. The business needs to recover safely without repeated manual charges or an operator mentally joining several dashboards.

Workflow and data contract

Existing financial workers create exception references carrying domain/source IDs, expected versus observed outcome, amount/currency, reconciliation window, age, severity and correlated event/command. One case can link several records but each authoritative service decides its own mutations. Classify delayed, missing, duplicate, inconsistent and unverifiable outcomes separately. Investigator assignment, evidence refresh, proposed remedy, independent approval, execution and verification are durable states. “Retry webhook,” “refresh provider evidence” and “refund” must be different actions with different authority.

Require provider/domain idempotency, unique source references and explicit limits. A retry after timeout queries the existing command/provider status first. Reconciliation can reopen after later contradictory evidence, preserving its prior decision. Use materiality policy approved by finance, separate currencies and privacy-scoped exports. Do not auto-write a balancing credit simply because a provider read failed. Connect support/customer notices only to confirmed customer-visible facts.

Operator design recommendations

Default the workspace to unresolved customer-impacting exceptions with oldest/material exposure first. Columns show exception ID, domain, legal entity, currency/minor amount, source completeness, expected/observed state, age, owner and safe next investigation action. Filters include provider, settlement window, missing webhook, repeated command, unknown outcome, approved recovery and customer complaint. Counts and amounts are grouped by currency; “evidence unavailable” is separate from zero discrepancy.

The case drawer displays a joined chronological timeline of provider events, domain commands, ledger entries, receipts and reconciliation runs without copying them into a new ledger. Each item links to its authoritative record with fresh access checks. Evidence timestamps identify stale provider reads. A comparison panel explains what is inconsistent and what is merely delayed. Customer notes remain separate from internal financial investigations and are revealed only when needed.

Recovery preview names the exact service action, source IDs, idempotency reference, amount/currency, ledger effect and required independent approver. A duplicate-safe retry returns its earlier outcome; an uncertain external result disables another financial execution while status refresh remains available. Approval cannot be performed by the requester where separation rules apply. Verification checks settlement/ledger/receipt consistency before closure, and leaves follow-up support tasks visible. Mobile prioritizes exposure, evidence freshness and the approved action; keyboard order follows investigation, proposal, approval and verification. Asynchronous export includes the predicate and as-of evidence time, not an unbounded download of every customer transaction.

Acceptance and business ownership

Financial Operations owns the queue; Payments/Credit Engineering retain mutation authority. Test duplicate/out-of-order webhooks, missing settlements, provider read timeout, charge timeout, concurrent investigators and approved refund retries. Prove one command causes at most one money movement, currencies stay separate and unauthorized regional staff cannot read another scope. Close a customer complaint using a verified reconciliation outcome and retained receipt. SLOs measure oldest unresolved exposure and verified recovery, not merely worker success rate.

Proposed workspace 3: Country Launch Readiness and Obligations

Why this management capability needs a dedicated view

Country Packs is configuration; Governance holds policies/contracts; Legal and payment integrations provide additional evidence. The business still needs a coordinated launch approval linking those facts per product and jurisdiction. This proposed workspace is the review/obligation view over those systems, not a new legal rules engine. Worldwide expansion cannot be certified by an active country row, translated navigation or a provider API key. A country may support one product but not delivery, payouts or credit; launch scope must reflect that distinction.

Workflow and data contract

Represent country-product launches as draft, evidence gathering, specialist review, approved bounded launch, active, restricted and suspended/draining. Each prerequisite has owner, evidence reference/revision, effective dates, expiry and reviewer decision: legal entity/agreements, payment and refund support, identity/insurance, privacy/residency, tax/invoicing, support hours/languages and actual operational coverage. Qualified specialists set requirements; engineering does not infer legal approval from a generic questionnaire. Publication changes existing launch configuration through its authorized service and verifies customer routes/offers afterwards.

Obligations continue after launch: insurance expiry, contract notice, policy update, provider restriction, locale/support coverage and mandated reports. Escalate missed evidence before expiry and record a proportionate response. Suspension stops specified new actions while supporting old commitments. Evidence retention and access follow legal/financial policies. Versions bind launch decisions to the real product configuration; unrelated country defaults cannot rewrite historic transactions.

Operator design recommendations

Provide a matrix of country by product with readiness, authorized launch scope, next deadline and accountable market owner. Tabs separate pending launches, active obligations and restrictions/draining commitments. Filters include missing specialist signoff, evidence expiring, provider capability unavailable and unsupported customer language. Status names explain whether a product is unavailable, coming soon, restricted or active; a single green country flag is never the summary.

The readiness drawer renders each prerequisite with source owner, independent reviewer, evidence revision, expiry and test outcome. A journey panel links representative test-mode intake, payment/refund, fulfillment and support evidence for that country/product. “Configured” and “successfully executed” appear as distinct states. An obligations calendar groups due tasks by country, working calendar and owner, with timezone and absence coverage explicit. Sensitive contracts remain behind their record-level access controls.

Approve launch previews exact public offers/routes, capacity bounds and rollback/suspension behavior. It lists unresolved limitations requiring explicit business acceptance and blocks mandatory gaps. A policy/provider restriction opens an impact queue of existing obligations and the approved new-intake limitation. The system must not cancel orders or redistribute money as a side effect of a launch-status edit. Conflicting readiness evidence forces a refreshed approval decision. During evidence outage show the last verified state and freeze unsafe publication. Mobile leads with due signoffs and expiring obligations; accessible textual grids work without map or flag interpretation and correctly isolate ISO codes within RTL layouts.

Acceptance and business ownership

The market owner coordinates Legal, Finance, Risk, Support and Operations signoff. Test a country with only food enabled, provider loss after launch, expiring insurance, absent reviewer and a language gap. Demonstrate launch approval, configuration publication, offer verification, restriction and continued resolution of historic refunds/support. Use one real specialist-approved country/product package as evidence. Do not label the business globally compliant without jurisdiction-specific review and ongoing control execution.

Proposed workspace 4: Recovery and Release Certification

Why this management capability needs a dedicated view

Reliability, backups, provider health and CI already supply pieces of evidence. What is not established is a single release/recovery certification workflow binding them to the exact artifact and to a business-usability decision. This proposed view extends Reliability rather than adding another manual release list. With a production Mac as a shared failure domain, a green uptime card cannot prove survival of host loss, corrupted data, expired provider access or an interrupted migration. The company needs an executed recovery drill that a second operator can reproduce.

Workflow and data contract

A certification run binds source revision, immutable artifact, migration checksums, tested configuration capabilities, CI coverage, scoped authenticated journeys, restore object/profile, load/failure evidence and approver. Record required, passed, failed, skipped and insufficient checks explicitly. Promotion requires applicable critical gates, with emergency exception decision/expiry and later review. Recovery uses an isolated target, private offsite retrieval, integrity checks, database/media/configuration restoration, invariant validation and representative business journeys before timing completion. No live charge is necessary.

Use additive compatibility across old/new workers and clear forward-recovery procedures. Separate backup age/RPO, offsite confirmation and usable-service RTO. Rehearse a populated upgrade, not merely two applications of migrations on an empty database. A disaster drill may intentionally fail but preserves its evidence and assigned remedy. The workflow verifies infrastructure observations; a self-reported checkbox is not equivalent to a measured test.

Operator design recommendations

The screen presents certification runs with artifact, environment, critical coverage, deployment identity, restore verification, measured recovery and final decision. Filters separate release candidates, verification incomplete, recovery overdue, failed drills and accepted temporary limitations. The header compares running identity to the approved artifact and labels missing metadata. A test that executed zero required scenarios is insufficient, not green because latency calculations returned no errors.

The run drawer lists mandatory evidence in dependency order: build, migration compatibility, provider capability, authenticated journeys, restored invariants, failure drills and operational signoff. Each result links to its actual run and names execution scope/time. Coverage includes denied/stale/concurrent paths, not only happy-path page loads. An operator can open unresolved findings and their closure tests directly; copying an old certificate to a new revision is forbidden.

The recovery wizard pins the chosen offsite object and isolated target, previews restore scope, and asks for explicit safety confirmation before any destructive target operation. Stages show verified checksum, restore, upgrade, workers, media and business checks. RTO starts at incident declaration and ends at usable verified service. A rollback panel explains non-reversible committed-domain effects and forward remedies. Emergency release requires named authority, reason and expiry, displayed prominently until review closes. Mobile puts an active drill/rollout and stop/escalation controls before charts. Screen readers receive stage transitions and scope warnings; no animation is required to understand progress.

Acceptance and business ownership

SRE/Continuity own execution; accountable business owners accept residual risk. Demonstrate one failed required gate blocking promotion, skipped authenticated coverage classified insufficient, wrong digest detection, production-sized upgrade and independent offsite recovery. Verify financial and inventory invariants and resume background work without duplicates. Have a second employee execute the runbook without relying on the first employee's memory. A successful certificate is revision/environment/time-specific and expires under the approved review cadence.

Proposed workspace 5: Cross-product Incident and Customer Impact

Why this management capability needs a dedicated view

Incident management, Support, Provider Health and Service Operations already handle portions of incidents. The enterprise need is a cross-product command view connecting measured customer impact, incident ownership, mitigations, communications and recovery verification. This is a coordination layer over existing incident/support records, not a parallel ticket system. At six million users, a provider outage can affect subscriptions, orders, delivery, messages and payout confidence at once. Separate teams must not issue contradictory status or retry financial work blindly.

Workflow and data contract

An incident links source alerts, affected capabilities/scopes, confirmed impact queries, commander, workstream owners, severity and communication cadence. Establish declared, investigating, mitigation proposed/approved, recovering, verified resolved and review states. Customer-impact sets are authorized, versioned predicates/IDs; estimates and confirmed records are distinguished. Mitigation commands use domain services with explicit scope and rollback/reconciliation plans. Customer notification uses existing localized infrastructure and approved factual templates, with per-recipient deduplication and consent handling.

Handoffs record outgoing/incoming commander acknowledgement and outstanding actions. Preserve timeline, evidence, unknown provider outcomes and decisions. Recovery verifies affected business journeys and queue health, not merely HTTP readiness. Post-incident actions become assigned engineering/runbook tasks with closure evidence; deleting an incident cannot clear obligations. Where safety, privacy or legal reporting is implicated, specialist escalation governs communications and retention.

Operator design recommendations

The command view shows declared severity, commander, affected products/regions, confirmed versus estimated customer impact, current mitigation and next communication deadline. Dedicated queues track unowned workstreams, blocked mitigations, provider outcome unknown, support escalation and recovery verification. Filters stay incident-scoped so a responder does not accidentally apply an action to the whole platform. A customer count includes measurement time and query completeness; missing telemetry is not zero affected people.

The incident timeline separates observations, proposals, approved actions, committed results and customer notices. Workstream cards identify responsible team, due time, dependency and recovery check. Provider Health links show probe scope versus actual transaction impact. The affected-record panel uses existing authorization, masked summaries and domain links; it does not expose every user's private messages to the incident room. Staff can inspect a representative failure and its correlation chain without dumping all payloads.

Mitigation preview names exact records/scope, authority, anticipated business effect and reconciliation. Replay is distinct from refund, cancel or suspend. Unknown external results block repeated money movement while evidence refresh remains possible. Communication preview shows localized copy, factual basis, affected audience and deduplication reference; internal routing details stay out of routine customer updates. Commander handoff is acknowledged explicitly, with next update time retained. Resolution requires journey/queue checks and lists residual support obligations. Mobile places commander, urgent tasks and communication deadline first; keyboard-accessible timelines and text severity labels support long shifts and RTL use without color dependence.

Acceptance and business ownership

Incident Command, Support and domain owners jointly operate this view. Run a test-mode provider outage affecting food and delivery, with lost webhook, stale realtime screen and customer escalation. Verify one consistent approved notice per recipient, no duplicate financial action, safe scoped mitigation and acknowledged shift handoff. Restore service and prove representative journeys plus reconciliation before resolution. Retain a post-incident action whose completion requires a real regression test, not only a written apology.

Current-source companion review: Provider Health

Route and business purpose

/admin/operations/providers is a cross-provider incident investigation view. It should tell a commander which customer capability is impaired, whether the fact is measured, the accountable responder and what safe recovery is possible. It is not a vendor billing dashboard or proof that a valid configuration can execute every business operation.

Verified implementation and findings

The registry defines capabilities, failure modes, owners, alert channels, retry/circuit policies, budget notes and credential-rotation procedure keys (server/services/providerHealthRegistry.js:3-31). Routes require admin authentication and manage_operations (server/routes/providerHealth.js:14-28). The UI renders provider cards and a two-click circuit reset; the controller records reset audit metadata (client/src/admin/pages/ProviderHealthPage.jsx:53-68; server/controllers/providerHealthController.js:63-80). Probe results cache for sixty seconds and definitions distinguish unknown from not configured (server/services/providerHealthRegistry.js:39-40, :546-608). These are real improvements, not an absent integration-management surface.

CONFIRMED DEFECT, P1 - misleading breaker outcomes. probeProvider executes a probe through the breaker, but provider probes frequently catch errors and return {status:'down'} normally (providerHealthRegistry.js:89-105, :563-568). providerCircuitBreaker.js:177-183 records any normally returned value as success. A down result can therefore reset failure history rather than open the circuit. Define which outcomes count as failures, with unknown/not-configured neutral, and test repeated down results and half-open trials.

DESIGN GAP, P1 - call-site adoption is incomplete. Source search found the new registry outcome APIs in its registry, controller and tests but not in the business provider callers. The advertised policies are mostly registry metadata until those callers adopt them. Inventory actual Stripe/maps/email/AI/calendar/tracker calls and bind capability-specific measurements without changing financial idempotency. Probe health and real transaction health must remain distinguishable.

CONFIRMED DEFECT, P2 - reset failures disappear. ProviderHealthPage.jsx:217-225 catches and discards a failed reset; no error is placed in state and reload happens only after success. The comment promising a later reload error does not report the failed mutation. Also, reset happens before audit write (providerHealthController.js:67-79): an audit failure may return an error after a state change. Show an explicit unresolved outcome and maintain a durable decision reference instead of prompting blind retries.

DESIGN GAP, P2 - hardcoded dark styling and weak triage. The screen uses dark-only colors (ProviderHealthPage.jsx:71, :230) and a long card wall without a capability/owner search or incident handoff. Budgets are declared notes, not measured spend (providerHealthRegistry.js:17-19). A default closed circuit does not establish that customer calls are working.

Operator screen design recommendations

Use a compact status bar for measured down capabilities, degraded capabilities, unknown observations and disconnected configuration. Put a definition and observation source behind each count. The header should show the production environment, last successful aggregation and a controlled “Refresh observations” action; distinguish registry refresh, fresh probe and real-call sample time.

Replace the card wall's first screen with a table: capability/provider, customer impact, observed state, probe age, actual-call success/error rate, open circuit, oldest pending retry, on-call owner and linked incident. Filter by affected product, owner, severity, measured versus configuration-only evidence and stale samples. Saved views should include “Checkout unavailable,” “Queued communications,” and “No recent real traffic.” Do not expose credential values; dependency names are enough.

Selecting a row opens tabs for Impact, Evidence, Recovery and History. Impact lists affected customer workflows and safe fallback copy. Evidence shows probe versus real-call measurements and redacted correlation references. Recovery shows the existing runbook, retry deadline and authorized circuit action. History shows resets, approval/reason, before/after and incident linkage. A probe is a diagnostic action with a vendor-budget limit, not unlimited polling by every staff member.

Before resetting a circuit, preview provider/capability, current failure evidence, incident owner, likely retried workload and any financial restrictions. Require a reason, latest circuit version and successful recovery prerequisites; never include payment capture in a reset. Display reset success, refusal or unknown outcome explicitly and read back state with the decision reference. On failure retain evidence and permit

source-specific retry. A configuration-only row says “not yet measured,” not healthy. Mobile prioritizes impact and owner; theme tokens, text labels and keyboard focus make the recovery workflow usable in light mode and Arabic.

Expansion scenario, acceptance and owner

A vendor outage produces down probes while actual calls continue without the registry breaker. Multiple operators reset it, one audit write fails, and no UI reports failure. SRE Integrations owns recovery semantics; each domain lead owns its capability caller. Test repeated down returns opening the appropriate circuit, unknown results not recording success, cross-node half-open trial concurrency, failed reset/audit outcomes visible, permission denial, actual caller instrumentation and no duplicate capture. Verify one incident can track all affected capabilities without exposing private messages or keys. Live provider verification remains separate from these proposed tests.

Current-source companion review: Media Safety and Processing Operations

Business purpose and existing foundation

Restaurant media now includes device-uploaded photos and videos, dish anchors, public feeds, flags, safe quarantine and a transcode worker. A separate media-safety queue is a legitimate missing operator capability, not a reason to build another upload service. Staff must distinguish content reported by users, material blocked by a safety verdict, content waiting for processing, and legal takedowns. Those decisions need different authority and evidence.

The protected API exists at /api/admin/restaurant-media/flags, gated by moderate_content (server/routes/adminRestaurantMedia.js:16-29). Normal removal transactionally hides media and resolves its open flags (server/controllers/restaurantMediaController.js:873-910); the specialized DMCA path also records an audit action (server/controllers/restaurantMediaVideoController.js:534-595). Video retry cannot clear a flagged safety verdict (:447-451). The worker's claim uses FOR UPDATE … SKIP LOCKED and transcode eligibility requires a passed safety gate (server/services/videoTranscodeWorker.js:153-175). These protections should be retained.

Concrete gaps and expansion failures

DESIGN GAP, P1 - unreachable review work. Search of the current client sources found no consumer of the restaurant-media flag list/review endpoints and no corresponding page in the admin route manifest. The API is not a workable daily queue for a moderation employee. The list returns the newest 200 flags without pagination metadata (restaurantMediaController.js:843-860); older unresolved flags become inaccessible through that list at scale. Add bounded cursor pagination, total/age evidence and consistent filtering rather than loading everything into the browser.

CONFIRMED DEFECT, P1 - claimed jobs can remain stranded after worker loss. Claiming writes running; normal polling selects only queued, and explicit by-ID processing refuses a running record (videoTranscodeWorker.js:162, :172, :384-390). The inspected queue schema and all current queue consumers contain no reclaim lease/heartbeat path. A process dying after claim leaves an item unable to proceed through these paths. Add durable lease owner/expiry, fencing and safe re-entry. Test interruption between each storage/database commit boundary; a completed media row and a still-running job also require reconciliation.

DESIGN GAP, P1 - ordinary review evidence is incomplete. The normal review transaction records the deciding user/time on flags but has no logAdminAction, reviewer reason or evidence reference (restaurantMediaController.js:889-910). The DMCA path has richer audit handling. Unify immutable decision evidence without rewriting prior decisions or treating all removals as legal takedowns.

UNVERIFIED, P1 - production media capability. Current readiness does not prove a configured safety vendor, enabled worker, usable ffmpeg/ffprobe or shared source-file access. The final CI repaired the earlier runner dependency failure and now passes; this does not verify the production worker. The worker resolves originals through a local-storage path (videoTranscodeWorker.js:305-307). Before distributing workers, declare shared-storage access and fail safely when unavailable. New uploads must never publish merely because processing or the safety provider is unavailable.

Operator screen design recommendations

Add a discoverable “Media safety” workspace within Trust Operations, backed by the existing API plus additive queue/detail projections. Use three clear queues: Reported content, Quarantined processing, and Legal takedowns. Show market/jurisdiction scope and independent queue freshness in the header. KPIs should measure unresolved flags, oldest unresolved age, blocked uploads, jobs past lease/processing deadline and legal deadlines; none should be inferred from the newest 200 rows.

For flags, columns should include media ID, restaurant, content type, grouped report reasons, open report count, first-report age, assigned reviewer, public/hidden state and due time. For processing, show safety verdict, processing stage, source readability, active lease, attempts and last error class. Filter by

queue, region, language, report category, overdue age and owner; saved views include “Oldest unreviewed,” “Worker lost,” and “Awaiting vendor recovery.” Avoid automatic rendering of sensitive flagged content in the list.

The detail drawer shows a deliberately opened restricted preview, original content references, restaurant/dish linkage, flag history, safety-provider verdict, processing attempts and decision history. Protect access to severe-content evidence according to policy and record purpose. A takedown links the appropriate legal case and review deadline. Dismiss, remove and retry processing are separate actions. No reviewer or merchant may override a blocked safety verdict through a retry button.

A removal preview identifies the exact content, affected feed/product links, required reason and notice task. Retry previews only eligible failed/stale processing work and never promises publication. Bulk assignment is safe with an exact ID preview; bulk dismissal or safety release requires individual evidence and should not be initially exposed. On a competing review, return current decision and allow history inspection rather than re-deciding. Missing source data disables release, not investigation. Mobile uses text-first queues and opt-in evidence preview; keyboard focus, Arabic layout, theme tokens and no-autoplay defaults protect staff usability.

Acceptance and accountable ownership

Trust and Safety owns review policy; Media Engineering owns processing integrity; Legal Operations owns takedown deadlines. Seed more than 200 unresolved flags and prove the oldest is reachable. Execute two reviewers against one flag: one committed decision, one truthful conflict and one retained audit trail. Kill a claimed worker and prove safe lease recovery with one final rendition set and no accidental publication. Inject vendor, source-storage and ffmpeg failures: quarantine remains closed, error reason and owner are visible, and the authorized recovery path works. Verify removed or flagged media never returns through public feed, dish links, direct assets or merchant editing. Test evidence-purpose authorization without logging private media payloads.

Source coverage appendix

audit-v2-operations-review: Coverage and evidence boundary

Reviewed current source at e47531c76b320c8145e15415774d05d698b456d1, October 3, 2026, for a target operating model of 3,500 worldwide employees and six million registered users. Earlier sections were drafted at 893f1bc3; their relevant source diff was rechecked after the shared fast-forward. This is a source-based engineering and operator-workflow review, not complete certification, a production penetration test, or a measured capacity claim. Only this document is written. No credentials, production APIs, databases, financial transactions, deployment actions, or product changes are used.

The user reports earlier green CI runs, including 37110020289, followed by latest run 37147612778 failing and a healthy readiness response at 22:30 UTC with migrations 661/661 and null Git SHA. These operational assertions are attributed to the user, not independently queried here; the latest CI cause is being investigated separately. Earlier CI failure is historical. The Customer360 mock repair at 4fe5470d, localization debt gates at 25790585, and process-state registry/Redis fencing at 893f1bc3 must not be described as unchanged defects. The isolated scale harness added at 7c64513f exists; measured global capacity remains unverified. Existing tests and migrations are inspected as source evidence; unless an execution result is explicitly recorded, their presence does not mean they were rerun or that migrations are applied in production. The prior docs/admin-super-admin-full-audit-2026-10-02.md supplies baseline questions only. Findings below are checked against current paths. Repository HEAD includes migrations 663-666; reported readiness 661/661 does not establish that these new features are deployed.

Citation convention: path:line points to the current checkout, relative to /Users/oxsteed/Documents/GitHub/6. CONFIRMED DEFECT identifies a demonstrable code/contract inconsistency, not observed exploitation. DESIGN GAP identifies missing operational capability in the inspected workflow. UNVERIFIED identifies a hypothesis requiring isolated or operational evidence. Proposed endpoints and screen elements are explicitly recommendations, not claims that they already exist. P1 is an expansion blocker for the affected workflow; P2 is an important improvement with a committed owner.

Tab	Actual route	Page source	API/router and principal implementation
1 Dashboard	/admin/dashboard	client/src/admin/pages/super/Dashboard.jsx	server/routes/admin.js; server/controllers/adminController.js; super variant server/controllers/superAdminController.js
2 Operations Overview	/admin/operations	client/src/admin/pages/OperationsOverview.jsx	GET /api/admin/operations/overview; server/controllers/adminOperationsController.js
3 Users	/admin/users	client/src/admin/pages/UsersList.jsx	/api/admin/users; server/controllers/superAdminController.js
4 Jobs	/admin/jobs	client/src/admin/pages/JobsList.jsx; JobDetail.jsx	/api/admin/jobs; server/controllers/superAdminController.js
5 Reports	/admin/reports	client/src/admin/pages/ReportsList.jsx	/api/admin/reports; server/controllers/adminController.js
6 Moderation	/admin/moderation	client/src/admin/pages/Moderation.jsx	/api/admin/moderation-queue; /api/admin/jobs/:jobId/action; server/controllers/adminController.js and superAdminController.js
7 Enforcement	/admin/enforcement	client/src/admin/pages/AdminEnforcementDashboard.jsx	server/routes/enforcement.js; server/controllers/enforcementController.js
8 Content Removals	/admin/content-removals	client/src/admin/pages/ContentRemovals.jsx	/api/admin/content, bid/review remove/restore; server/controllers/adminController.js
9 Skills	/admin/skills	client/src/admin/pages/SkillsManager.jsx	/api/admin/skills-lookup; inline handlers in server/routes/admin.js
10 Support Tickets	/admin/support	client/src/admin/pages/SupportTickets.jsx	/api/admin/support/tickets; server/controllers/supportController.js

Tab	Actual route	Page source	API/router and principal implementation
11 Live Support	/admin/support/chat	client/src/admin/pages/SupportAgentChat.jsx; client/src/components/support/SupportConversationWorkspace.jsx	server/routes/support.js; server/controllers/supportConversationsController.js; conversationAssignmentController.js
12 Quote Requests	/admin/quote-requests	client/src/admin/pages/QuoteRequestsInbox.jsx	server/routes/quoteRequests.js; server/controllers/quoteRequestsController.js
13 Direct Ops	/direct/admin, with dashboard/inbox/calendar/job/payment/customer subroutes	client/src/pages/DirectAdminPortalPage.jsx; client/src/App.jsx	server/routes/direct.js; server/controllers/directController.js; quoteRequestsController.js
14 Direct Service Areas	/admin/direct-service-areas	client/src/admin/pages/DirectServiceAreas.jsx	/api/direct/admin/service-areas; server/services/directServiceAreas.js
15 Direct Catalog	/admin/direct-catalog; alternate /direct/admin/catalog	client/src/admin/pages/DirectCatalog.jsx	/api/direct/admin/catalog; server/services/directCatalogService.js; directCapability.js; directSlotEngine.js
16 Messages	/admin/messages	client/src/admin/pages/AdminMessages.jsx	/api/admin/messages/conversations; server/controllers/adminMessageController.js
17 Search	/admin/search	client/src/admin/pages/AdminSearch.jsx	/api/admin/search; server/controllers/adminSearchController.js

The shared route contract starts at client/src/admin/adminRouteManifest.js:21; actual admin mounting is client/src/admin/AdminApp.jsx:230-258. Server admin authentication already verifies current active role, token revocation, and mandatory TOTP (server/middleware/adminAuth.js:43-80). Current permission middleware rejects region-only grants on global endpoints and verifies delegated grantor authority (server/middleware/adminAuth.js:126-188). This prevents regional grants becoming global access; it also means regional operations remain a product-design requirement rather than an existing scoped version of every page.

audit-v2-commerce-review: Coverage and actual routes

Routes are verified in client/src/admin/adminRouteManifest.js:41-58; the additional approvals route is at :112. API mounts are in server/index.js; each review names its concrete router/controller paths.

Tab	Actual browser route	Current page source	API route and implementation files
18 Businesses	/admin/businesses	client/src/admin/pages/BusinessModeration.jsx	server/routes/admin.js; server/controllers/businessModerationController.js
19 Tool Rentals	/admin/tool-rentals	client/src/admin/pages/ToolRentalModeration.jsx	server/routes/admin.js; server/controllers/businessModerationController.js; owner path server/routes/toolRentals.js
20 Planned Needs	/admin/planned-needs	client/src/admin/pages/PlannedNeedsAdmin.jsx	server/routes/admin.js; server/controllers/businessModerationController.js; server/controllers/plannedNeedsController.js; server/jobs/plannedNeedsScheduler.js
21 Payments	/admin/payments	client/src/admin/pages/PaymentsList.jsx	server/routes/admin.js; server/controllers/paymentsAdminController.js; server/routes/payments.js; server/services/paymentRefundService.js
22 Orders	/admin/orders	client/src/admin/pages/OrderManagement.jsx; client/src/admin/components/OrderDetailModal.jsx	server/routes/adminOrders.js; server/controllers/adminOrdersController.js; server/services/marketplaceRefundService.js
23 Templates	/admin/templates	client/src/admin/pages/TemplateManagement.jsx	server/routes/templates.js; server/controllers/templateController.js
24 Consulting	/admin/consulting	client/src/admin/pages/ConsultingDashboard.jsx	server/routes/consultingAdmin.js; server/controllers/adminConsultingController.js

Tab	Actual browser route	Current page source	API route and implementation files
25 Consulting Pay	/admin/consulting/payments	client/src/admin/pages/ConsultingPayments.jsx	server/routes/consultingAdmin.js; server/routes/consultingPayment.js; server/controllers/adminConsultingController.js; server/controllers/consultingPaymentController.js; server/services/externalRefundService.js
26 Scheduling	/admin/consulting/scheduler	client/src/admin/pages/ConsultingScheduler.jsx	server/routes/consultingAdmin.js; server/routes/consultantScheduling.js; server/controllers/adminConsultingController.js; server/controllers/consultantSchedulingController.js
27 Business Launch	/admin/consulting/business-launch	client/src/admin/pages/BusinessLaunchAdmin.jsx; client/src/admin/pages/BusinessLaunchRequestDetail.jsx	server/routes/businessLaunch.js; server/controllers/businessLaunchController.js; server/controllers/businessLaunchNotesController.js
28 Consulting Sessions	/admin/consulting/sessions	client/src/admin/pages/ConsultingSessionsAdmin.jsx; client/src/admin/pages/ConsultingSessionDetail.jsx	server/routes/consultingAdmin.js; server/controllers/adminConsultingController.js
29 Org Chart	/admin/corporate/org-chart	client/src/admin/pages/super/CorporateOrgChart.jsx	server/routes/corporateRoles.js; server/controllers/corporateRolesController.js; server/services/corporateRolesService.js
30 Corporate Titles	/admin/corporate/assignments	client/src/admin/pages/super/CorporateAssignments.jsx	server/routes/corporateRoles.js; server/controllers/corporateRolesController.js; server/services/corporateRolesService.js
31 Logistics Ops	/admin/logistics	client/src/admin/pages/logistics/LogisticsAdminDashboard.jsx	server/routes/logisticsAdmin.js; server/controllers/logisticsAdminController.js; server/middleware/requireInternalLogisticsWorkspace.js; server/services/logisticsAccessService.js; server/services/logisticsStateMachine.js
32 Learn Submissions	/admin/learn/submissions	client/src/admin/pages/LearnSubmissionsAdmin.jsx	server/routes/learn.js; server/controllers/learnController.js (including publishDueGuideVideos)
33 Learn Product Links	/admin/learn/product-links	client/src/admin/pages/LearnProductLinksAdmin.jsx	server/routes/learn.js; server/controllers/learnController.js; server/services/learnTagging.js
34 Learn AI Suggestions	/admin/learn/ai-suggestions	client/src/admin/pages/LearnAiSuggestionsAdmin.jsx	server/routes/learn.js; server/controllers/learnController.js; server/services/learnTagging.js
Additional finance scope: Restaurant Top-up Approvals	/admin/super/restaurant-topups	client/src/admin/pages/super/RestaurantTopupApprovals.jsx	server/routes/adminRestaurantTopups.js; server/controllers/adminRestaurantTopupController.js; server/services/restaurantTopupService.js; customer route server/routes/restaurantTopups.js

audit-v2-finance-access-review: Coverage table

Route contracts: client/src/admin/adminRouteManifest.js:109-143; component mounts are in client/src/admin/AdminApp.jsx (the added top-up route is at :325-326). API paths below are relative to /api.

Tab	Actual browser route	Current page file	Principal API/controller/service files
35 Super Dashboard	/admin/super/dashboard	client/src/admin/pages/super/Dashboard.jsx	server/routes/admin.js; server/controllers/superAdminController.js
36 Money	/admin/super/money	client/src/admin/pages/super/MoneyPage.jsx	server/routes/admin.js; server/controllers/superAdminController.js

Tab	Actual browser route	Current page file	Principal API/controller/service files
37 Restaurant Credit	/admin/super/restaurant-credit	client/src/admin/pages/super/RestaurantCreditAdmin.jsx	server/controllers/restaurantCreditAdminController.js; server/services/foodWalletService.js; server/services/foodWalletOperationsService.js
Additional Restaurant Top-up Approvals	/admin/super/restaurant-topups	client/src/admin/pages/super/RestaurantTopupApprovals.jsx	server/routes/adminRestaurantTopups.js; server/controllers/adminRestaurantTopupController.js; server/services/restaurantTopupService.js
38 Credit overview	/admin/super/oxsteed-credit	client/src/admin/pages/super/oxsteedCredit/OverviewPage.jsx	server/controllers/oxsteedCreditController.js; server/services/oxsteedCreditService.js
38 Approvals	/admin/super/oxsteed-credit/approvals	client/src/admin/pages/super/oxsteedCredit/ApprovalsPage.jsx	server/services/oxsteedCreditPendingActionService.js; server/controllers/agentCreditAdjustmentController.js
38 Disputes	/admin/super/oxsteed-credit/disputes	client/src/admin/pages/super/oxsteedCredit/DisputesPage.jsx	server/services/oxsteedCreditDisputeService.js
38 Settlements	/admin/super/oxsteed-credit/settlements	client/src/admin/pages/super/oxsteedCredit/SettlementsPage.jsx	server/services/oxsteedCreditAdminService.js; server/services/oxsteedCreditPendingActionService.js
38 Reconciliation	/admin/super/oxsteed-credit/reconciliation	client/src/admin/pages/super/oxsteedCredit/ReconciliationPage.jsx	server/services/oxsteedCreditExternalReconciliation.js
38 Compliance	/admin/super/oxsteed-credit/compliance	client/src/admin/pages/super/oxsteedCredit/CompliancePage.jsx	server/services/complianceCaseService.js
38 Reserves	/admin/super/oxsteed-credit/reserves	client/src/admin/pages/super/oxsteedCredit/ReservesPage.jsx	server/services/oxsteedCreditReserveService.js
38 Partners	/admin/super/oxsteed-credit/partners	client/src/admin/pages/super/oxsteedCredit/PartnersPage.jsx	server/services/oxsteedCreditAdminService.js; server/services/oxsteedCreditPartnerLedgerService.js
38 Health	/admin/super/oxsteed-credit/health	client/src/admin/pages/super/oxsteedCredit/HealthPage.jsx	server/services/oxsteedCreditWorkerHeartbeat.js; server/services/oxsteedCreditExternalReconciliation.js
38 Audit	/admin/super/oxsteed-credit/audit	client/src/admin/pages/super/oxsteedCredit/AuditPage.jsx	server/controllers/oxsteedCreditController.js; server/services/oxsteedCreditAdminService.js
38 Customers	/admin/super/oxsteed-credit/customers	client/src/admin/pages/super/oxsteedCredit/CustomersPage.jsx	client/src/admin/pages/super/OxsteedCreditCustomerDetail.jsx; server/services/oxsteedCreditService.js
38 Money Out	/admin/super/oxsteed-credit/money-out	client/src/admin/pages/super/oxsteedCredit/MoneyOutPage.jsx; client/src/admin/pages/super/OxsteedCreditMoneyOut.jsx	server/services/oxsteedCreditWithdrawalService.js; server/services/oxsteedCreditFeeScheduleService.js
38 Cards	/admin/super/oxsteed-credit/cards	client/src/admin/pages/super/oxsteedCredit/CardsPage.jsx; client/src/admin/pages/super/OxsteedCreditCards.jsx	server/controllers/oxsteedCreditCardController.js; server/services/oxsteedCreditCardService.js
38 Ledger	/admin/super/oxsteed-credit/ledger	client/src/admin/pages/super/oxsteedCredit/LedgerPage.jsx	server/services/oxsteedCreditService.js; server/controllers/oxsteedCreditController.js

Tab	Actual browser route	Current page file	Principal API/controller/service files
39 Revenue	/admin/super/revenue	client/src/admin/pages/super/Revenue.jsx	server/controllers/superAdminController.js
40 Fee Config	/admin/super/fees	client/src/admin/pages/super/FeeConfig.jsx	server/routes/feeConfig.js; server/controllers/feeConfigController.js; server/utils/feeCalculator.js
41 Pricing Config	/admin/super/pricing	client/src/admin/pages/super/PricingConfig.jsx	server/routes/config.js; server/controllers/configController.js
42 Pricing Overrides	/admin/super/pricing-overrides	client/src/admin/pages/super/PricingOverrides.jsx	server/routes/pricingOverrides.js; server/controllers/pricingOverrideController.js; server/services/pricingService.js
43 Financials	/admin/super/financials	client/src/admin/pages/super/Financials.jsx	server/controllers/superAdminController.js
44 Payouts	/admin/super/payouts	client/src/admin/pages/super/Payouts.jsx	server/controllers/superAdminController.js; server/migrations/014_phase4_create_payments.sql
45 Conversations	/admin/super/conversations	client/src/admin/pages/super/ConversationsAdmin.jsx	server/controllers/conversationsAdminController.js; server/services/chatThreadAccess.js
46 Admin Accounts	/admin/super/admin-accounts	client/src/admin/pages/super/AdminAccounts.jsx	server/controllers/superAdminController.js; server/middleware/adminAuth.js
47 Permissions	/admin/super/permissions	client/src/admin/pages/super/PermissionGrants.jsx	server/controllers/adminPermissionController.js; server/middleware/adminAuth.js
48 Regions	/admin/super/regions	client/src/admin/pages/super/RegionsManager.jsx	server/controllers/adminRegionController.js; server/middleware/regionAuth.js
49 Internal Access	/admin/super/logistics-internal-access	client/src/admin/pages/super/LogisticsInternalAccess.jsx	server/controllers/logisticsInternalAccessController.js; server/middleware/requireInternalLogisticsWorkspace.js
50 Gate Lockouts	/admin/super/access-gate-lockouts	client/src/admin/pages/super/AccessGateLockouts.jsx	server/controllers/logisticsInternalAccessController.js; server/services/accessGateRateLimitService.js
51 Create User	/admin/super/users/create	client/src/admin/pages/super/CreateUser.jsx; client/src/components/CreateUserForm.jsx	server/controllers/superAdminUserController.js
52 Search Logs	/admin/super/search-logs	client/src/admin/pages/super/SearchLogs.jsx	server/controllers/adminSearchController.js; server/utils/encryption.js

audit-v2-governance-review: Coverage and actual entry points

Route contracts are in client/src/admin/adminRouteManifest.js:72, with corresponding guarded rendering in client/src/admin/AdminApp.jsx. The numbered scope is retained despite later navigation additions. Provider Health is main-owned; Restaurant Top-up Approvals is finance-owned and is not silently substituted for any assigned tab.

Assigned tab	Actual browser route	Current page	API/controller/service paths
53 Settings	/admin/super/settings	client/src/admin/pages/super/PlatformSettings.jsx	server/routes/admin.js, server/controllers/superAdminController.js, server/routes/settingsChanges.js, server/controllers/settingsChangeController.js
54 Audit Log	/admin/super/audit-log	client/src/admin/pages/super/AuditLog.jsx	server/routes/admin.js, server/controllers/superAdminController.js, server/services/auditService.js

Assigned tab	Actual browser route	Current page	API/controller/service paths
55 User Action Log	/admin/super/audit-log/user-actions	client/src/admin/pages/super/AuditLogUserActions.jsx	server/controllers/auditController.js, server/services/auditService.js
56 Legal, all routed subpages	/admin/legal/acceptances; /admin/legal/audit-log; /admin/legal/compliance-metrics; /admin/legal/ccpa-requests	client/src/admin/pages/AdminLegalAcceptancesPage.jsx, client/src/admin/pages/AdminLegalAuditLogPage.jsx, client/src/admin/pages/AdminComplianceMetricsPage.jsx, client/src/admin/pages/AdminCCPARequestsPage.jsx	server/routes/legalDocuments.js, server/controllers/legalDocumentsController.js, server/controllers/ccpaController.js
57 Visitors	/admin/super/visitors	client/src/admin/pages/VisitorHistoryPage.jsx	server/routes/siteVisits.js, server/controllers/siteVisitController.js, server/services/siteVisitService.js
58 Profile Visibility	/admin/super/profile-visibility	client/src/admin/pages/ProfileVisibilityPage.jsx	server/controllers/profileVisibilityController.js, server/services/profileVisibilityApprovals.js
59 One Truth	/admin/super/one-truth	client/src/admin/pages/super/OneTruthPage.jsx	server/routes/oneTruth.js, server/controllers/oneTruthController.js
60 Franchise HQ	/admin/super/franchises	client/src/admin/pages/super/FranchiseHQPage.jsx	server/routes/franchise.js, server/controllers/franchiseController.js, server/controllers/franchiseTerritoryController.js
61 Careers	/admin/super/careers	client/src/admin/pages/super/CareersAdmin.jsx, client/src/components/careers/HiringPipelineBoard.jsx	server/routes/careers.js, server/controllers/careerController.js
62 Country Packs	/admin/super/country-packs	client/src/admin/pages/super/CountryPacks.jsx	server/routes/countryConfig.js, server/controllers/countryPackController.js, server/services/countryPackService.js
63 Corporate Setup	/admin/super/corporate/departments; dependent /admin/corporate/org-chart, /admin/corporate/assignments	client/src/admin/pages/super/CorporateDepartments.jsx	server/routes/corporateRoles.js, server/controllers/corporateRolesController.js, server/services/corporateRolesService.js
64 Mode Policies	/admin/super/mode-policies	client/src/admin/pages/super/ModePolicies.jsx	server/routes/modePlatformApprovals.js, server/services/modePlatformApprovalService.js
65 Mode Approvals	/admin/super/mode-approvals	client/src/admin/pages/super/ModeApprovals.jsx	server/controllers/modePlatformApprovalController.js, server/services/modePlatformApprovalService.js
NEW Workforce	/admin/super/workforce	client/src/admin/pages/super/workspaces/WorkforcePage.jsx	server/routes/workforce.js, server/controllers/workforceController.js
NEW Treasury	/admin/super/treasury	client/src/admin/pages/super/workspaces/TreasuryPage.jsx	server/routes/treasury.js, server/controllers/treasuryController.js
NEW Service Operations	/admin/super/service-ops	client/src/admin/pages/super/workspaces/ServiceOpsPage.jsx	server/routes/serviceOps.js, server/controllers/serviceOpsController.js
NEW Reliability	/admin/super/reliability	client/src/admin/pages/super/workspaces/ReliabilityPage.jsx	server/routes/reliability.js, server/controllers/reliabilityController.js, server/controllers/opsActionsController.js
NEW Governance	/admin/super/governance	client/src/admin/pages/super/workspaces/GovernancePage.jsx	server/routes/governance.js, server/controllers/governanceController.js, server/services/investigationAccessService.js